

A Cost Criterion for Post-Quantum Signature Verification in a Zero-Knowledge Virtual Machine

A Thesis Submitted to the Department of Computer Science and Communications Engineering, the Graduate School of Fundamental Science and Engineering of Waseda University in Partial Fulfillment of the Requirements for the Degree of Master of Engineering

Takumi Otsuka

The Department of Computer Science and Communications Engineering, the
Graduate School of Fundamental Science and Engineering of Waseda University
5124FG15-6

Submission Date : July 20th, 2026

Research Guidance: : Research on Cryptographic Protocols and Blockchain

Advisor: : Prof. Kazue Sako

Abstract

Zero-knowledge proofs demonstrate that a computation is correct without revealing its private inputs. There are two types of zero-knowledge proof system. In a zkSNARK, what will be proved must be decided before the proof system is built. In contrast, in a zkVM, the computation to prove can be given as an input, so it is flexible. An anonymous credential lets a holder prove a fact about themselves while revealing nothing else. What a holder must prove can differ from one verifier to the next, so a zkVM is the more suitable choice. This flexibility has a cost. Showing a credential requires verifying a signature, and for BDEC, a post-quantum anonymous credential, one such verification did not finish within five hours on a personal laptop. In this thesis, we make a post-quantum anonymous credential run inside a zkVM on personal hardware. We give a cost criterion, proven correct, that decides before building whether a plain zkVM suffices or a custom accelerator, a precompile, is needed to speed up the costly operations of its main primitives. A precompile for the signature's hash brings one verification down to 14.24 minutes on average, at 80-bit security, since the full 128-bit level does not complete on this hardware. To our knowledge, this is the first post-quantum anonymous credential run inside a zkVM. The one limitation is that the zkVM studied in this thesis cannot make a single proof that is both post-quantum and zero-knowledge. Achieving both properties in one proof will remain as future work.

Contents

1	Introduction	8
1.1	Background	8
1.2	From Constructing to Deploying	8
1.3	The Precompile Suite, and the Question It Answers	8
1.4	Contributions of This Thesis	9
1.5	Thesis Organisation	11
2	Related Work	12
2.1	Constructing an Anonymous Credential Directly	12
2.2	Proving Signature Verification in a Static Circuit	13
2.3	Running a Signature Inside a zkVM	13
2.4	Accelerating a zkVM with a Precompile	14
2.5	The Security a zkVM Receipt Carries	14
2.6	The Gap and the Unreported Dimension	14
3	Preliminaries	16
3.1	Notation	16
3.2	Cryptographic Primitives	16
3.2.1	Power residue PRF	16
3.2.2	Algebraic hash function.	17
3.2.3	Merkle commitment.	18
3.3	Digital Signature Scheme.....	18
3.3.1	Loquat: a Legendre-PRF post-quantum signature with low R1CS verification cost.	20
3.3.2	PLUM: a power-residue-PRF post-quantum signature with low R1CS verification cost.	21
3.4	zkSNARK	23
3.4.1	Aurora: Transparent Succinct Arguments for R1CS	24
3.5	Anonymous Credential	25
3.5.1	Blockchain-based Digital Education Credentials (BDEC).....	27
3.6	Zero-Knowledge Virtual Machines	31
3.6.1	SP1	33
3.6.2	Static-circuit and zkVM proof systems as comparators	33
4	The Field-Mismatch Tax and the Cost Criterion	34
4.1	Cost Dimensions	34
4.2	The Field-Mismatch Tax.....	35
4.3	The Field-Matching Cost Criterion	38
4.4	Update Churn	39
4.5	Cost Decomposition of the Verification Predicate.....	41

5	The Precompile Suite and the Credential Workload	42
5.1	Scope of the Implementation	42
5.2	The Field-Arithmetic Substrate: $\mathbb{F}_{p^{192}}$ Multiplication via the Host Precompile	43
5.3	The Bespoke Griffin- $\mathbb{F}_p^{192}$ Permutation AIR (SP1)	44
5.4	The Sponge and Compression Wrapper	45
5.5	Soundness of the Precompile Suite, and Two Open Questions	45
5.6	Integration into the Credential Relations (BDEC Instance)	49
6	Security Analysis	50
6.1	Security Model and Threat Model	50
6.2	Security Preservation	59
6.3	The Dual Obstruction, Reduced to One Remaining Obstruction	69
7	Evaluation	78
7.1	Setup and Methodology	78
7.1.1	Settings Justification	79
7.1.2	Reproducibility	79
7.2	PLUM Verification With and Without the Precompile	80
7.3	Cost Attribution	82
7.4	The Credential Relations and the Consumer-Hardware Frontier	83
7.4.1	The post-quantum frontier	84
7.5	Calibrating the Cost Model	86
7.5.1	Validation against the measured points	86
7.5.2	Matched-field control	88
7.5.3	The already-served case, and the break-even that decides it	89
7.6	The Circuit-SNARK Reference Point	90
7.7	Update Churn	91
7.8	Threats to Validity	93
7.9	Summary	93
8	Discussion	94
8.1	The Anonymity–Deployability Tension	94
8.2	A Substrate-Selection Rule	94
8.3	What Generalises and What Does Not	96
8.4	Comparison to Prior Evaluations	96
9	Conclusion	97
9.1	Main Findings	97
9.2	Answer to the Research Question	99
9.3	Limitations	99
9.4	Future Work	100
9.5	Closing	102
	Acknowledgement	103

References	104
A Formal Development of the Security Preservation	113
A.1 Building-block properties	113
A.2 Relation preservation	114
A.3 Proof of the preservation theorem.....	115
B Evidence for Assumption 6.1: Specification-Level Soundness of the Griffin-$\mathbb{F}_p^{192}$ AIR	120
B.1 Setup.....	121
B.2 Soundness	121
B.3 Per-family lemmas	122
B.4 Completeness	125
B.5 The argument layer	125
B.6 The open step	125

List of Figures

1	The field-mismatch tax and what a precompile absorbs	44
2	The dual obstruction to post-quantum anonymity.	71
3	Within-pipeline proof-size collapse (same scheme, same runs).	82

List of Tables

1	Prior work mapped against the axes that separate it from this thesis; every row is a post-quantum construction, so that shared baseline is stated here. Symbols: ● = provided; ○ = a credential-adjacent object rather than a full anonymous credential; ⊙ = not achieved (Section 6); — = not applicable.	12
2	Griffin round margin versus broken CICO instances.	68
3	Property-survival matrix across proof configurations. The parenthesised entry is expected from the measured PLONK wrap, not itself measured.	70
4	Evaluation platform and measurement setup.	79
5	PLUM verification, prove mode ($\lambda = 80$; the Griffin instance runs at 14 rounds, one below the security-conservative 15 of Table 2).	80
6	Cycle-level cost attribution of the field-arithmetic optimizations.	82
7	Credential relations: execute and prove-mode cost ($\lambda = 80$).	83
8	Proof-mode feasibility: produced, zero-knowledge, post-quantum. [‡] Zero-knowledge only over a BN254 pairing (classical, not post-quantum); [§] zero-knowledge supported for this circuit but disabled in this run; [¶] includes a one-time 2.96 h core-shape pass (decomposition in §7.5, where the statement-bound $k=2$ wrap, 10.21 h, is also reported).	86
9	Aurora reference: one Loquat verification, three runs.	90
10	Regeneration footprint per localised relation change (Def. 4.7). *Loquat→PLUM regenerates the field-specific Griffin AIR.	91
11	Per-change recompile cost: Aurora versus Fractal.	92

1 Introduction

1.1 Background

An anonymous credential lets a holder prove a statement about themselves while revealing nothing else. Education is one place they are starting to be used. A school issues a credential, and a student uses it to prove a single fact, such as being enrolled or eligible for a programme, without revealing the rest of their record. Forged credentials cause real harm. Falsified admission letters were behind recent student removal proceedings [1]. The data a credential carries is also a target. The 2021 scraping of some 700 million LinkedIn profiles is one example [2]. This threat is also growing over time. Standards bodies now tell organisations to migrate to post-quantum cryptography [3], and industry is following. Google has committed to finish its own migration by 2029 [4]. A credential issued today must therefore be three things at once: unforgeable, anonymous, and secure against a future quantum computer. A recent construction, BDEC [5], shows the three can be had together, by combining a post-quantum signature with a post-quantum-secure proving system. Showing a credential means proving you hold a valid signature without revealing it, and the proving system is what makes that possible.

Building the credential once, however, is not the end. One meant to last a decade must keep working as two things around it change. First, what the holder must prove can change. A verifier may later ask for something the credential did not prove before. Second, the signature the credential is built on may be replaced as post-quantum standards mature. In a zkSNARK, what will be proved must be decided before the proof system is built, so either change forces the whole verification to be rebuilt and re-checked. A zkVM (zero-knowledge virtual machine) instead takes the computation as an input, so either change is only an edit to the program it runs.

1.2 From Constructing to Deploying

This flexibility has a cost. A zkVM can run any program, but whenever a program computes on numbers larger than the prover handles natively, it pays for it, and the signatures we care about do exactly that. BDEC’s efficiency rests on the signature scheme Loquat [6], for which this thesis substitutes its successor PLUM [7]. PLUM reports smaller signatures and faster verification at the same security level (Section 2). PLUM computes on very large numbers (192-bit, and a concrete 199-bit prime as instantiated; Section 3), while a zkVM works natively only with numbers of 31 bits. Each operation of the signature must therefore be emulated with many small-number operations, at high cost. Section 3 names this cost the *field-mismatch tax*. It is a real barrier. On the target hardware, verifying a single signature this way ran for hours and never produced a proof (Section 7).

This is the tension the thesis addresses. It asks what is required to run PLUM inside a zkVM on the consumer hardware a student or an institution would actually own, and at what cost. The accounting includes the changes that motivate the zkVM in the first place, both to what the credential proves and to the signature itself.

1.3 The Precompile Suite, and the Question It Answers

The zkVM measured here is SP1 [8], chosen because it lets the builder add *application-defined precompiles*. A precompile is a custom accelerator for one operation, which the program calls directly, so the operation runs at full speed instead of being emulated. The more work a proof performs, the

more it costs. A precompile removes the field-mismatch tax for an operation that would otherwise dominate that cost, in exchange for the smaller cost of its own accelerator. This thesis builds a suite of precompiles for PLUM verification and uses it as its instrument of study. Building the suite turns a verification that never finishes into one that completes, with a cost that can be measured and explained. It also forces a sharper question. Is a custom precompile worth building? The answer depends on what you compare it against. Without a precompile the hash never finishes, so the precompile for PLUM’s hash, Griffin, is what lets PLUM run at all. Yet a plain software SHA-3, which needs no precompile, proves *faster* than the accelerated Griffin (Section 7). A precompile can therefore be necessary to make an operation run and still not be worth building, when a simpler alternative proves faster. The SHA-3 version is a diagnostic control, not a deployable form of PLUM. This smaller question sits inside the central question of this thesis:

Can a post-quantum anonymous credential run inside a zkVM on consumer hardware without breaking the security properties it must provide, and when plain execution cannot finish, what makes it run?

The question separates into two layers. The feasibility layer is priced by a cost model (Section 4) that weighs the cost a precompile adds against the cost it removes. The security layer asks which of the three requirements survive the move, and each becomes a property the proof must have. Unforgeability needs the proof to be *knowledge-sound*, meaning only someone who genuinely holds a valid signature can produce a convincing proof. The zkVM provides this, so unforgeability transfers, under the assumptions of Section 6. Anonymity needs the proof to be *zero-knowledge*, revealing nothing beyond the fact it proves. Quantum-safety needs the proof to stay sound against an attacker with a quantum computer. *Deployable post-quantum anonymity* (Definition 6.3) asks for all three at once, in a single proof within the memory budget, and it is the one requirement SP1 cannot currently meet. Its default proof is quantum-safe but not zero-knowledge. The only proof its toolchain adds to supply zero-knowledge is pairing-based (Section 3), and it rests on elliptic-curve cryptography that a quantum computer breaks. So on the route SP1 ships today, a single proof is quantum-safe or anonymous, never both. This is a limit of the primitives available today, not a fact about zero-knowledge virtual machines. A fixed circuit reaches both at once, but only by giving up the *flexibility* the zkVM was chosen for. The deployment choice thus trades flexibility against post-quantum anonymity. The dual obstruction is formalised in Section 6.

The credential construction (BDEC [5]) instantiated with PLUM is the concrete workload through which the cost criterion built on that model (Section 4) is calibrated and measured on SP1. The criterion decides one question, whether a precompile is worth building. The choice of substrate is a second decision, weighed by the substrate-selection rule of Section 8. Both belong to the operator of a real deployment, such as a university consortium choosing a proving substrate for digital diplomas today. Such an operator must commit to a substrate before knowing which predicates its verifiers will demand or which parameter sets the standards process will finalise. The operator needs to know both what the precompile delivers and where it stops.

1.4 Contributions of This Thesis

This thesis analyses which of the credential’s security properties survive the move into a zkVM, locating the one that does not, deployable post-quantum anonymity; distils a reusable cost criterion for

when a precompile pays; builds the precompile suite that turns an emulation that does not complete within the budget into a measurable proof; and reports the first empirical characterisation of PLUM verification inside a zkVM. The contributions are as follows.

1. **The dual obstruction to post-quantum anonymity, and the everlasting-anonymity boundary.** The credential’s unforgeability transfers to the substrate, because it reduces to the zkVM’s knowledge soundness, which the zkVM provides. Deployable post-quantum anonymity does not transfer, and the obstruction has two sides. SP1’s default proof (the object its toolchain calls a *receipt*) is feasible and post-quantum, but it is not zero-knowledge (the first obstruction). The zero-knowledge wrap SP1’s toolchain adds does fit the consumer memory budget, but it is pairing-based, resting on elliptic-curve cryptography that quantum computers break, so it is not post-quantum (the second obstruction). Feasibility is therefore not the obstruction; a limit of the available primitives is. A deployer who accepts classical security can close the first obstruction with the classical wrap, which leaves the second as the one that survives. Pushing the question to its limit, we prove that *everlasting* anonymity, safety even against an attacker with unlimited computing time after the fact, cannot be established in the standard model (that is, without idealised oracles) for the class of hash-committed showings this construction uses, and we give the conditional route out (Section 6). The obstruction is formalised in Section 6 and measured in Section 7.
2. **A cost criterion for when a precompile pays, and the hash-cost inversion it predicts.** Whether a precompile *pays* is really two questions the one word runs together. Against doing the operation without a precompile, it pays when the cost it removes outweighs the cost of its own accelerator, a feasibility test. Against the best alternative for the same operation, it pays only if it adds less total proving cost than that alternative. Our cost model (Section 4) answers both from a precompile’s size, before it is built. The Griffin hash precompile over PLUM’s field ($\mathbb{F}_p^{192}$) [9] passes the first test and fails the second. It recovers a proof that emulation cannot finish, yet proves *slower* than a software SHA-3 control, because its own accelerator adds proving cost a plain hash never pays. That comparison is run at $\lambda=80$, below PLUM’s headline security level, because the full level does not complete on the target hardware. The control is matched on security level, not on field or hash, and is a diagnostic, not a deployable variant (Section 7). The model separates the three precompiles the thesis ships the same way. Griffin pays for feasibility but loses to the SHA-3 control. The field-multiplication precompile is predicted not to pay, because SP1 already ships a multiplication precompile (its `UINT256_MUL` accelerator) that serves it (Section 5). The power-residue check, which unlike the hash cannot be moved to a smaller field (Section 5), still pays on total proving cost (Section 7). The criterion is reusable within this class. For another algebraic primitive on a small-field zkVM, a builder can decide from the specification alone whether a precompile lowers total proving cost, and the model gives a testable matched-field forecast, checked in Section 7.
3. **A precompile suite for PLUM verification over a 199-bit prime.** We build three precompiles over the same field, one for the Griffin hash, one for field multiplication, and one for the power-residue check, each with a written argument that its accelerator computes the same function as the reference specification (this argument is partially discharged at the lowest level; Section 6). All three are built and test-proved, but only the Griffin precompile is on the measured path (Section 5). The Griffin precompile is what recovers feasibility.

4. **The first empirical characterisation of PLUM-in-a-zkVM.** We write the credential’s issuing and showing steps as SP1 programs with PLUM in place of Loquat, and run PLUM verification on its own, measuring both how many steps it takes and how long it takes to prove (Section 7, where every figure appears with its full configuration). Without a precompile, verification was stopped unfinished after five hours. With the Griffin precompile, five proofs averaged 14.24 minutes at $\lambda = 80$. To our knowledge, this is the first such measurement of PLUM verification inside a zkVM.

Security scope. We give one conditional theorem that the security is preserved when PLUM runs inside the zkVM (Section 6). In the classical random-oracle model, it bounds how much an attacker can gain against the credential system’s unforgeability, anonymity, and unlinkability. Anonymity and unlinkability are the two parts of the anonymity requirement from Section 1. Whether anonymity survives a quantum attacker is the subject of the dual obstruction. The positive results rest on five explicitly named open assumptions. They also rest on one named conjecture (an extractor conjecture), needed because a long run’s proof is split into pieces and recursively combined, and the argument that this combined proof is knowledge-sound is not yet complete (Section 3, Section 6).

1.5 Thesis Organisation

The thesis proceeds as follows: related work (Section 2) and primitives (Section 3); the cost model and update churn at both levels (Section 4); the construction and precompile suite (Section 5); the security-preservation theorem and its open assumptions (Section 6); the measurements (Section 7); the deployment decision framework (Section 8); and the conclusion (Section 9).

2 Related Work

A post-quantum anonymous credential can already be built and proved; what no prior work asks is whether that proof can be generated on the holder’s own consumer machine. On a service operator’s server, a changed predicate is absorbed by rebuilding and re-auditing the static verification circuit, so flexibility buys little there. Only once the holder proves on hardware they own does the cost of every such change start to matter. Each regenerated circuit artefact must then be redistributed to and re-trusted by every holder’s device, with no operator on hand, as there would be behind a service boundary, to re-audit and swap it. This is the case where a zkVM’s ability to edit the program, rather than rebuild the circuit, matters, and it is why a zkVM is needed here. Table 1 places some prior work against that baseline: every construction shares the post-quantum, privacy-preserving goal (two rows reach credential-adjacent objects rather than a full anonymous credential, marked $\circ$), so the table maps only the axes that separate them.

The *flexibility* column is where this thesis stands alone, and the last column names what it currently stops short of: a single proof that is at once post-quantum-sound and zero-knowledge. The sections below take the attempts in turn, asking at each step why the attempt stops short of the object measured here: constructing a credential directly, proving a signature in a static circuit, running one inside a zkVM, accelerating that execution with a precompile, then what security a zkVM receipt carries, and finally the gap this thesis fills.

Work	Anonymous credential?	Substrate	Flexibility	PQ zero-knowledge
Jeudy et al. [10]	•	—	Low	•
BDEC [5]	•	Aurora	Low	•
Policharla et al. [11]	$\circ$	STARK	Low	•
CAPSS [12]	$\circ$	SmallWood	Low	•
This thesis	•	zkVM + precompile	High	$\oslash$

Table 1: Prior work mapped against the axes that separate it from this thesis; every row is a post-quantum construction, so that shared baseline is stated here. Symbols: • = provided; $\circ$ = a credential-adjacent object rather than a full anonymous credential; $\oslash$ = not achieved (Section 6); — = not applicable.

2.1 Constructing an Anonymous Credential Directly

The oldest approach builds an anonymous credential as a dedicated protocol. Chaum introduced the idea [13], Camenisch and Lysyanskaya formalised it [14], and the line runs through BBS, Pointcheval–Sanders, and Coconut [15, 16, 17] into the W3C Verifiable Credentials data model [18]; Rosenberg et al. [19] reach the same goal by proving statements over existing identity infrastructure. All of these rest on classical assumptions, RSA, pairing, or discrete logarithm, and so do not survive a quantum adversary, which is why we treat them as deployment baselines rather than post-quantum baselines. The post-quantum case has been pursued largely with lattices: Jeudy et al. at Crypto 2023 [10] give an

efficient scheme on SIS and LWE, refined by later constructions [20, 21]. These deliver the credential, but at large proof and key sizes, and each is a bespoke protocol rather than an existing scheme’s verification run as a program.

2.2 Proving Signature Verification in a Static Circuit

A second approach reaches a post-quantum credential generically, by proving in zero knowledge that one knows a valid signature, and compiles that verification into a fixed arithmetic circuit. The standardised post-quantum signatures ML-DSA, SLH-DSA, and Falcon [22, 23, 24] verify cheaply in the clear but embed into such a circuit only at large constraint counts, so this line instead uses signatures written for a low constraint count: the MPC-in-the-head family (Picnic, Banquet, Limbo [25, 26, 27], after Katz–Kolesnikov–Wang [28]) and the Legendre/power-residue family begun by LegRoast [29]. Loquat [6] bases unforgeability on the Legendre PRF and is engineered to fit a small rank-1 constraint system (R1CS); PLUM [7], the scheme this thesis deploys, replaces that PRF with the t -th power-residue PRF and FRI with STIR [30] (two protocols for the same task, testing that a committed function is close to a low-degree polynomial; Section 3), reporting at 128-bit security signatures roughly $1.5\times$ smaller and $1.28\times$ fewer constraints, with verification estimated up to $4\times$ faster, and the family has since been extended to ring signatures [31].

Built on such a signature, several works reach a credential or credential-adjacent application. BDEC (Li, Zhang et al.) [5] is the closest prior work. It instantiates a post-quantum anonymous credential with Loquat and the transparent argument Aurora [32], proving unforgeability, anonymity, unlinkability, conditional linkability, and revocability, improving on the lattice baseline. But BDEC’s guarantees hold for a static circuit, so any change to the signature or the disclosed predicate recompiles and re-audits it, and its anonymity requires the argument to be zero-knowledge, which the consumer-hardware zkVM receipt this thesis produces does not yet satisfy (Section 6). CAPSS (Feneuil and Rivain) [12] pushes the constraint count $5\text{--}8\times$ below Loquat. It reports measured figures on SmallWood, its authors’ own hash-based proving system, for an application built on the signature rather than a full credential scheme. So measured proving of an algebraic-hash signature (one built from a hash defined by algebraic operations, in the vocabulary of Section 3) predates this thesis. But it uses a different signature family from the PLUM scheme this thesis deploys, optimised for static-circuit constraint count. Policharla et al. [11] take the same static route to a Privacy Pass, credential-adjacent rather than a full anonymous credential, proving a *modified* Dilithium inside a hand-rolled STARK built on the Winterfell prover library. Across this line the verification is proved as a native circuit and recompiled on every change (preprocessing variants such as Fractal [33] and Marlin [34] only trade that recompile for a large per-relation setup).

2.3 Running a Signature Inside a zkVM

A third approach runs a post-quantum signature as an ordinary program inside a zkVM. s2morrow [35] verifies Falcon-512 and SPHINCS+ in Cairo with no custom precompile, HAPPIER [36] verifies XMSS through a built-in SHA-256 precompile, and an ML-DSA implementation [37] runs on SP1 [8] with its number-theoretic transform (NTT) implemented as guest code. These efforts run on Cairo, RISC Zero [38], and SP1. The broader zkVM landscape also includes Jolt [39] and the custom-chip-native OpenVM [40], while compiler front-ends such as zkLLVM [41], Circom [42], and Noir [43] lock

the predicate at compile time and so behave like the static-circuit regime above. Yet each demonstration verifies a *standardised or hash-based* signature, authors no custom precompile for a large-prime-field algebraic hash, and is a bare signature rather than an anonymous credential.

2.4 Accelerating a zkVM with a Precompile

Where a primitive dominates a zkVM’s cost, prior work replaces its slow emulated version with a custom accelerator (a precompile) that the program calls directly: SP1 ships SHA-256 and ECDSA precompiles [8], and a community RSA precompile is reported to cut verification by roughly two orders of magnitude.¹ Such a precompile joins the trusted computing base and must be audited: Arguzz [44] found eleven soundness or completeness bugs across three of the six zkVMs it tested, which is why each precompile of this thesis ships a written soundness argument. Two 2025 lines automate the accelerate-or-not decision, powdr’s autoprecompiles [45] and the vApps mispricing analysis [46]; both rank or reprice from profiling data gathered *after* the accelerator exists, whereas the criterion of Section 4 decides from the specification, before any build, whether a precompile over the signature’s own large field removes the field-mismatch tax at all. No prior precompile targets a 199-bit-prime algebraic hash, and none carries a from-specification pay criterion.

2.5 The Security a zkVM Receipt Carries

A separate line asks not what a zkVM proves but what security its receipt actually carries. Haböck and Kindi [47] show that the extra step these proof systems need to keep the witness hidden is easy to get wrong and a recurring source of implementation error, and RISC Zero’s own advisory [48] records a period in which receipts advertised as zero-knowledge leaked witness information; production toolchains therefore restore zero-knowledge with an outer pairing-based wrap in the Groth16 family [49, 50], whose security does not survive a quantum adversary, and only emerging hash-based zero-knowledge layers [51] would remove that pairing. Each of these facts is established on its own; to our knowledge no prior work states their conjunction as an obstruction to a *post-quantum* application, nor analyses which security properties of a credential survive execution on a zkVM, the question Section 6 takes up. At the information-theoretic limit, everlasting privacy against a later unbounded adversary was introduced by Moran and Naor [52] and shown unattainable from standard setup assumptions by Müller-Quade and Unruh [53], with recent work bringing it to credential-adjacent tokens [54]; the everlasting-anonymity non-establishability result of Section 6 is the analogue of that boundary for hash-committed credential showings, which no prior work states.

2.6 The Gap and the Unreported Dimension

The prior work has thus reached every piece of the problem and none of the whole: credentials built directly but never run in a zkVM; signature verification proved efficiently but in a recompiled static circuit; signatures run in a zkVM but standardised, without a custom precompile, and not as credentials; precompiles built but never for a large-field algebraic hash; and receipt security studied but never

¹Reported in vendor and community engineering write-ups without a peer-reviewed measurement; we cite it only as a qualitative indication of the pattern.

as a credential-anonymity obstruction. No prior work runs an algebraic-hash post-quantum anonymous credential inside a zkVM to learn whether it runs at all, at what proving cost, and with which security properties intact. A further dimension goes unreported throughout: the cost each change to the verification predicate imposes, which crypto-agility work [55] names as a design concern but no evaluation measures, and which the recent zkVM systematisation [56] omits alongside its classical-workload benchmarks. This thesis fills the gap and reports that dimension as *update churn*: it develops a cost criterion for when a precompile pays (Section 4), analyses which security properties survive the move (Section 6), and measures the deployability and the update churn of BDEC-with-PLUM on consumer hardware (Section 7).

3 Preliminaries

3.1 Notation

Let λ denote the security parameter. A function $f(\lambda)$ is negligible if for every polynomial p there exists N such that, for all $\lambda > N$, $f(\lambda) < 1/p(\lambda)$. We write $\text{negl}(\lambda)$ for an unspecified negligible function. All algorithms and adversaries considered below are classical probabilistic polynomial-time unless stated otherwise. Security statements inherited from the underlying signature schemes hold in the classical random-oracle model, an idealisation in which every party evaluates the hash function only by querying a truly random function, and any quantum variant (QROM, in which the adversary may query that function in superposition) is flagged explicitly where it arises. For a randomized algorithm A , the notation $y \leftarrow A(x)$ means that A is run on input x using fresh randomness and outputs y . For a finite set S , $y \leftarrow S$ means that y is sampled uniformly from S . We write $[n] = \{1, \dots, n\}$. Attribute vectors are denoted by $A = (a_1, \dots, a_m)$, and predicates over attributes are denoted by ϕ . Public keys, secret keys, messages, and signatures are denoted by pk , sk , m , and σ , respectively.

Definition 3.1 (Efficiently decidable relation). A relation $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$ is efficiently decidable if there exists a deterministic polynomial-time algorithm D_R such that, for every pair (x, w) ,

$$D_R(x, w) = 1 \iff R(x; w) = 1.$$

We call x the public instance and w the witness. The statement $R(x; w) = 1$ means that w is a valid witness for the public instance x .

3.2 Cryptographic Primitives

This subsection collects the primitives that appear inside both the signature schemes (Loquat, PLUM) and the BDEC construction below. We work with a single odd prime p throughout. The same field is reused by every primitive in any one instantiation, eliminating cross-primitive field conversions. PLUM achieves this fully. Loquat still lifts values into a larger extension field, as the quantitative comparison below notes.

3.2.1 Power residue PRF.

Definition 3.2 (t -th power residue PRF). Let p be an odd prime and let $t \geq 2$ be an integer with $t \mid (p-1)$. Let $g \in \mathbb{F}_p^*$ be a generator and let

$$\zeta_t = g^{(p-1)/t} \in \mathbb{F}_p^*$$

be a primitive t -th root of unity, so that the cyclic subgroup

$$\mu_t = \langle \zeta_t \rangle = \{1, \zeta_t, \zeta_t^2, \dots, \zeta_t^{t-1}\}$$

is the set of t -th roots of unity in $\mathbb{F}_p^*$. For a key $K \in \mathbb{F}_p$ and an input $a \in \mathbb{F}_p$ with $K + a \neq 0$, define

$$\chi_t(K, a) = (K + a)^{(p-1)/t} \in \mu_t,$$

and let $L_t(K, a) \in \mathbb{Z}/t\mathbb{Z}$ be the unique exponent for which

$$\chi_t(K, a) = \zeta_t^{L_t(K, a)}.$$

The keyed function $L_t(K, \cdot) : \mathbb{F}_p \setminus \{-K\} \rightarrow \mathbb{Z}/t\mathbb{Z}$ is the t -th power residue PRF.

The special case $t = 2$ makes $\chi_2(K, a) \in \{+1, -1\}$ the Legendre symbol of $K + a$ modulo p . The corresponding L_2 is the Legendre PRF used by Loquat. Taking $t > 2$ yields more entropy per evaluation. The PLUM construction described later in this section pins $t = 256$. Definition 3.2 only fixes the algebraic map. Its use as a PRF rests on a separate hardness assumption, namely that $L_t(K, \cdot)$ is pseudorandom for a uniformly random key K . This assumption generalises the Legendre-PRF assumption underlying Loquat and inherits that assumption’s cryptanalytic status rather than being implied by the map being well-defined.

3.2.2 Algebraic hash function.

We require an algebraic hash function whose internal computation is dominated by low-degree field operations over the same field $\mathbb{F}_p$ used by the signature scheme and the PRF. We use the Griffin family for this purpose. Following the cited literature, a Griffin instance is parameterised by a state width t_{state} and a number of rounds. Each round consists of constant-addition, linear mixing by a fixed matrix, and a low-degree S-box substep using small-degree power maps, namely a forward degree- d power map on one lane, its inverse $x \mapsto x^{1/d}$ (well-defined since $\gcd(d, p-1) = 1$) on a second, and a Horst-type quadratic map on the remaining lanes (§5.3). Each constraint then has degree at most d while the per-round map has high algebraic degree. We write

$$H_{\text{Griffin}} : \mathbb{F}_p^* \rightarrow \mathbb{F}_p^c$$

for a Griffin-based sponge or compression hash producing a c -element digest, following the construction of [9]. The instantiation used throughout has state width $t_{\text{state}} = 4$, capacity 2 (hence rate 2 and digest length $c = 2$), S-box degree $d = 3$ (the smallest integer ≥ 3 with $\gcd(d, p-1) = 1$ over PLUM’s prime; §5.3), and 14 rounds, the value the Griffin round-count argument of [9] yields for this width-4, 199-bit instance (computed and checked in the reference implementation, §5.3). The digest length $c = 2$ over the 199-bit field is intended to provide collision resistance at the targeted security level, conditional on the security analysis of [9] applying to this width-4, $d = 3$, 14-round instance; the deployed Merkle path moreover truncates the two-lane digest to 32 bytes (one field element). Its collision resistance is therefore birthday-capped at $\sqrt{p} \approx 2^{99.5}$, regardless of the analysis. This ceiling is disclosed with the assumption bundle of Section 6.

The implemented linear layer is the standard Griffin width-4 MDS matrix M_4 , with rows $(5, 7, 1, 3)$, $(4, 6, 1, 1)$, $(1, 3, 5, 7)$, $(1, 1, 4, 6)$, the genuine MDS matrix (branch number 5) of [9], and the nonlinear layer is the standard one, with the lane-3 combination L_3 built from the round-*input* lane x_2 exactly as in the published Griffin- π . The permutation used on the Fp192 PLUM path, which carries the central measurements of this thesis, is therefore the published Griffin- π of [9] at these parameters. It was validated against an independent from-specification reference implementation on known-answer vectors, which agree on every lane.

The Fp127 Loquat instantiation kept only as the in-SNARK reference of Section 7.6 still carries Loquat’s inherited linear and nonlinear layers. That choice leaves the R1CS constraint count of a Griffin permutation unchanged, so it is cost-neutral for the constraint- and time-measurement that reference

serves, and we make no claim that this path computes the published permutation. What remains conditional is the round-count security *margin*. Whether the analysis of [9] bounds the algebraic-attack cost of this width-4, $d = 3$, 199-bit, 14-round instance is a separate question, taken up in the security chapter, and is unaffected by the arithmetic fixed here.

When a zkVM (the machinery of syscalls and precompiles is defined at the end of this section) exposes Griffin as a precompile we will write the corresponding syscall as $\text{Griffin}_{\mathbb{F}_p^{192}}$, realised in the implementation as `GRIFFIN_FP192_PERMUTE` (§5.4), indicating that the precompile operates natively over the same prime field used by PLUM (nominally labelled 192-bit, concretely the 199-bit prime of Remark 3.5). The cost of one Griffin permutation under this precompile is, by definition, one zkVM syscall: one guest cycle plus the syscall ABI’s constant overhead.

3.2.3 Merkle commitment.

Definition 3.3 (Merkle commitment). Let $H : \mathbb{F}_p^c \times \mathbb{F}_p^c \rightarrow \mathbb{F}_p^c$ be a collision-resistant compression function and let $v = (v_1, \dots, v_n)$ with $v_i \in \mathbb{F}_p^c$ be a vector of leaves. The Merkle root

$$\text{rt} = \text{MerkleRoot}_H(v) \in \mathbb{F}_p^c$$

is defined by iterated pairwise application of H along a complete binary tree over v , padding to the next power of two as needed. For index $i \in [n]$, the membership proof path_i is the sequence of sibling digests along the root path. The verification predicate

$$\text{MerkleVerify}(\text{rt}, v_i, \text{path}_i) \in \{0, 1\}$$

returns 1 iff recomputing the root from (v_i, path_i) yields rt .

We instantiate the Merkle commitments of the BDEC system below with $H = H_{\text{Griffin}}$ over $\mathbb{F}_p$, so that each path step reduces to a small number of algebraic-hash invocations. The ledger trees so committed are checked by the relying party outside the proof, while the in-proof Griffin Merkle cost arises from the BCS commitments internal to PLUM verification. A *sparse* Merkle tree is the same structure indexed over the digest range $\{0, 1\}^\lambda$ rather than $[n]$. Non-membership is shown by a membership proof of a domain-separated sentinel leaf at the queried index, and we write $\text{NonMemberVerify}(\text{rt}, x, \text{path})$ for the corresponding predicate.

3.3 Digital Signature Scheme

Definition 3.4 (Digital signature). A digital signature scheme is a tuple of probabilistic polynomial-time algorithms

$$\text{Sig} = (\text{Sig.Setup}, \text{Sig.KeyGen}, \text{Sig.Sign}, \text{Sig.Verify}).$$

On input the security parameter, the setup algorithm outputs public parameters

$$\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda).$$

The parameters pp_{Sig} determine a public-key space $\mathcal{PK}_{\text{ppSig}}$, a secret-key space $\mathcal{SK}_{\text{ppSig}}$, a message space $\mathcal{M}_{\text{ppSig}}$, and a signature space Σ_{ppSig} . The algorithms have the following syntax:

$$\begin{aligned} (\text{sk}, \text{pk}) &\leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}}), & (\text{sk}, \text{pk}) &\in \mathcal{SK}_{\text{ppSig}} \times \mathcal{PK}_{\text{ppSig}}, \\ \sigma &\leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}, m), & m &\in \mathcal{M}_{\text{ppSig}}, \sigma \in \Sigma_{\text{ppSig}}, \\ b &\leftarrow \text{Sig.Verify}(\text{pp}_{\text{Sig}}, \text{pk}, m, \sigma), & b &\in \{0, 1\}. \end{aligned}$$

When a higher-level protocol uses Sig , every value to be signed must first be represented as an element of the message space $\mathcal{M}_{\text{ppSig}}$. We write

$$\text{Encode}_{\text{Sig}} : \{0, 1\}^* \rightarrow \mathcal{M}_{\text{ppSig}}$$

for this encoding map. Thus, if a protocol value v is signed, the actual message given to Sig.Sign is

$$m = \text{Encode}_{\text{Sig}}(v) \in \mathcal{M}_{\text{ppSig}}.$$

We require $\text{Encode}_{\text{Sig}}$ to be deterministic and injective, so that distinct protocol values encode to distinct messages; the credential unforgeability reduction of Section 6 consumes exactly this injectivity.

Correctness. Correctness requires that for all λ , all $\text{ppSig} \leftarrow \text{Sig.Setup}(1^\lambda)$, all $(\text{sk}, \text{pk}) \leftarrow \text{Sig.KeyGen}(\text{ppSig})$, and all messages $m \in \mathcal{M}_{\text{ppSig}}$, if $\sigma \leftarrow \text{Sig.Sign}(\text{ppSig}, \text{sk}, m)$, then

$$\Pr[\text{Sig.Verify}(\text{ppSig}, \text{pk}, m, \sigma) = 1] \geq 1 - \text{negl}(\lambda),$$

where the probability is over the randomness of the algorithms.

Existential Unforgeability under Chosen Message Attack. For unforgeability, define $\text{Exp}_{\text{Sig}, \mathcal{A}}^{\text{euf-cma}}(\lambda)$ as follows. The challenger samples

$$\text{ppSig} \leftarrow \text{Sig.Setup}(1^\lambda), \quad (\text{sk}, \text{pk}) \leftarrow \text{Sig.KeyGen}(\text{ppSig}),$$

and gives $(\text{ppSig}, \text{pk})$ to $\mathcal{A}$. The adversary has oracle access to

$$\mathcal{O}_{\text{Sign}}(m) : \sigma \leftarrow \text{Sig.Sign}(\text{ppSig}, \text{sk}, m),$$

for messages $m \in \mathcal{M}_{\text{ppSig}}$, and the challenger records every queried message in a set Q . Eventually $\mathcal{A}$ outputs (m^*, σ^*) . The experiment outputs 1 iff

$$m^* \in \mathcal{M}_{\text{ppSig}} \quad \wedge \quad m^* \notin Q \quad \wedge \quad \text{Sig.Verify}(\text{ppSig}, \text{pk}, m^*, \sigma^*) = 1.$$

The signature scheme is EUF-CMA secure if for every adversary $\mathcal{A}$,

$$\Pr[\text{Exp}_{\text{Sig}, \mathcal{A}}^{\text{euf-cma}}(\lambda) = 1] \leq \text{negl}(\lambda).$$

Plain EUF-CMA suffices for the credential *unforgeability* reduction of Section 6; its anonymity and unlinkability reductions additionally assume signature-transcript simulatability (Assumption 6.2) and, for issuance anonymity, signature key-privacy (Assumption 6.3). BDEC's showing relation (defined with the construction later in this section) binds the signed message inside the proof, and the unforgeability reduction extracts a *witness* (a signature on a statement outside the issued set) rather than relying on signature uniqueness or non-malleability, so neither strong unforgeability nor resistance to signature re-randomisation is required.

Signature-Verification Predicate. Fix public parameters $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$. For $\text{pk} \in \mathcal{PK}_{\text{ppSig}}$, $m \in \mathcal{M}_{\text{ppSig}}$, and $\sigma \in \Sigma_{\text{ppSig}}$, define

$$\text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}, m, \sigma) = 1 \iff \text{Sig.Verify}(\text{ppSig}, \text{pk}, m, \sigma) = 1.$$

The signature σ is computed by $\sigma \leftarrow \text{Sig.Sign}(\text{ppSig}, \text{sk}, m)$. The predicate does not compute σ . It only checks whether a given σ is valid for (pk, m) . When this predicate is inserted into a zkSNARK relation, the public instance and private witness must be specified. For example,

$$R_{\text{Sig}}((\text{pk}, m); \sigma) = 1 \iff \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}, m, \sigma) = 1,$$

where $x = (\text{pk}, m)$ is the public instance and $w = \sigma$ is the witness.

3.3.1 Loquat: a Legendre-PRF post-quantum signature with low R1CS verification cost.

Loquat is a post-quantum digital signature scheme based on the Legendre PRF and collision-resistant hash functions, designed so that its verification algorithm can be implemented by a relatively small rank-1 constraint system (R1CS) circuit (a system of bilinear constraints of the form $(Az) \circ (Bz) = Cz$ over $\mathbb{F}_p$, made precise where Aurora is introduced below). Its post-quantum claim rests on the conjectured hardness of the Legendre PRF, collision resistance of the hash, and the Fiat–Shamir/BCS (Ben-Sasson–Chiesa–Spooner) transform [57], the compiler that turns an interactive oracle proof into a non-interactive argument. It is a research scheme and is not standardised in the manner of ML-DSA or SLH-DSA.

The Loquat literature [6] describes such verification as “SNARK-friendly”. We do not use that term, because it mixes several separate cost properties. We name instead the structural quantity that matters here, a low constraint count for the verification predicate when encoded as an R1CS.

Formally, a Loquat instance is a tuple of probabilistic polynomial-time algorithms

$$\text{Loquat} = (\text{Loquat.Setup}, \text{Loquat.KeyGen}, \text{Loquat.Sign}, \text{Loquat.Verify}).$$

On input the security parameter, the setup algorithm outputs public parameters

$$\text{pp}_{\text{Loquat}} \leftarrow \text{Loquat.Setup}(1^\lambda).$$

These parameters fix, among other things, a prime field $\mathbb{F}_p$, a public list $I = (I_1, \dots, I_L) \in \mathbb{F}_p^L$ for some integer L , and auxiliary parameters for the underlying Legendre-PRF key-identification protocol and its compilation to a non-interactive proof system.

The key-generation algorithm samples a secret Legendre-PRF key and its corresponding public key as a vector of PRF evaluations:

$$\begin{aligned} \text{sk} &= K \leftarrow \text{Loquat.KeyGen}(\text{pp}_{\text{Loquat}}), \\ \text{pk} &= (L_2(K, I_1), \dots, L_2(K, I_L)) \in \{0, 1\}^L, \end{aligned}$$

where $L_2(K, \cdot)$ is the keyed Legendre pseudorandom function, the $t = 2$ case of Definition 3.2, and each $L_2(K, I_\ell) \in \mathbb{Z}/2\mathbb{Z} \cong \{0, 1\}$ is the Legendre-symbol output bit on input $K + I_\ell$.

The message space $\mathcal{M}_{\text{Loquat}}$ and signature space Σ_{Loquat} are those induced by the non-interactive key-identification protocol obtained from the underlying interactive-oracle proof via the Fiat–Shamir/BCS

transformation. On input $(\text{pp}_{\text{Loquat}}, \text{sk}, m)$ with $m \in \mathcal{M}_{\text{Loquat}}$, the signing algorithm runs this protocol using the secret key K as the witness and outputs a signature

$$\sigma \leftarrow \text{Loquat.Sign}(\text{pp}_{\text{Loquat}}, \text{sk}, m) \in \Sigma_{\text{Loquat}}.$$

The verification algorithm is deterministic and checks the hash chain, Merkle-tree commitments, univariate-sumcheck proof, low-degree test, and quadratic-residuosity conditions implied by the Legendre-PRF relation:

$$b \leftarrow \text{Loquat.Verify}(\text{pp}_{\text{Loquat}}, \text{pk}, m, \sigma) \in \{0, 1\}.$$

Loquat-Verification Predicate. Fix $\text{pp}_{\text{Loquat}} \leftarrow \text{Loquat.Setup}(1^\lambda)$. For $\text{pk} \in \mathcal{PK}_{\text{Loquat}}$, $m \in \mathcal{M}_{\text{Loquat}}$, and $\sigma \in \Sigma_{\text{Loquat}}$, define

$$\text{Pred}_{\text{Verify}}^{\text{Loquat}, \text{pp}_{\text{Loquat}}}(\text{pk}, m, \sigma) = 1 \iff \text{Loquat.Verify}(\text{pp}_{\text{Loquat}}, \text{pk}, m, \sigma) = 1.$$

When this predicate is encoded as a zkSNARK relation, we take, for example,

$$R_{\text{Loquat}}((\text{pk}, m); \sigma) = 1 \iff \text{Pred}_{\text{Verify}}^{\text{Loquat}, \text{pp}_{\text{Loquat}}}(\text{pk}, m, \sigma) = 1,$$

where $x = (\text{pk}, m)$ is the public instance and $w = \sigma$ is the witness.

3.3.2 PLUM: a power-residue-PRF post-quantum signature with low R1CS verification cost.

PLUM is a post-quantum digital signature scheme that follows the same overall structure as Loquat but replaces the Legendre PRF and FRI-based (Fast Reed–Solomon Interactive Oracle Proof of Proximity [58]) low-degree test with a t -th power residue PRF and the STIR proximity test [30]² (Reed–Solomon proximity testing with fewer queries than FRI), and selects a prime field tailored to these primitives. These substitutions reduce the polynomial degrees and query complexity in the underlying proof system, leading to smaller signatures and fewer R1CS constraints in the verification circuit while, per the PLUM authors’ analysis, retaining EUF-CMA security at the claimed level (subject to the prime-instantiation caveat of Remark 3.5).

Formally, a PLUM instance is a tuple of probabilistic polynomial-time algorithms

$$\text{Plum} = (\text{Plum.Setup}, \text{Plum.KeyGen}, \text{Plum.Sign}, \text{Plum.Verify}).$$

On input the security parameter, the setup algorithm outputs public parameters

$$\text{pp}_{\text{Plum}} \leftarrow \text{Plum.Setup}(1^\lambda).$$

The setup chooses an odd prime p and an integer $t \geq 2$ such that $t \mid (p - 1)$, fixes a generator $g \in \mathbb{F}_p^*$, and defines the t -th power residue PRF L_t as in Definition 3.2. It then samples a public list $I = (I_1, \dots, I_L) \in \mathbb{F}_p^L$ and sets the parameters for the univariate-sumcheck and STIR low-degree test (including the code domain U , folding parameter, degree bounds, and query-repetition parameters) as in Algorithm 1 of [7].

²WHIR [59], from the same authors, is a later Reed–Solomon proximity test with faster verification that a future PLUM instantiation could adopt in place of STIR; PLUM as published uses STIR.

Remark 3.5 (Prime instantiation). The implementation evaluated in this thesis fixes a concrete 199-bit prime $p = 2^{64} p_0 + 1$ with p_0 a 135-bit prime,³ chosen so that $t = 256 \mid (p-1)$ and the 2-adicity of $p-1$ is at least 64, as the STIR low-degree test requires. The value of p_0 used is *not* the decimal printed in the published parameter table [7, §3.3]. That printed value is composite (its smallest prime factor is 97, witnessed by a primality test in the artefact), almost certainly a typographical error in the manuscript, and is therefore unusable as a field modulus. We substitute a nearby 199-bit value satisfying all the structural requirements above. The substitution is benign for the measurements of this thesis, whose cost is a function of the field *bit-width* and smoothness rather than of the exact decimal value. It is recorded explicitly because any concrete security-bit claim that depends on the specific value of p is conditional on this instantiation, and a paper-faithful or interoperable deployment should confirm the intended modulus with the PLUM authors. The security analysis (Section 6) carries this conditionality as an explicit hypothesis.

The key-generation algorithm samples a secret PRF key and derives the public key as a vector of t -th power residue PRF evaluations:

$$\begin{aligned} \text{sk} &= K \leftarrow \text{Plum.KeyGen}(\text{pp}_{\text{Plum}}), \\ \text{pk} &= (L_t(K, I_1), \dots, L_t(K, I_L)) \in (\mathbb{Z}/t\mathbb{Z})^L, \end{aligned}$$

where each $L_t(K, I_\ell) \in \mathbb{Z}/t\mathbb{Z}$ is a symbol of the t -th power residue PRF on input $K + I_\ell$ (for $t = 2$ this degenerates to the Loquat binary Legendre symbol). The public key thus occupies $L \lceil \log_2 t \rceil$ bits. Compared to Loquat, taking $t > 2$ increases the entropy per symbol and allows PLUM to use fewer public-key symbols L and fewer challenged symbols B at a given security level.

The message space $\mathcal{M}_{\text{Plum}}$ and signature space Σ_{Plum} are induced by the non-interactive key-identification protocol obtained from the PLUM-specific interactive-oracle proof via the Fiat–Shamir/BCS transformation, with FRI replaced by STIR in the low-degree test component. On input $(\text{pp}_{\text{Plum}}, \text{sk}, m)$ with $m \in \mathcal{M}_{\text{Plum}}$, the signing algorithm runs this protocol using the secret key K as the witness and outputs a signature

$$\sigma \leftarrow \text{Plum.Sign}(\text{pp}_{\text{Plum}}, \text{sk}, m) \in \Sigma_{\text{Plum}}.$$

The verification algorithm is deterministic and checks the hash chain, Merkle-tree commitments, univariate-sumcheck proof, STIR low-degree test, and t -th-power-residue conditions implied by the PLUM PRF relation:

$$b \leftarrow \text{Plum.Verify}(\text{pp}_{\text{Plum}}, \text{pk}, m, \sigma) \in \{0, 1\}.$$

For the 128-bit parameter set, PLUM uses $t = 256$, a public-key length of $L = 2^{12}$ PRF symbols, and $B = 28$ challenged symbols, and verifying a single signature with Griffin as the hash instantiation requires about 1.16×10^5 R1CS constraints, versus $\approx 1.48 \times 10^5$ for Loquat under comparable assumptions [7, §4.2]. The corresponding signature size is about 37 KB (versus 57 KB for Loquat). The signing and verification runtimes are *estimated*, from the Loquat implementation rather than a from-scratch PLUM measurement [7, Table 2], to be up to a factor 4 faster than Loquat at the same security level.

³The implementation prime is 199-bit (with an approximately 135-bit p_0). The “192-bit” / “ $\mathbb{F}_p^{192}$ ” naming used elsewhere in this thesis for the PLUM field and the Griffin precompile is a loose label rather than the exact bit-width. PLUM’s own source is internally inconsistent on this point, the prose and the printed p_0 disagreeing.

PLUM-Verification Predicate. Fix $\text{pp}_{\text{Plum}} \leftarrow \text{Plum.Setup}(1^\lambda)$. For $\text{pk} \in \mathcal{PK}_{\text{Plum}}$, $m \in \mathcal{M}_{\text{Plum}}$, and $\sigma \in \Sigma_{\text{Plum}}$, define

$$\text{Pred}_{\text{Verify}}^{\text{Plum}, \text{pp}_{\text{Plum}}}(\text{pk}, m, \sigma) = 1 \iff \text{Plum.Verify}(\text{pp}_{\text{Plum}}, \text{pk}, m, \sigma) = 1.$$

When this predicate is encoded as a zkSNARK relation, we can take, for example,

$$R_{\text{Plum}}((\text{pk}, m); \sigma) = 1 \iff \text{Pred}_{\text{Verify}}^{\text{Plum}, \text{pp}_{\text{Plum}}}(\text{pk}, m, \sigma) = 1,$$

where $x = (\text{pk}, m)$ is the public instance and $w = \sigma$ is the witness.

Quantitative Comparison to Loquat. At the same 128-bit security level and with both schemes instantiated using Griffin as the algebraic hash, PLUM reduces signature size from 57 KB to 37 KB, reduces the verification circuit from about 148,825 (Loquat [6]) to 116,285 R1CS constraints, and is expected to decrease signing and verification time by up to a factor of four, according to the analysis and parameter choices of [7]. These improvements come from (i) using a t -th power residue PRF to halve the polynomial degrees in the sumcheck relation, (ii) replacing FRI with the lower-query-complexity STIR protocol, and (iii) choosing a prime p that simultaneously supports the PRF and the smooth multiplicative subgroups required by STIR, avoiding the field-lifting overhead present in Loquat. These are *static R1CS* advantages. Section 7 shows they do not translate cleanly into zkVM proving cost, where the field-mismatch tax on Griffin (the extra cost of emulating its wide-field arithmetic on the small prover field; defined below) dominates.

3.4 zkSNARK

Let R be an efficiently decidable relation and let

$$L_R = \{x : \exists w \text{ such that } R(x; w) = 1\}$$

be the corresponding language. A zkSNARK for R is a tuple

$$\Pi = (\Pi.\text{Setup}, \Pi.\text{Prove}, \Pi.\text{Verify}).$$

Two independent axes matter here, *transparency* (whether $\Pi.\text{Setup}$ needs a relation-specific trusted setup) and *universality* (whether the proving key/preprocessing is independent of the concrete relation R). Update churn is driven by the latter. A transparent but relation-specific argument such as Aurora still re-derives relation-dependent parameters on every change. We first state the universal/relation-independent case, in which the setup is independent of the concrete relation:

$$\text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda).$$

If a relation-specific proof system is used instead, setup should be written as $\text{crs}_R \leftarrow \Pi.\text{Setup}(1^\lambda, R)$. The proving and verification algorithms are

$$\pi \leftarrow \Pi.\text{Prove}(\text{crs}, x, w), \quad b \leftarrow \Pi.\text{Verify}(\text{crs}, x, \pi) \in \{0, 1\}.$$

Completeness. Completeness requires that for every (x, w) such that $R(x; w) = 1$,

$$\Pr[\Pi.\text{Verify}(\text{crs}, x, \Pi.\text{Prove}(\text{crs}, x, w)) = 1] \geq 1 - \text{negl}(\lambda).$$

Soundness. Soundness requires that for every adversary $\mathcal{A}$,

$$\Pr[x \notin L_R \wedge \Pi.\text{Verify}(\text{crs}, x, \pi) = 1 : \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda), (x, \pi) \leftarrow \mathcal{A}(\text{crs})] \leq \text{negl}(\lambda).$$

Knowledge soundness (argument of knowledge). A property strictly stronger than soundness, and the one a higher-level reduction needs when it must *extract* a witness rather than merely conclude that one exists. The argument is *knowledge-sound* if there is a probabilistic polynomial-time extractor $\mathcal{E}$ such that, for every prover $\mathcal{A}$ producing an accepting proof for a statement x with non-negligible probability, $\mathcal{E}$, given $\mathcal{A}$'s code and randomness (or rewinding/oracle access), outputs a witness w with $R(x; w) = 1$, except with negligible *knowledge error*. BDEC's unforgeability reduction relies on this property of Π to pull a credential and its signatures out of an accepting showing. Plain soundness, which only certifies that a witness exists, does not suffice. The formal extractor-and-knowledge-error statement is Definition A.1 (Appendix A.1).

Zero-knowledge. Zero-knowledge requires that there exists a simulator Sim such that, for every x and w satisfying $R(x; w) = 1$, the distributions

$$\{(\text{crs}, x, \pi) : \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda), \pi \leftarrow \Pi.\text{Prove}(\text{crs}, x, w)\}$$

and

$$\{(\text{crs}, x, \tilde{\pi}) : \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda), \tilde{\pi} \leftarrow \text{Sim}(\text{crs}, x)\}$$

are computationally indistinguishable. Unless stated otherwise this is computational zero-knowledge with a simulator in the classical random-oracle model. Whether a given instantiation's simulator extends to the QROM is flagged where the distinction affects the receipt-mode claims of Section 5.

3.4.1 Aurora: Transparent Succinct Arguments for R1CS

Aurora [32] is a transparent zkSNARK for the rank-1 constraint system (R1CS) relation. Given an R1CS instance (A, B, C) over a finite field $\mathbb{F}$ and a candidate assignment $z \in \mathbb{F}^n$ satisfying the Hadamard identity

$$(Az) \odot (Bz) = Cz,$$

Aurora produces a succinct, publicly verifiable, zero-knowledge proof of the existence of a satisfying witness, with polylogarithmic proof size and linear verifier complexity in n . Aurora is transparent in the sense that it requires no relation-specific trusted setup beyond a collision-resistant hash function. The Aurora reference in this thesis is measured in both modes: with zero-knowledge disabled (`zk=false`), a mode that bounds proving cost only and says nothing about anonymity, and with zero-knowledge enabled. The measurements, and the security level they achieve, are reported in Section 7.

When instantiated for a fixed relation R , the R1CS encoding fixes the matrices (A, B, C) at compile time. Any change to R that affects which gates appear in the verification predicate requires recompilation of these matrices and re-derivation of any parameters that depend on the constraint count. This is the property that motivates the zkVM approach. The underlying relation is fixed to the ISA semantics once, and program changes are expressed inside the witness rather than in the matrices.

The BDEC construction of [5] instantiates its zkSNARK Π with Aurora over the R1CS encoding of the credential-issuance and credential-showing relations defined later in this section. The dominant component of proof generation in this regime is Aurora's low-degree-test (LDT) sub-protocol. Verification is instead dominated by the verifier's linear pass over the explicit R1CS matrices [32, Thm. 1.2].

3.5 Anonymous Credential

Definition 3.6 (Anonymous credential). Let λ be a security parameter, let Attr be an attribute space, and let Φ be a class of predicates over one or more attribute vectors. An anonymous credential scheme is a tuple of algorithms or interactive protocols

$$\text{AC} = (\text{AC.Setup}, \text{AC.UserKeyGen}, \text{AC.IssuerKeyGen}, \text{AC.Issue}, \text{AC.Show}, \text{AC.Verify}).$$

Here $\text{pp} \leftarrow \text{AC.Setup}(1^\lambda)$ are the public parameters, $(\text{usk}, \text{upk}) \leftarrow \text{AC.UserKeyGen}(\text{pp})$ a user key pair, and $(\text{isk}, \text{ipk}) \leftarrow \text{AC.IssuerKeyGen}(\text{pp})$ an issuer key pair, as in the correctness statement below. For an attribute vector $A = (a_1, \dots, a_m) \in \text{Attr}^m$, issuance gives a user a credential cred . For a predicate $\phi \in \Phi$, the showing algorithm outputs a public statement and proof

$$(\text{stmt}, \pi) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}, \text{usk}, \text{ipk}, A, \phi).$$

The statement contains intentionally disclosed information. The proof proves membership in a relation

$$R_{\text{AC}}(\text{stmt}; w) = 1,$$

where the witness contains at least $w = (\text{cred}, \text{usk}, A)$ and the relation checks that cred is valid and that $\phi(A) = 1$.

Correctness. Correctness requires that honestly generated credentials and honestly generated showing proofs are accepted by the verifier. More formally, for every λ , every

$$\text{pp} \leftarrow \text{AC.Setup}(1^\lambda),$$

every honestly generated user key pair and issuer key pair

$$(\text{usk}, \text{upk}) \leftarrow \text{AC.UserKeyGen}(\text{pp}), \quad (\text{isk}, \text{ipk}) \leftarrow \text{AC.IssuerKeyGen}(\text{pp}),$$

every honestly issued credential

$$\text{cred} \leftarrow \langle \text{User}(\text{pp}, \text{usk}, \text{upk}, A), \text{Issuer}(\text{pp}, \text{isk}, \text{ipk}, \text{upk}, A) \rangle.$$

and every predicate $\phi \in \Phi$, if $\phi(A) = 1$, then

$$\Pr[\text{AC.Verify}(\text{pp}, \text{ipk}, \text{stmt}, \pi) = 1] \geq 1 - \text{negl}(\lambda),$$

where

$$(\text{stmt}, \pi) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}, \text{usk}, \text{ipk}, A, \phi).$$

The probability is taken over the randomness of the setup, key-generation, issuance, showing, and verification algorithms.

Unforgeability. Unforgeability requires that no adversary can produce an accepting showing proof unless it knows a credential honestly issued by an issuer and a corresponding user secret key satisfying the statement. Let $\mathcal{L}$ be the issuance log containing all tuples

$$(\text{ipk}, \text{upk}, A, \text{cred})$$

honestly issued during the experiment. The adversary may interact with the issuance oracle $\mathcal{O}_{\text{Issue}}$ and the showing oracle $\mathcal{O}_{\text{Show}}$, and then outputs $(\text{ipk}^*, \text{stmt}^*, \pi^*)$. The adversary wins if

$$\text{AC.Verify}(\text{pp}, \text{ipk}^*, \text{stmt}^*, \pi^*) = 1$$

and there is no tuple

$$(\text{ipk}^*, \text{upk}^*, A^*, \text{cred}^*) \in \mathcal{L}$$

and no user secret key usk^* such that $(\text{usk}^*, \text{upk}^*)$ is an honestly generated user key pair such that, for

$$w^* = (\text{cred}^*, \text{usk}^*, A^*),$$

we have

$$R_{\text{AC}}(\text{stmt}^*; w^*) = 1.$$

We write $\text{Adv}_{\text{AC}}^{\text{unf}}(\mathcal{A})$ for the probability that $\mathcal{A}$ wins this experiment. The scheme is unforgeable if $\text{Adv}_{\text{AC}}^{\text{unf}}(\mathcal{A}) \leq \text{negl}(\lambda)$ for every probabilistic polynomial-time $\mathcal{A}$.

Anonymity. Anonymity requires that a showing proof does not reveal which honest user generated it, beyond the information intentionally disclosed in the public statement. This is a cryptographic indistinguishability notion. Non-cryptographic side channels, such as wall-clock timing, network and ledger-access patterns, revocation-update timing, and issuer or wallet metadata observed out of band, are outside the model and are not covered by the bound below. In the anonymity experiment, the adversary designates two honestly issued credentials by their entries in the issuance log,

$$(\text{cred}_0, A_0), \quad (\text{cred}_1, A_1),$$

whose users' secret keys $\text{usk}_0, \text{usk}_1$ remain with the challenger (which runs the showings below), and a predicate ϕ , such that both credentials satisfy the predicate,

$$\phi(A_0) = \phi(A_1) = 1,$$

and the corresponding public statements reveal the same allowed leakage. More formally, if

$$(\text{stmt}_0, \pi_0) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}_0, \text{usk}_0, \text{ipk}, A_0, \phi)$$

and

$$(\text{stmt}_1, \pi_1) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}_1, \text{usk}_1, \text{ipk}, A_1, \phi),$$

then the challenge is well formed only if

$$\text{Leak}(\text{stmt}_0) = \text{Leak}(\text{stmt}_1),$$

where $\text{Leak}(\text{stmt})$ is a scheme-specified projection of the public statement onto its intentionally-disclosed components; its instantiation for BDEC is fixed with the construction below. Statement components outside the projection are either freshly sampled at showing time or witness-derived and made independent of the challenge bit by the simulators of the anonymity argument (Appendix A). The challenger samples $b \leftarrow \{0, 1\}$ and returns

$$(\text{stmt}, \pi_b) \leftarrow \text{AC.Show}(\text{pp}, \text{cred}_b, \text{usk}_b, \text{ipk}, A_b, \phi).$$

The adversary outputs b' . We write $\text{Adv}_{\text{AC}}^{\text{anon}}(\mathcal{A}) = |\Pr[b' = b] - \frac{1}{2}|$. The scheme is anonymous if $\text{Adv}_{\text{AC}}^{\text{anon}}(\mathcal{A}) \leq \text{negl}(\lambda)$ for every probabilistic polynomial-time $\mathcal{A}$.

Unlinkability. Unlinkability requires that a shown credential cannot be linked to the pseudonym under which it was issued, unless the public statements intentionally contain a linking tag. In the unlinkability experiment, with the oracle interface of the unforgeability experiment, the challenger creates two pseudonyms $\text{ppk}^0, \text{ppk}^1$, freshly and identically sampled, each with its user-side binding signature psk^i , and an attribute set $A^\downarrow$; it samples $b \leftarrow \{0, 1\}$, issues a single credential $c^{(b)}$ with respect to $\text{ppk}^{(b)}$ on $A^\downarrow$, and forwards to the adversary the *showing receipt* produced from it (the public statement and proof, not the raw credential). The pseudonyms are internal to the challenger, their only exposure being through the challenge receipt; this withheld-pseudonym view follows base BDEC's experiment, whose challenge forwards only the raw credential, so the receipt-forwarding form here gives the adversary strictly more. The adversary outputs b' . We write $\text{Adv}_{\text{AC}}^{\text{unlink}}(\mathcal{A}) = |\Pr[b' = b] - \frac{1}{2}|$, and the scheme is unlinkable if it is $\text{negl}(\lambda)$ for every probabilistic polynomial-time $\mathcal{A}$. When the statements intentionally contain a public linking tag, linkability is allowed only as determined by that tag.

3.5.1 Blockchain-based Digital Education Credentials (BDEC)

BDEC is a blockchain-based anonymous credential system for educational credentials. It is parameterized by a digital signature scheme Sig and a zkSNARK Π . The main parties are a user U , teaching authorities TA , and a verifier V .

The blockchain stores credential-record digests, authorized teaching-authority keys, and revocation state. We assume that H and H_{rev} are collision-resistant and domain-separated hash functions; we reuse the symbol H for both the two-to-one Merkle compression of Definition 3.3 and the variable-arity record digests below, the sponge construction absorbing arbitrary-length input and domain separation keeping the uses distinct. In the privacy-preserving case, the blockchain stores record digests ρ rather than raw credential attributes.

Construction 1. BDEC instantiates the minimal anonymous-credential syntax introduced above with the signature scheme Sig and the zkSNARK Π . The interface maps onto that syntax as follows: AC.Setup is BDEC.Setup ; AC.UserKeyGen and AC.IssuerKeyGen are Sig.KeyGen for users and teaching authorities respectively; AC.Issue is the issuance step below (pseudonym binding, teaching-authority signature, ledger record), with CreGen/CreVer the user-side proof of the issued credential; AC.Show is ShowCre and AC.Verify is ShowVer , with $\text{stmt} = x_{\text{show}}$ and $R_{\text{AC}} = R_{\text{show}}^{\text{Sig}}$. Π proves relations containing signature-verification predicates, while the ledger-membership, authorized-TA, and revocation checks are performed by the verifier *outside* the proof against the published roots (see BDEC.ShowVer).

below). We give the two proof relations this thesis measures ($R_{\text{cre}}^{\text{Sig}}$ and $R_{\text{show}}^{\text{Sig}}$) and the verifier-side checks in full. The key-generation, issuance, and verification algorithms follow base BDEC [5] and are summarised next.

Setup, keys, pseudonyms, and issuance. BDEC.Setup runs Sig.Setup and Π .Setup to fix $\text{par} = (\text{pp}_{\text{Sig}}, \text{crs})$, and at epoch e publishes the authenticated roots $\text{st}_e = (\text{rt}_{\text{cred},e}, \text{rt}_{\text{TA},e}, \text{rt}_{\text{LR},e})$ (the accepted credential-record, authorized-TA, and revocation roots). User and teaching-authority keys are sampled with Sig.KeyGen. A pseudonym $\text{ppk}_{U,\text{TA}}$ is bound to the user’s key by a user-side signature $\text{psk}_{U,\text{TA}} \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_U, \text{Encode}_{\text{Sig}}(\text{ppk}_{U,\text{TA}}))$. On issuance the teaching authority signs the attributes, $\sigma_{\text{cred},U,\text{TA}} \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_{\text{TA}}, \text{Encode}_{\text{Sig}}(\text{pk}_U, \text{ppk}_{U,\text{TA}}, A, \text{aux}))$, and the ledger stores only the record digest $\rho_{U,\text{TA}} = H(\text{pk}_{\text{TA}}, \text{ppk}_{U,\text{TA}}, A, \text{aux}, \sigma_{\text{cred},U,\text{TA}})$, not the attributes in the clear. These steps follow base BDEC [5]. The two relations this thesis measures are given next.

BDEC.CreGen. On input $(\text{par}, \text{sk}_U, \text{pk}_U, \text{ppk}_{U,\text{TA}}, \text{psk}_{U,\text{TA}}, A, \text{aux})$, the algorithm first computes the attribute digest $h_{U,\text{TA}} = H(A)$ and the credential $c_{U,\text{TA}} \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_U, h_{U,\text{TA}})$. It then generates a proof with respect to the zkSNARK statement and witness. In a zkVM instantiation the public-instance components x_{cre} below must be committed to the receipt journal (the receipt structure is defined at the end of this section) for the proof to bind to them. This is a statement-binding requirement on which soundness depends, and its status in the prototype is treated in Section 5,

$$\begin{aligned} x_{\text{cre}} &= (c_{U,\text{TA}}, h_{U,\text{TA}}, \text{ppk}_{U,\text{TA}}), \\ w_{\text{cre}} &= (\text{pk}_U, \text{psk}_{U,\text{TA}}), \end{aligned}$$

where the proved relation is the conjunction of two signature verifications, both under the user’s long-term public key pk_U :

$$\begin{aligned} R_{\text{cre}}^{\text{Sig}}(x_{\text{cre}}; w_{\text{cre}}) = 1 &\iff \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{pp}_{\text{Sig}}}(\text{pk}_U, h_{U,\text{TA}}, c_{U,\text{TA}}) = 1 \\ &\wedge \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{pp}_{\text{Sig}}}(\text{pk}_U, \text{Encode}_{\text{Sig}}(\text{ppk}_{U,\text{TA}}), \text{psk}_{U,\text{TA}}) = 1. \end{aligned}$$

Equivalently, the circuit proved by the zkSNARK is the AND concatenation of two signature-verification algorithms [5]. The first attests that $c_{U,\text{TA}}$ is a valid credential on the attribute digest, the second that the user controls the pseudonym $\text{ppk}_{U,\text{TA}}$. Compute

$$\varepsilon_{c_{U,\text{TA}}} \leftarrow \Pi.\text{Prove}(\text{crs}, x_{\text{cre}}, w_{\text{cre}})$$

and output $(c_{U,\text{TA}}, \varepsilon_{c_{U,\text{TA}}})$.

Remark 3.7 (Optional Merkle accumulator; not part of the base relation). The base $R_{\text{cre}}^{\text{Sig}}$ above is exactly the two-verification relation of [5], for which that work’s unforgeability and anonymity theorems are proved. To additionally *hide* attributes at presentation time, the same work suggests an optional Merkle-accumulator variant. The user signs the Merkle *root* of the attributes in BDEC.CreGen rather than $H(A)$, and the membership (authentication-path) checks are performed by the verifier in BDEC.ShowVer, not inside $R_{\text{cre}}^{\text{Sig}}$. This thesis implements and measures the base two-verification relation. The accumulator variant, and any consequent re-derivation of the security reductions for an enlarged relation, are out of scope and would constitute a strengthening beyond the cited construction rather than a re-instantiation of it.

BDEC.CreVer. On input $(\text{par}, \text{st}_e, x_{\text{cre}}, \varepsilon_{c_{U,TA}})$, output

$$b \leftarrow \Pi.\text{Verify}(\text{crs}, x_{\text{cre}}, \varepsilon_{c_{U,TA}}) \in \{0, 1\}.$$

BDEC.ShowCre. On input $(\text{par}, \text{st}_e, \text{pk}_U, (\text{ppk}_{U,TA}^{(j)}, \text{psk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V}, \text{psk}_{U,V}, c_{U,V}, A^\downarrow, \phi)$, where $A^\downarrow$ are the selectively disclosed attributes, ϕ the claimed presentation predicate, and k the number of accepted credential records used, the algorithm forms the public instance

$$x_{\text{show}} = (A^\downarrow, (\text{ppk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V}).$$

The private witness is

$$w_{\text{show}} = (\text{pk}_U, (\text{psk}_{U,TA}^{(j)})_{j=1}^k, \text{psk}_{U,V}, c_{U,V}).$$

Following the base relation of [5], the witness is the user's long-term verification key pk_U , used here as a *private* witness despite being a public key, together with the pseudonym-binding signatures and the shown credential. The showing-time inputs are formed by the holder, who holds sk_U : the verifier-facing pseudonym binding $\text{psk}_{U,V} \leftarrow \text{Sig.Sig}(\text{pp}_{\text{Sig}}, \text{sk}_U, \text{Encode}_{\text{Sig}}(\text{ppk}_{U,V}))$ for the freshly sampled $\text{ppk}_{U,V}$, and the shown credential $c_{U,V} \leftarrow \text{Sig.Sig}(\text{pp}_{\text{Sig}}, \text{sk}_U, \text{Encode}_{\text{Sig}}(A^\downarrow))$, the user's signature on the encoded disclosed attributes (the presentation analogue of $c_{U,TA}$; the encoding m_{show} is fixed below). Hiding pk_U is what delivers anonymity, since the showing is then not tied to the user's identity. The pseudonym public keys and the disclosed attributes $A^\downarrow$ are public, so the messages verified inside the proof are formed from public values and the verifier evaluates ϕ on $A^\downarrow$. For the anonymity experiment above, BDEC's leakage projection is $\text{Leak}(x_{\text{show}}) = (A^\downarrow, \{\text{ppk}_{U,TA}^{(j)}\}, \text{any linking tag carried in } \phi)$; the verifier-facing pseudonym $\text{ppk}_{U,V}$, freshly sampled per showing, is outside the projection. Membership, non-revocation, and the disclosure check are performed by the verifier in BDEC.ShowVer, which additionally takes the disclosed predicate ϕ , the published roots $\text{rt}_{\text{cred},e}$, $\text{rt}_{TA,e}$, $\text{rt}_{LR,e}$, the credential commitments $(\rho^{(j)})_{j=1}^k$ (the per-credential record digests, $\rho^{(j)} = \rho_{U,TA(j)}$), the membership paths, and the revocation tag r_U (defined below). These are verifier-side inputs and are not part of the proof statement or witness. Concealing the attributes outside $A^\downarrow$ is the optional Merkle-accumulator strengthening of Remark 3.7, which this thesis does not implement.

Concrete presentation predicates. The predicate class Φ is the set of Boolean combinations of arithmetic comparisons and equalities over the attribute values of one or more shown credentials (the arity matching the credential count k). Two predicates serve as running examples throughout the thesis and as the workload for the update-churn measurement of Section 7. The first, used at credential count $k = 1$, is a single-attribute threshold. Writing $A.\text{gpa}$ for the grade-point attribute,

$$\phi_1(A) = [A.\text{gpa} > 3.5].$$

The second, used at $k = 2$, is a conjunction over two credentials drawn from possibly distinct issuers. Writing $A.\text{degree}$, $A.\text{gradYear}$, and $A.\text{major}$ for the relevant attributes,

$$\phi_2(A^{(1)}, A^{(2)}) = [A^{(1)}.\text{degree} = \text{Bachelor}] \wedge [A^{(1)}.\text{gradYear} > 2020] \wedge [A^{(2)}.\text{major} = \text{CS}].$$

Both predicates are evaluated by the verifier on the disclosed attribute set $A^\downarrow$, *outside* the proof relation $R_{\text{show}}^{\text{Sig}}$. Following base BDEC (Remark 3.7), the attributes named in ϕ are disclosed in the

clear and the verifier checks $\phi(A^\downarrow) = 1$ on the revealed values. For ϕ_1 the GPA is disclosed and the verifier checks the threshold. For ϕ_2 the degree, graduation year, and major are disclosed and the verifier checks the conjunction. The privacy the showing provides is *anonymity*, in that the proof is not linked to the user's identity pk_U , together with unlinkability on the verifier-facing pseudonym $\text{ppk}_{U,V}$, which is freshly sampled per showing. It is not blanket unlinkability across all showings: two showings that disclose the same teaching-authority pseudonym $\text{ppk}_{U,TA}$ remain linkable by that intentionally-disclosed handle, the conditional linkability base BDEC provides by design (Section 6). It is *not* concealment of the disclosed attribute *values*. Proving a predicate over an attribute that stays hidden (e.g. “gpa > 3.5” without revealing the GPA) is beyond the base relation and would require the optional Merkle-accumulator strengthening that Remark 3.7 places out of scope. The two predicates differ in arity ($k = 1$ versus $k = 2$), in issuer multiplicity (one issuer versus two), and in Boolean structure (atomic versus a three-clause conjunction). Because ϕ_1 and ϕ_2 require a *different number of credentials*, moving between them changes the presentation *structure*, namely the credential count k , and hence the relation $R_{\text{show}}^{\text{Sig}}$ itself, which is the source of the static-circuit recompilation cost measured in Section 7. A policy change at fixed structure (e.g. retuning the GPA threshold) is absorbed entirely by the verifier-side check and forces no recompilation. For the threat model of Section 6, the timing of ϕ matters. It is chosen by the verifier at presentation time and is not known when the credential is issued.

For each $j \in [k]$, define $m_{\text{nym}}^{(j)} = \text{Encode}_{\text{Sig}}(\text{ppk}_{U,TA}^{(j)})$. Also define $m_{\text{nym},U,V} = \text{Encode}_{\text{Sig}}(\text{ppk}_{U,V})$ and $m_{\text{show}} = \text{Encode}_{\text{Sig}}(A^\downarrow)$, the encoding of the disclosed attributes carried by the shown credential $c_{U,V}$. The showing relation is

$$\begin{aligned} R_{\text{show}}^{\text{Sig}}(x_{\text{show}}; w_{\text{show}}) = 1 &\iff \bigwedge_{j=1}^k \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}_U, m_{\text{nym}}^{(j)}, \text{psk}_{U,TA}^{(j)}) = 1 \\ &\quad \wedge \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}_U, m_{\text{nym},U,V}, \text{psk}_{U,V}) = 1 \\ &\quad \wedge \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{ppSig}}(\text{pk}_U, m_{\text{show}}, c_{U,V}) = 1. \end{aligned}$$

Following BDEC [5], the proof circuit contains $k + 2$ signature-verification predicates, namely k pseudonym-ownership checks (each binding a teaching-authority pseudonym to pk_U), one verifier-facing pseudonym check, and one check on the shown credential $c_{U,V}$ over the disclosed attributes. The relying party additionally checks, *outside* the proof, accepted-ledger membership and authorised-TA membership against the published roots, non-revocation against the revocation tree, and that the disclosed subset $A^\downarrow$ satisfies the presentation predicate ϕ . Selective disclosure is realised by the chosen subset $A^\downarrow$. No in-circuit predicate enforces it. Compute

$$\varepsilon_{c_{U,V}} \leftarrow \Pi.\text{Prove}(\text{crs}, x_{\text{show}}, w_{\text{show}})$$

and output $\varepsilon_{c_{U,V}}$.

BDEC.ShowVer. On input $(\text{par}, \text{st}_e, x_{\text{show}}, \varepsilon_{c_{U,V}})$, output

$$b \leftarrow \Pi.\text{Verify}(\text{crs}, x_{\text{show}}, \varepsilon_{c_{U,V}}) \in \{0, 1\}.$$

The verifier first checks the zkSNARK proof against the public statement, which attests the $k + 2$ signature-verification predicates of $R_{\text{show}}^{\text{Sig}}$. The relying party then performs the ledger-membership,

authorised-TA, non-revocation, and selective-disclosure checks *outside* the proof, against the published roots, the revocation tree, and the disclosed subset $A^\downarrow$, consistent with the relation above.

Revocation, conditional linkability, and revocability. Base BDEC additionally provides revocation (a tag $r_U = H_{\text{rev}}(\text{pk}_U)$ checked for non-membership against the revocation root rt_{LRe}), conditional linkability (an optional user-chosen public tag τ that links two showings iff $\tau_0 = \tau_1 \neq \perp$, and otherwise reduces to unlinkability), and the corresponding revocability guarantee [5]. These are verifier-side or deployment-level features that the measurements of this thesis do not exercise, and are out of scope here.

3.6 Zero-Knowledge Virtual Machines

A zero-knowledge virtual machine (zkVM) is a zkSNARK whose relation is fixed once, to the correctness of executing a guest program on a deterministic instruction-set architecture. In contrast to a relation-specific zkSNARK such as Aurora, where the rank-1 constraint system encoding a relation R is compiled together with the proof system, a zkVM commits to a single universal relation R_{vm} describing its instruction-set semantics, and treats the guest program as an input to that relation. Background surveys and step-by-step expositions of this paradigm are given in [60, 61].

Definition 3.8 (Zero-knowledge virtual machine). Let ISA be a deterministic instruction-set architecture with a fixed cycle-level semantics. A zkVM for ISA is a zkSNARK

$$\Pi_{\text{vm}} = (\Pi_{\text{vm}}.\text{Setup}, \Pi_{\text{vm}}.\text{Prove}, \Pi_{\text{vm}}.\text{Verify})$$

for the universal relation

$$R_{\text{vm}}((\text{img}, \text{in}_{\text{pub}}, \text{out}); (P, \text{in}_{\text{priv}})) = 1$$

iff $\text{img} = \text{digest}(P)$ and executing the guest program P on ISA with inputs $(\text{in}_{\text{pub}}, \text{in}_{\text{priv}})$ terminates and emits public output out . The public instance is $x_{\text{vm}} = (\text{img}, \text{in}_{\text{pub}}, \text{out})$ and the witness is $w_{\text{vm}} = (P, \text{in}_{\text{priv}})$.

Host-guest model. A zkVM partitions execution into a *guest* program that runs on the virtual processor under the proof system, and a *host* program that runs natively on the prover machine. The host orchestrates proof generation, supplies inputs to the guest via a read channel, and receives the guest’s public output via a commit channel. Values placed on the commit channel become part of the public statement out . Values placed on the read channel that the guest does not commit remain in the witness in_{priv} . In the BDEC instantiation considered in this thesis, the user’s long-term secret key sk_U and the credential signatures appear only via the read channel and are never committed, so they remain in w_{vm} across the entire zkSNARK relation.

Receipt structure and the zero-knowledge property. The output of $\Pi_{\text{vm}}.\text{Prove}$ is a *receipt* consisting of a journal of committed outputs together with a succinct proof. Concrete instantiations distinguish two regimes:

- A *succinct STARK receipt* (STARK, for Scalable Transparent ARgument of Knowledge) is a proof from a transparent, hash-based proof system, needing no trusted setup and plausibly post-quantum secure. It is succinct in the sense that it verifies in time logarithmic in the number of executed cycles, while its size remains large relative to the constant-size wrapped receipt below. Whether such a receipt satisfies the formal zero-knowledge property defined with the zkSNARK above depends on the specific instantiation. In the systems we evaluate the default succinct STARK receipt is not formally zero-knowledge.
- A *SNARK-wrapped receipt* is obtained by composing the succinct STARK with an outer zero-knowledge SNARK (e.g. Groth16 or PLONK). The wrapped receipt is small and zero-knowledge, but the wrap step itself has additional prover memory and time cost.

Any privacy claim about an anonymous-credential scheme instantiated on a zkVM that relies on the underlying zkSNARK being zero-knowledge therefore holds only for the wrapped receipt. We make this dependency explicit when stating anonymity and unlinkability of the BDEC instantiation in Section 6.

Execution trace, chips, and shards. To prove a guest execution, the prover records it as tables of field elements, one per component of the virtual machine (the CPU, memory, and each precompile, among others). Each component, together with the constraints that govern its table, is a *chip*; the table the chip fills is its *execution trace*: a row is one step or invocation of the component and a column a piece of its state. The prover commits to every chip’s trace, and total proving cost scales quasi-linearly with the number of committed cells summed across all chips, a quantity we call the total *trace area*. A precompile contributes a chip of its own, trading its fixed block of rows against the general-purpose CPU rows its syscall would otherwise occupy. For an execution too long to prove in one piece, the prover splits it into fixed-size segments called *shards*, proves each separately, and combines them recursively into the single receipt defined above. The terms *trace area*, *chip*, and *shard* recur in the cost model of Section 4 and the measurements of Section 7.

Precompiles. A *precompile* is a fixed sub-circuit recognised by the zkVM’s instruction decoder as a syscall and proved natively by the prover rather than via emulated instruction-level execution. From the guest’s perspective a precompile call costs a single syscall instruction. From the prover’s perspective the cost is the proof of one execution of the precompile’s underlying AIR (algebraic intermediate representation): the AIR is the chip’s constraint system, and the chip’s execution trace is the table those constraints govern. Precompiles trade proof-system generality for performance on algebraic primitives whose software emulation over the prover’s native field would otherwise dominate the trace. A precompile is sound only if (i) its AIR enforces exactly the intended function, accepting no more inputs than the reference specification, and (ii) its memory inputs and outputs are bound to the surrounding VM execution trace via the cross-table lookup argument. Otherwise the syscall could be satisfied by values unrelated to the guest’s computation. This functional-equivalence-plus-trace-binding condition is addressed for the Griffin precompile in Section 6, its equivalence half as Assumption 6.1 and its binding half as Assumption 6.6, where their status is examined.

We refer to the cost of emulating n -bit field arithmetic in software when the underlying prover field is small as the *field-mismatch tax*. A precompile that implements the same arithmetic natively absorbs this tax. The Griffin- $\mathbb{F}_p^{192}$ precompile introduced and measured in this thesis is the central instance of this pattern.

Update churn. *Update churn* denotes the engineering and verification cost incurred when a deployed proof system must be changed in response to evolving policy, security parameters, or primitive substitutions. In a relation-specific zkSNARK every such change requires regenerating and re-auditing the relation-specific circuit and its parameters. This regeneration-and-re-audit footprint, rather than the recompile wall-clock (which can be small for a transparent argument), is the dominant cost. In a zkVM the same change is a guest-program edit. The universal relation R_{vm} and the trusted setup (if any) are unchanged. Update churn is the evaluation dimension we add to runtime cost in Section 7.

3.6.1 SP1

SP1 [8] is the zkVM instantiation this thesis measures, whose ISA is RV64IM (the guest target of the SP1 v6.2.1 fork evaluated here) and whose prover is SP1’s `slop` multilinear-commitment stack (Basefold and jagged polynomial commitments) over the KoalaBear field ($2^{31} - 2^{24} + 1$), one of a family of small-characteristic fields, including BabyBear, Goldilocks, and Mersenne-31 (the field of Circle STARKs [62]), that modern prover toolkits such as Plonky3 [63] support. Per the SP1 documentation, the default prove path produces a succinct STARK receipt that is not formally zero-knowledge. A zero-knowledge receipt is obtained by wrapping the STARK in a SNARK (the `groth16()` or `plonk()` proof modes in the SP1 SDK, which internally compress the STARK before wrapping). SP1 supports application-defined precompiles via its custom-precompile framework.

3.6.2 Static-circuit and zkVM proof systems as comparators

We will instantiate BDEC in two regimes that this thesis compares directly:

1. A *static-circuit* regime, in which the CreGen and ShowCre relations are encoded into a fixed R1CS and proved via Aurora (Section 3 above). Each change to the relation requires recompilation of the R1CS.
2. A *zkVM* regime, in which the same relations are expressed as guest programs on SP1. Changes to the relation become guest-program edits. The universal relation R_{vm} is unchanged.

The runtime cost, memory footprint, and update-churn properties of these two regimes are the empirical subject of Section 7.

4 The Field-Mismatch Tax and the Cost Criterion

This section sets up the formal cost framework used to decide whether a precompile recovers feasibility and at what prove-time cost. We name the dimensions along which the static-circuit and zkVM regimes will be compared (Section 4.1), formalise the *field-mismatch tax* as the quantitative manifestation of the arithmetic mismatch (Section 4.2), develop the criterion that prices a precompile against that tax (Section 4.3), formalise *update churn* as a deployment-lifecycle dimension (Section 4.4), and decompose the cost of the verification predicate to locate the precompile target (Section 4.5). Automated precompile synthesis and pricing (powdr’s autoprecompiles [45] and the vApps analysis [46]) rank or reprice accelerators from profiling data after the fact. The criterion developed here (Section 4.3) instead decides, from the specification and before any build, whether a native precompile lowers total trace area. The soundness obligation such a precompile must meet is that it computes the same function as the reference and its output is tied to the proof, stated with the precompile definition (Section 3) and examined in Section 6.

4.1 Cost Dimensions

We measure each system regime along four dimensions.

Time. Prover and verifier wall-clock time, measured in seconds on the target hardware. For the prover this is the time from invocation of $\Pi.\text{Prove}$ (or $\Pi_{\text{vm}}.\text{Prove}$) until the receipt is produced. For the verifier it is the time from invocation of the verification predicate until the binary outcome is returned.

Memory. Peak resident-set size (peak RSS) of the prover process on the target hardware, measured in gigabytes. Verifier memory is reported when it materially differs from prover memory. Otherwise it is dominated by the receipt size.

Proof size. The size of the produced receipt, measured in kilobytes for the relevant receipt class. Of the two receipt regimes of Section 3, the succinct STARK receipt and the SNARK-wrapped receipt, we report the size of whichever is the deployment-relevant figure for the comparison at hand. Where the wrap is the deployable artefact but is not produced on the target hardware, its constant size is cited from the toolchain documentation and the workload-dependent core receipt is not serialised (Section 7).

Update churn. The footprint of artefacts that must be regenerated and re-audited when the verification predicate changes by one localised modification. We formalise this dimension in Section 4.4 below.

The first three dimensions are standard in the zkSNARK and zkVM literature [6, 7, 38, 8, 39]. The fourth dimension is introduced in this thesis as a deployment-lifecycle accounting frame. Unlike the first three it is a footprint characterisation rather than a performance metric, and Section 7 shows that it does not translate into a wall-clock advantage against the non-preprocessing static baseline. That baseline (such as Aurora) rebuilds its per-relation pieces cheaply at each change, rather than through a large one-time setup (Section 2).

4.2 The Field-Mismatch Tax

This section targets one of two distinct field mismatches. The *protocol-level* mismatch, Loquat’s extension-field $\mathbb{F}_{p^2}$ arithmetic, is already eliminated by PLUM’s single-prime design. What remains, and what this section formalises, is the *substrate-level* mismatch, PLUM’s large prime emulated over a zkVM’s small native field. The verification predicate of a post-quantum signature such as Loquat or PLUM is dominated by algebraic-hash (Griffin) permutations together with arithmetic over a prime field $\mathbb{F}_p$ of large characteristic (in PLUM, a nominal 192-bit / concrete 199-bit prime; see Section 3). Concretely, PLUM [7, §4.2] reports that hash evaluations account for 105,952 of the 116,285 R1CS constraints ($\approx 91\%$) of a PLUM-128 verification, the field-arithmetic constraints making up the remaining $\approx 9\%$. The tax below is carried by this hash-dominated, large-field workload as a whole, and not by field arithmetic on its own. This 91% is a constraint count in a relation-specific zkSNARK and is not a measured zkVM-cycle share. It does not, on its own, cap how much the zkVM cost can improve, and the measured zkVM-cycle attribution is reported separately in Section 7. This constraint figure is reported at the PLUM-128 parameter set, whereas every zkVM prove-mode measurement in this thesis is taken at $\lambda = 80$, the execute-mode cost-attribution analysis alone at $\lambda = 128$ (*execute mode* runs the guest and counts cycles without producing a proof; *prove mode* produces one; Section 7). When such a predicate is proved by a relation-specific zkSNARK over $\mathbb{F}_p$, every field operation in the predicate is a single constraint over the same field. When it is proved by a zkVM whose prover field $\mathbb{F}_{p_{\text{vm}}}$ is small (BabyBear, $p_{\text{vm}} = 2^{31} - 2^{27} + 1$, in RISC Zero; KoalaBear, $p_{\text{vm}} = 2^{31} - 2^{24} + 1$, in the evaluated SP1 fork), each $\mathbb{F}_p$ operation must be emulated by software running on the zkVM’s RISC-V instruction set, which in turn is encoded in $\mathbb{F}_{p_{\text{vm}}}$. The number of native RISC-V instructions required to emulate one $\mathbb{F}_p$ operation by multi-limb arithmetic is a fixed function of the bit-widths of p and the limb size of the underlying integer encoding.

The field-mismatch tax this definition names is characterised qualitatively in the recent zkVM systematisation [56]. The definition below is a cost-accounting frame that names, in one symbol, the per-operation overhead the precompile suite is built to remove.

Definition 4.1 (Field-mismatch tax). Let op be a field operation over $\mathbb{F}_p$ used inside a verification predicate Pred , and let $\text{cyc}_{\text{em}}(\text{op})$ denote the number of zkVM guest cycles required to emulate op by software on a zkVM whose native field is $\mathbb{F}_{p_{\text{vm}}}$. Let $\text{cyc}_{\text{nat}}(\text{op})$ denote the number of zkVM guest cycles that the same operation would consume if the prover field were $\mathbb{F}_p$ itself, so that no multi-limb emulation is needed; under this idealisation $\text{cyc}_{\text{nat}}(\text{op})$ is the cycle count of a single native field instruction. Because the target zkVMs expose no native $\mathbb{F}_p$ instruction, cyc_{nat} is a conceptual normaliser, not a directly measurable quantity; the empirically measured quantity is the emulated cost Cyc_{em} reported in Section 7. Both quantities are in the same unit, so their ratio is dimensionless. The *field-mismatch tax* of op is

$$\text{FMT}(\text{op}) = \frac{\text{cyc}_{\text{em}}(\text{op})}{\text{cyc}_{\text{nat}}(\text{op})}.$$

Writing $\text{ops}(\text{Pred})$ for the multiset of field operations executed by an honest evaluation of Pred , the cumulative *emulated cost* of Pred is the distinct quantity

$$\text{Cyc}_{\text{em}}(\text{Pred}) = \sum_{\text{op} \in \text{ops}(\text{Pred})} \text{cyc}_{\text{em}}(\text{op}).$$

The tax is not merely a name for a measured overhead. It admits a lower bound that depends only on the two field sizes. Write $\ell = \lceil \log_2 p / \lfloor \log_2 p_{\text{vm}} \rfloor \rceil$ for the number of $\mathbb{F}_{p_{\text{vm}}}$ limbs needed to represent one element of $\mathbb{F}_p$.

Proposition 4.2 (Field-mismatch lower bound). *On any zkVM whose native field is $\mathbb{F}_{p_{\text{vm}}}$ and which exposes no native $\mathbb{F}_p$ multiplication, an emulated $\mathbb{F}_p$ element occupies at least ℓ trace cells, and emulating one $\mathbb{F}_p$ multiplication by a limb-aligned multi-limb routine requires at least ℓ native $\mathbb{F}_{p_{\text{vm}}}$ multiplications. Hence*

$$\text{FMT}(\text{mult}) \geq \ell = \Omega\left(\frac{\log p}{\log p_{\text{vm}}}\right).$$

Schoolbook multi-limb multiplication attains $\Theta(\ell^2)$, so for a limb-aligned routine the realised tax per multiplication lies between ℓ and ℓ^2 ; a non-limb-aligned path, such as the general-bignum software arithmetic on the evaluated emulation path, lies outside the limb-aligned argument, is bounded below only by the ℓ -cell representation count, and can exceed ℓ^2 (Section 7).

Proof. Representation. An injective encoding of $\mathbb{F}_p$ into tuples of $\mathbb{F}_{p_{\text{vm}}}$ elements needs k components with $p_{\text{vm}}^k \geq p$, so any encoding uses at least $\lceil \log_2 p / \log_2 p_{\text{vm}} \rceil$ cells. Under the base- B limb encoding each cell carries at most $\lfloor \log_2 p_{\text{vm}} \rfloor$ bits, so ℓ limbs, hence ℓ trace cells, are required. *Multiplication.* Decompose the operands into limbs $a = \sum_{i < \ell} a_i B^i$ and $b = \sum_{j < \ell} b_j B^j$, with $B = 2^{\lfloor \log_2 p_{\text{vm}} \rfloor}$. For any fixed nonzero b , multiplication by b is a bijection of $\mathbb{F}_p$, so $a \cdot b \bmod p$ takes distinct values on inputs differing only in the limb a_i , and the routine’s output depends on each a_i . Restrict to a *limb-aligned* routine, in which each native $\mathbb{F}_{p_{\text{vm}}}$ multiplication has at most one limb of a among its two factors and no affine combination of distinct a -limbs is formed before a multiplication; schoolbook multi-limb routines are of this form, while Karatsuba-style routines that pre-combine distinct limbs into a single factor are not. The product is a degree-two function of the inputs, so a_i enters no purely additive term of the output and can influence it only through native multiplications that carry a_i as a factor. Were no such multiplication present, a_i could reach the output only through a b -independent additive coefficient, which cannot reproduce the product’s coefficient $b B^i$ of a_i ; so at least one native multiplication carries each a_i . A limb-aligned multiplication carries at most one of the ℓ distinct limbs $a_0, \dots, a_{\ell-1}$, so these are ℓ distinct multiplications. Normalising by the single native-instruction cost gives $\text{FMT}(\text{mult}) \geq \ell$; the schoolbook method forms all ℓ^2 limb products, attaining the upper end. $\square$

Remark 4.3. For PLUM ($\log_2 p \approx 199$) on a 31-bit host field ($\lfloor \log_2 p_{\text{vm}} \rfloor = 30$, for both RISC Zero’s BabyBear and the SP1 fork’s KoalaBear), $\ell = 7$, so Proposition 4.2 places the per-multiplication tax between 7 and 49. The measured precompile-free-to-precompiled cycle ratio of $58.5\times$ at $\lambda = 80$ (Cell 1 / Cell 2 in the measurement-cell naming of Section 7: no precompile / Griffin precompile) is a whole-workload ratio, dominated by the Griffin permutation’s collapse to a syscall rather than by bare field-multiplication emulation; it lands in the same order as the schoolbook $\ell^2 \approx 49$ bound of Proposition 4.2, which is consistent with the field-mismatch tax rather than a direct instance of the per-multiplication bound (that $58.5 > 49$ already precludes reading it literally).

A precompile, as defined in Section 3, replaces $\text{cyc}_{\text{em}}(\text{op})$ for a designated operation by a single zkVM syscall, whose cost from the guest’s perspective is one cycle plus the constant overhead of the syscall ABI. Writing $\text{Pre} \subseteq \text{ops}(\text{Pred})$ for the set of operations covered by precompiles and $\text{cyc}_{\text{syscall}}$ for the per-syscall overhead, the cumulative emulated cost of Pred under a precompile-enabled zkVM

is therefore

$$\text{Cyc}_{\text{em}}^{\text{pre}}(\text{Pred}) = \sum_{\text{op} \in \text{ops}(\text{Pred}) \setminus \text{Pre}} \text{cyc}_{\text{em}}(\text{op}) + |\{\text{op} \in \text{ops}(\text{Pred}) \cap \text{Pre}\}| \cdot \text{cyc}_{\text{syscall}}.$$

The performance benefit of adding a primitive to the precompile set is, by this accounting, the difference $\text{cyc}_{\text{em}}(\text{op}) - \text{cyc}_{\text{syscall}}$ summed over its occurrences. This is a *guest-trace* reduction, where it counts cycles removed from the emulated execution, not the prover’s wall-clock cost. The precompile shifts those cycles into a relation-specific AIR whose proving cost is charged on the host side, so a positive guest-cycle benefit need not translate into a faster prover, and Section 7 reports a primitive (Griffin) for which it does not.

The host-side charge admits the same style of accounting. A STARK-based zkVM prover commits, per execution shard, one trace matrix per chip, and its work scales quasi-linearly with the total number of committed trace cells. The deployed SP1 toolchain prices its own proving this way. Shards (the fixed-size segments into which the prover splits an execution, each proved separately) are split on a trace-area threshold, and the executor’s gas formula weights trace area three-to-one over instruction complexity (both read from the fork’s source, pinned in §7.1.2). Write $\bar{a}_{\text{core}}$ for the average number of committed core-chip cells (CPU, memory, and byte-range bookkeeping) per guest cycle, so that the core trace area of Pred under the precompile-enabled zkVM is $A_{\text{core}}(\text{Pred}) = \bar{a}_{\text{core}} \cdot \text{Cyc}_{\text{em}}^{\text{pre}}(\text{Pred})$. Each precompile P additionally contributes its own chip area

$$A_P(\text{Pred}) = E_P (r_P w_P + w_{\text{ctrl},P}),$$

where E_P is the number of syscall events Pred issues to P , r_P the number of trace rows the chip emits per event, w_P its main-trace width in columns, and $w_{\text{ctrl},P}$ the width of the controller row that binds each event’s memory traffic. The per-shard core-proving component of prover wall-clock is then modelled as

$$T_{\text{prove}}(\text{Pred}) \approx \kappa_{\text{core}} A_{\text{core}}(\text{Pred}) + \sum_P \kappa_P A_P(\text{Pred}), \quad (1)$$

where each κ is an effective per-cell cost for the chip family in question, absorbing commitment, constraint evaluation, lookup-argument columns, and padding. The absorbed terms only enlarge κ_P , so the accounting errs conservatively against precompiles. The guest-trace benefit computed above is the reduction in A_{core} . The term it omits is A_P . Both terms price the per-shard *core* prove. The recursive-compression and wrap stage is a separate cost regime, measured empirically in Section 7 and not represented in this model.

Proposition 4.4 (Precompile break-even). *Under the trace-area model of Equation (1), with the per-cycle core area $\bar{a}_{\text{core}}$ common to the precompiled and unprecompiled executions, adding an operation op to the precompile set lowers the core-proving wall-clock if and only if, per covered occurrence,*

$$(\text{cyc}_{\text{em}}(\text{op}) - \text{cyc}_{\text{syscall}}) \cdot \bar{a}_{\text{core}} \kappa_{\text{core}} > (r_P w_P + w_{\text{ctrl},P}) \cdot \kappa_P, \quad (2)$$

that is, when the core area removed outweighs, at the chips’ per-cell rates, the AIR area added.

Proof. From Equation (1), replacing the emulated occurrences of op by syscalls lowers A_{core} by $(\text{cyc}_{\text{em}}(\text{op}) - \text{cyc}_{\text{syscall}}) \bar{a}_{\text{core}}$ per occurrence while adding the chip area A_P ; the net change in T_{prove} is negative exactly under the stated inequality. $\square$

The two prove-mode outcomes of Section 7 are this inequality evaluated at two operating points. Against its own emulated baseline, the Griffin permutation removes millions of guest cycles per event at a per-event chip area on the order of 10^4 cells, so the inequality holds by a wide margin and the precompile recovers feasibility. Against the software SHA-3 control of Section 7, a predicate whose hash runs as native RISC-V software, the covered operation does not occur on the control side at all. The precompiled arm executes more guest cycles, a measured datum (11% more in execute mode, 1.23×10^8 versus 1.11×10^8 ; Section 7) rather than a consequence of the model, and carries the full chip area besides, so both terms of Equation (1) are larger on that arm and the model places it behind the control for every non-negative choice of the κ 's, provided $\bar{a}_{\text{core}}$ is common to the two arms. Section 7 instantiates the dimensions and calibrates the constants. The same accounting explains why vendor precompiles in production zkVMs report cycle-count collapses far larger than their prove wall-clock gains. The cycle ratio bounds the wall-clock ratio from above, and the bound is approached only when the precompile's per-event chip area is negligible against the remaining core area, which for permutation-sized chips it is not. SP1's own cost table prices its Keccak chip at 2,640 columns (`rv64im_costs.json`, in the fork pinned in §7.1.2) with 24 rows per permutation, a per-event area comparable to the bespoke Griffin chip's.

4.3 The Field-Matching Cost Criterion

Proposition 4.4 prices a single precompile decision. Read across the field-mismatch width it becomes the thesis's *cost criterion* (the *field-matching* criterion), predicting from a primitive's specification whether a precompile can help at all and when the help reaches prove-time.

The benefit is a mismatched-field phenomenon. By Proposition 4.2, the emulated cost of an algebraic primitive scales with the mismatch width $\ell = \lceil \log p / \lfloor \log p_{\text{vm}} \rfloor \rceil$. Each $\mathbb{F}_p$ multiplication costs at least ℓ native multiplications, $\Theta(\ell^2)$ under a limb-aligned schoolbook routine, against $\text{cyc}_{\text{nat}} = \Theta(1)$.

Proposition 4.5 (Field-matching). *For an algebraic primitive whose cost is dominated by $\mathbb{F}_p$ arithmetic, the field-mismatch cost a precompile removes is zero at $\ell = 1$ (the primitive's native field equals the prover's field) and positive for $\ell > 1$, at least linear in ℓ and reaching $\Theta(\ell^2)$ under a limb-aligned schoolbook routine. Hence a precompile can lower core-proving cost on the field-mismatch axis only if the primitive is field-mismatched; a field-matched algebraic primitive admits no such benefit.*

Proof. Immediate from Proposition 4.2 and Definition 4.1, at $\ell = 1$ an $\mathbb{F}_p$ element occupies one limb and one multiplication is one native multiplication, so $\text{cyc}_{\text{em}} = \text{cyc}_{\text{nat}}$ and the benefit term ($\text{cyc}_{\text{em}} - \text{cyc}_{\text{syscall}}$) of Equation (2) is non-positive while the chip area $A_P > 0$, so the inequality fails; for $\ell > 1$ the benefit term is at least linear in ℓ by Proposition 4.2, and $\Theta(\ell^2)$ under a limb-aligned schoolbook routine. $\square$

Mismatch is necessary but not sufficient. The converse fails, where a mismatched primitive need not pay, because the chip area A_P of Equation (2) also grows with ℓ and the realised prove-time outcome turns on the per-cell rates $\kappa_{\text{core}}, \kappa_P$, not on ℓ alone. The Griffin precompile ($\ell = 7$) pays against its own emulated baseline, recovering feasibility from a time-budget-exceeding state, yet proves *slower* than a control predicate built on a field-agnostic bitwise hash (the software SHA-3 control of Section 7), which never incurs the mismatched-field work the chip is built to absorb. The sufficient condition is

Equation (2) itself, evaluated with the chips' calibrated κ 's. Mismatch makes the benefit term nonzero, but whether it clears the chip area is a separate, quantitative question. A distinct arithmetic-axis no-pay case sits at full mismatch itself. When the primitive's multi-limb arithmetic is *already* served by an existing precompile, each covered operation is already priced at $\text{cyc}_{\text{syscall}}$, so its arithmetic benefit term $(\text{cyc}_{\text{em}} - \text{cyc}_{\text{syscall}})$ in Equation (2) is essentially zero and the *arithmetic-axis* core saving $\Delta A_{\text{core}}^{\text{arith}} \approx 0$; a naive dedicated chip only adds field-op area on that axis. The pay, where it comes, is on a different axis. What the chip removes is the per-symbol *control* overhead of the emulated loop, its per-step dispatch, memory access, and cross-table lookup. (Here *control* means the loop's dispatch machinery, not an experimental control arm.) The power-residue-PRF case measures this as a $10.33\times$ reduction on total committed area (Section 7, §7.5.3).

The field-mismatch tax only. Proposition 4.5 concerns algebraic primitives, whose cost is $\mathbb{F}_p$ arithmetic. A primitive whose cost is *bitwise* (SHA-2, Keccak) carries a different tax (the cost of emulating Boolean operations on a small arithmetic field) and can warrant a precompile even when field-matched, which is why production zkVMs ship Keccak precompiles over their native field. The field-matching criterion does not govern those. It predicts only the field-mismatch component, and its class boundary is the algebraic, large-prime family (Loquat, PLUM, and the power-residue/Legendre line) whose verification is dominated by large-prime arithmetic.

Two-sided criterion. A field-matched algebraic primitive ($\ell = 1$) should show *no* field-mismatch benefit, and a strongly mismatched one ($\ell = 7$, PLUM) a large one. The mismatched side is measured in Section 7 (the Griffin precompile recovers feasibility). The matched side is the keystone control (the matched-field control of Section 7), a primitive native to the prover field, which Proposition 4.5 predicts will not pay. That prove-mode prediction at $\ell = 1$ is a *deductive* consequence of the trace-area model rather than an independent test: as $\ell \rightarrow 1$ the removed core area shrinks toward a single native operation while the added chip area does not, so the model cannot but predict no-pay there. The *necessity* direction, by contrast, admits a direct execute-mode test, which confirms it: the per-multiplication field-mismatch tax measures $164\times$ from $\ell = 1$ to $\ell = 7$ (Section 7), a general-bignum (`num_bigint`), non-limb-aligned per-multiplication ratio that legitimately exceeds the limb-aligned schoolbook ceiling $\ell^2 = 49$ because only the ℓ lower bound of Proposition 4.2 governs that path, not the ℓ^2 upper bound (Section 7). The criterion is therefore established analytically (Proposition 4.5) and *postdictively*, by fitting the model to the mismatched arm after the fact, with its necessity direction confirmed in execute mode and its $\ell = 1$ prove-mode corollary following deductively from the model. PLUM has no matched-field primitive on which to test that corollary independently; the criterion's one out-of-sample empirical test is instead the already-served power-residue break-even (Section 7, §7.5.3).

4.4 Update Churn

Definition 4.6 (Localised verification-predicate change). A *localised verification-predicate change* is a modification to the verification predicate Pred that can be described as one of the following: substitution of one signature scheme for another at matching security level; retuning of a single numerical security parameter (e.g. λ , t , η); a change to the presentation *structure* of a showing relation (the number of credentials shown, or the set of disclosed attributes); or a one-line change to the encoding of a

public input. A change that only alters which condition the verifier checks on the *disclosed* attributes, a presentation *policy* at fixed structure, is *not* a verification-predicate change: following BDEC it is evaluated outside the proof relation (Section 3) and touches neither Pred nor its compiled artefacts.

The deployment-lifecycle cost of a localised change is dominated by the artefacts that must be regenerated and re-audited as a consequence, rather than by the change itself. We formalise this cost below and measure it for BDEC in Section 7 (Table 10).

Definition 4.7 (Update churn). Fix a deployment $\mathcal{D}$ that proves Pred in either the static-circuit or the zkVM regime, and let Δ denote a localised verification-predicate change. The *update-churn footprint* of Δ in $\mathcal{D}$ is the tuple

$$\text{Churn}(\Delta, \mathcal{D}) = (N_{\text{src}}(\Delta, \mathcal{D}), N_{\text{circ}}(\Delta, \mathcal{D}), N_{\text{param}}(\Delta, \mathcal{D}), N_{\text{audit}}(\Delta, \mathcal{D})),$$

where

N_{src} is the number of source-code files that must be modified in response to Δ .

N_{circ} is the number of arithmetic circuits or AIR programs that must be regenerated, counted as the distinct circuit or AIR artefacts whose serialised form changes between the pre- and post- Δ builds.

N_{param} is the number of proof-system parameters (e.g. commitment parameters, R1CS-shape-dependent constants) that must be re-derived.

N_{audit} is the number of distinct components requiring re-audit before redeployment, counted at the granularity of the deployment’s existing security-review boundary. This is an *audit-footprint* count, not a weighted effort model: it reports how many components re-enter the review boundary, not the person-time or risk class of reviewing them.

For the static-circuit regime, a localised change to Pred typically yields a non-trivial N_{circ} (the R1CS must be recompiled), a non-trivial N_{param} (constants that depend on the constraint count must be re-derived), and a non-trivial N_{audit} (the new circuit shape must be audited). For the zkVM regime, the same change always yields a non-trivial N_{src} (the guest program is edited) and zero regenerations of the *universal* relation R_{vm} , which is unchanged. The remaining components depend on whether the change touches a precompiled primitive. The bespoke Griffin₁₉₂ AIR is itself a relation-specific circuit. A change that alters the hash family or the field width therefore regenerates and re-audits that AIR, so $N_{\text{circ}} \geq 1$, $N_{\text{param}} \geq 1$, and $N_{\text{audit}} \geq 1$. The zero-circuit, zero-parameter case applies only to changes that leave every precompiled primitive intact (e.g. raising the credential count k from 1 to 2), for which $N_{\text{circ}} = 0$, $N_{\text{param}} = 0$, and N_{audit} is confined to the changed guest source. The configuration that would need zero new circuits for *any* change is the pure-emulation zkVM without precompiles, which Section 7 measures as infeasible (does not finish within the resource bound on the target hardware).

The empirical evaluation of Section 7 tabulates the circuit and parameter components N_{circ} and N_{param} for both regimes and discusses the source-edit and audit components N_{src} and N_{audit} in prose, under a fixed set of localised changes that we exhibit as benchmarks, namely signature substitution (Loquat $\rightarrow$ PLUM), security-parameter retuning ($\lambda = 80 \rightarrow 128$), and a presentation-structure change (raising the credential count k from 1 to 2).

4.5 Cost Decomposition of the Verification Predicate

The unit of cost is the PLUM verification predicate $\text{Pred}_{\text{Verify}}^{\text{Sig}, \text{PPSig}}$. A credential relation is a fixed number of its invocations, two in CreGen, $k + 2$ in ShowCre (Section 3), and the verifier-side membership, revocation, and disclosure checks fall outside the proof. Inside one predicate the dominant components are (i) the t -th power-residue PRF symbol checks, (ii) the BCS Merkle-commitment recomputations, and (iii) the low-degree-test invocations. The cost of (ii) and (iii) is dominated by the algebraic hash H_{Griffin} over $\mathbb{F}_p$. By Definition 4.1 every such Griffin permutation is a tax-bearing $\mathbb{F}_p$ operation on a small-native-field zkVM, so the field-mismatch tax of a whole credential relation is concentrated in the predicate’s algebraic hashing. This locates the dominant term (the Griffin permutation count) and identifies it as the precompile target of the next chapter. What Equation (2) and Section 7 answer, in both directions, is whether precompiling it recovers feasibility and whether it also lowers prove-time.

Writing $C_{\text{ver}}(\lambda, h)$ for the execute-mode cycle cost of one predicate invocation at security level λ under hasher h , and C_0 for the fixed per-run overhead (input parsing and witnessing), this invocation structure gives an additive cost model for the two credential relations,

$$C_{\text{CreGen}}(\lambda, h) \approx 2 C_{\text{ver}}(\lambda, h) + C_0, \quad (3)$$

$$C_{\text{ShowCre}}(k, \lambda, h) \approx (k + 2) C_{\text{ver}}(\lambda, h) + C_0, \quad (4)$$

linear in the verification count because everything outside the verifications, and the verifier-side checks outside the proof entirely, is dominated by the $\Theta(k)$ algebraic-hash work inside the predicates. Section 7 validates this additive form against the measured credential-relation points and calibrates the constant C_{ver} .

5 The Precompile Suite and the Credential Workload

The theoretical analysis of Section 4 identified the algebraic-hash and large-prime-field operations of PLUM verification as the operations that carry the field-mismatch tax inside a small-field zkVM. This section presents the precompile suite we built to absorb that tax so that the remaining proving cost can be measured. The suite is one reusable interface, instantiated three times. We then integrate it into the credential-generation relation. The suite serves the measurement reported in Section 7. Its purpose is to move the dominant cost out of emulated guest code so that what remains can be measured, attributed, and bounded. Throughout, we grade the evidence supporting each component, distinguishing what is proved from what is empirically tested and what is trusted from upstream, since the suite is not uniformly sound. Two soundness questions are left explicitly open (§5.5). A measurement whose correctness rested on unstated assumptions would be worth little, so stating the open questions is part of the work. The same Griffin chip that lowers the cost of the base (*core*) proof also enlarges the machine that recursively verifies the shards. That enlargement, quantified in Section 7, drives the wall-clock cost of the wrap’s recursive-compression stage (its *compress tail*). This cost is distinct from the post-quantum wrap obstruction of Section 6. By the field-matching criterion (Proposition 4.5), a precompile is warranted here only because PLUM verification is field-mismatched ($\ell = 7$). A primitive native to the prover field would need none, so the suite below is that criterion’s mismatched case made concrete, not a general claim that precompiling always helps.

5.1 Scope of the Implementation

We clearly define the scope of our implementation, as the integrity of the measurements in Section 7 depends on it.

- **Fp192 modular multiplication via the host precompile** (§5.2), routed through SP1’s `UINT256_MUL`. *Built; reduction argued; empirically tested (7/7 adversarial cases).*
- **A bespoke Griffin- $\mathbb{F}_p^{192}$ permutation AIR for SP1** (§5.3), the essential precompile, since PLUM’s constraint budget is dominated by Griffin. The 192 in the name is PLUM’s nominal field width; the deployed prime is the 199-bit prime of Remark 3.5. All ten constraint families implemented ($\approx 3,900$ lines of Rust across the chip and executor-compute modules, over 24 commits). A single-permutation smoke proof completes (execute, prove, and verify) in 13.0 s on the target hardware. *Built; per-row families proved and fault-tested, two of the three lookup-borne families fault-tested (memory-binding partial); full constraint-level chip-equivalence, that no wrong witness satisfies the constraints, not verified and out of scope here (§5.5).*
- **A dedicated `FP192_MUL` field-multiplication chip for SP1** (§5.5), a second instance of the precompile interface, built to test the construction’s generality. Its field arithmetic is inherited from the audited `FieldOpCols` gadget. *Built; core smoke proof passes (execute+prove+verify); not on the measured path.*
- **A `FP192_POW_RES` power-residue-PRF chip for SP1** (§5.5), a third instance, composing the field multiplication into $a^{(p-1)/t} \bmod p$ under a fixed square-and-multiply schedule bound row-to-row by a cross-table lookup. *Built; core and multi-symbol prove pass; chain-binding soundness adversarially verified; user/`mprotect` path disabled; not on the measured path.*

- **PLUM signature verification on SP1**, Griffin AIR and host `UINT256_MUL` active. *Built; proves end-to-end* (prove time in Section 7).
- **PLUM-in-BDEC credential generation (CreGen) on SP1**. *Built; validated in execute mode* (both signature verifications accepting). The relation completes a succinct (non-zero-knowledge) prove. The prove time and the shard-configuration provenance are reported in Section 7.
- **PLUM-in-BDEC presentation (ShowCre) on SP1**, the $k + 2$ -verification generalisation of CreGen. *Built; validated in execute mode* at $k = 1, 2$ (all $k + 2$ verifications accepting). It completes a succinct prove at both k (Section 7).

5.2 The Field-Arithmetic Substrate: $\mathbb{F}_{p192}$ Multiplication via the Host Precompile

Field and naming. PLUM operates over the prime field $\mathbb{F}_p$ with $p = 2^{64} \cdot p_0 + 1$, p_0 a 135-bit prime, so p is a 199-bit prime. We retain the label “ $\mathbb{F}_p^{192}$ ” for parity with the PLUM paper, whose §3.3 titles the parameter “192-bit smooth prime field”; the superscript here labels the field’s bit-width, not a vector-space dimension, and is distinct from the lane-count superscripts of the Griffin state ($\mathbb{F}_p^4$ for compression, $\mathbb{F}_p^2$ for expansion). All soundness arguments use the true 199-bit modulus, which is fixed in the implementation and audited against the paper.

Construction. Rather than write a dedicated 199-bit multiplication circuit, we reuse the 256-bit modular-multiplication precompile SP1 already ships modulo a runtime-supplied modulus, `UINT256_MUL` [8]. Each $\mathbb{F}_{p192}$ multiplication $a \cdot b \bmod p$ is computed by zero-padding a , b , and p into the precompile’s 256-bit slots and invoking the syscall with modulus $m = p$.

Soundness (reduction).

Lemma 5.1 ($\mathbb{F}_{p192}$ -multiplication functional equivalence). *Suppose that for every row of the host modular-multiplication table in a verifying proof the host AIR enforces $x' \equiv x \cdot y \pmod{m}$ and $x' \in [0, m)$ when $m \neq 0$ (the upstream guarantee). Then each $\mathbb{F}_{p192}$ multiplication $a \cdot b \bmod p$ routed through the precompile returns the canonical value in $[0, p)$, exactly the value the software fallback computes.*

Proof. $\mathbb{F}_{p192}$ operands are maintained in $[0, p)$ (structurally for the reducing constructor, by input domain for the small-integer constructors). Since $p < 2^{200} < 2^{256}$, the zero-pad is the identity on the bit pattern. The upstream guarantee with $m = p$ yields $x' = a \cdot b \bmod p \in [0, p)$, exactly the value the software fallback computes; the implementation reconstructs the field element directly from x' , omitting a redundant reduction, with a debug-build assertion guarding against regression of the upstream range check. $\square$

The upstream guarantee is instantiated for SP1 by the two specific AIR constraints that enforce the modular product and the output range check. The precompile is that of SP1 v6.2.1 (forked), the version the repository build resolves. The assumption is thus pinned to this specific version and not to an unversioned “upstream release process.”

Edge cases. The argument addresses a malicious modulus $m = 0$ (prevented by loading the modulus from a compile-time constant the zkVM’s memory checking binds), non-canonical operand truncation (guarded by a debug assertion on operand width), and regression of the upstream range check (under which the omitted reduction would become essential, detected by the same assertion).

Empirical corroboration. An adversarial probe exercises an honest control plus six single-field tamper cases (corrupted public key, message, and four signature fields, namely the σ_1 Merkle root, the PRF tag, a virtual-oracle response, and a final STIR coefficient) on SP1. It does not characterise malicious-prover behaviour completely, and we report it as such.

5.3 The Bespoke Griffin- $\mathbb{F}_p^{192}$ Permutation AIR (SP1)

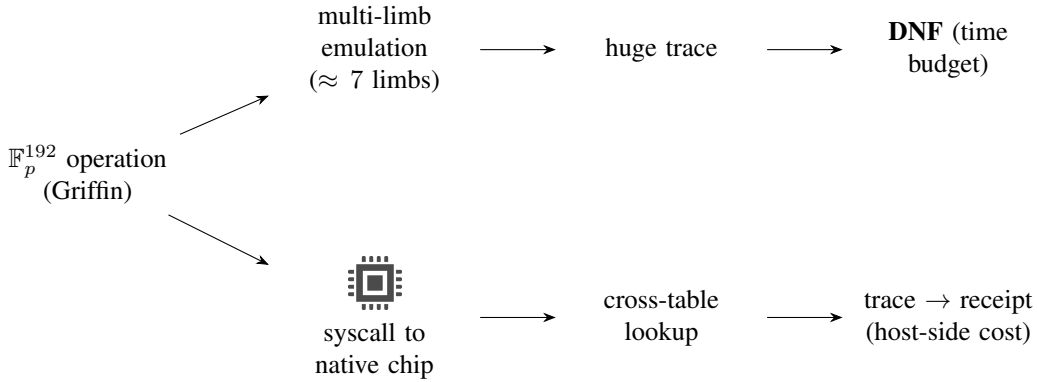


Figure 1: The field-mismatch tax and what a precompile absorbs

No existing SP1 chip computes a Griffin permutation over a 199-bit prime field (Figure 1 sketches the tax the missing chip leaves and what a precompile absorbs). The closest upstream pattern is SP1’s Poseidon2 chip [8], implementing the Poseidon2 permutation [64], which has the same shape (a permutation precompile with single-pointer state I/O, per-round columns, and memory binding) but operates over SP1’s 31-bit base field (KoalaBear, $2^{31} - 2^{24} + 1$, in the evaluated fork). We therefore implement a custom AIR for one Griffin- π permutation over $\mathbb{F}_p^4$, exposed to the guest as a single syscall on a state of 16 64-bit words (four per lane, four lanes; each 256-bit slot holding one 199-bit element).

Per-round constraints. Each of the 14 rounds is encoded by five per-round constraint families (the round-function families of Appendix B): a degree-3 forward S-box on one lane; an *inverse* S-box on another lane, constrained in the backward direction ($y^d = x$ with the root witnessed) because the forward inverse exponent is a 199-bit number that would blow past the constraint-degree budget; a quadratic layer on the remaining two lanes; the linear layer (the standard Griffin width-4 MDS matrix M_4 , branch number 5); and the SHAKE256-derived round constants. The backward constraint is valid because $\gcd(d, p - 1) = 1$ makes the cube map a bijection, so the witnessed root is unique. This side condition is *not* automatic, and we verify it for the implemented modulus: with $d = 3$ and $p = 2^{64}p_0 + 1$ we have $p - 1 = 2^{64}p_0 \equiv p_0 \equiv 1 \pmod{3}$ (since $2^{64} \equiv 1 \pmod{3}$, and $p_0 \equiv 1 \pmod{3}$ for the implemented 199-bit modulus, verified by computation), hence $3 \nmid (p - 1)$ and $\gcd(3, p - 1) = 1$.

Were $3 \mid (p - 1)$ the cube would be three-to-one and a malicious prover could witness a wrong root. The chip’s remaining five families (round count, memory binding, output canonicity, row scheduling, parameter immutability) constrain the trace as a whole, as described next. The modulus, round count, linear-layer entries, round constants, and quadratic coefficients are bound in preprocessing rather than supplied per row, so a prover cannot choose them. A separate controller chip binds the syscall’s memory reads and writes, and a per-lane output range check enforces output canonicity (§5.2’s cross-precompile contract). Two audit-driven hardening passes fixed a polynomial-degree blow-up and an all-zero-padding-row identity failure.

5.4 The Sponge and Compression Wrapper

The chip exposes one permutation. Hashing needs a mode built around it. Algorithm 1 gives the guest-side 2-to-1 compression built on the syscall, and the construction follows.

Algorithm 1 Guest-side 2-to-1 compression H_2 via the GRIFFIN_FP192_PERMUTE syscall

Require: child digests $x = (x_0, x_1) \in \mathbb{F}_p^2$, $y = (y_0, y_1) \in \mathbb{F}_p^2$, canonical in $[0, p)$

Ensure: digest $H_2(x, y) \in \mathbb{F}_p^2$

- | | |
|---|---|
| 1: $s \leftarrow (x_0, x_1, y_0, y_1)$ | ▷ width $t_{\text{state}} = 4$, capacity $c_{\text{cap}} = 2$, rate 2 |
| 2: write s to the state buffer in limb encoding | ▷ layout fixed by the syscall ABI; see §5.3 |
| 3: syscall GRIFFIN_FP192_PERMUTE(& s) | ▷ one cycle from the guest’s perspective (§3) |
| 4:  (s_0, s_1) | ▷ truncate to the rate lanes |
-

The permutation is wrapped in the sponge/compression construction of the Loquat instantiation to provide the 2-to-1 compression used by PLUM’s internal Merkle (BCS) commitments, the in-proof Griffin Merkle cost, and by the BDEC ledger trees the relying party checks outside the proof (Section 3). The state width is $t_{\text{state}} = 4$, the capacity $c_{\text{cap}} = 2$, and the rate $r = t_{\text{state}} - c_{\text{cap}} = 2$, so a digest is the first two lanes of the output state. The 2-to-1 compression of two child digests $x = (x_0, x_1)$ and $y = (y_0, y_1) \in \mathbb{F}_p^2$ is $H_2(x, y) = (\pi(x_0, x_1, y_0, y_1))_{0,1}$, one application of the width-4 permutation π , truncated to the rate lanes. The SHAKE256 domain-separation seed and the linear-layer matrix are matched to the reference hasher, and cross-codebase tests pin the modulus, round count, linear-layer matrix, and seed against the reference implementation.

5.5 Soundness of the Precompile Suite, and Two Open Questions

This subsection states the soundness claim and the two questions it leaves open. The per-constraint audit is deferred to Appendix B.

The theorem below is a functional (input–output) equivalence statement, and, conditional on the black-box-transfer hypothesis, a transfer of PLUM’s EUF-CMA reduction. It is not a standalone cryptographic-security claim. The assumptions it names are stated formally in Section 6; each hypothesis carries its content here so the theorem is readable in place.

Theorem 5.2 (Precompile soundness, conditional). *Assume that*

1. *the host zkVM’s cross-table lookup argument is sound (Assumption 6.6);*

2. *the Griffin AIR’s constraints are satisfied only by a witnessed transition equal to the reference Griffin- π permutation (Assumption 6.1);*
3. *PLUM’s EUF-CMA reduction treats Griffin and field multiplication as black-box input–output maps, inspecting no intermediate representation such as a limb of a field product (Assumption 6.5(iii)); and*
4. *the upstream modular-multiplication guarantee of §5.2 holds (Lemma 5.1).*

Then substituting the precompile suite for the emulated operations preserves the input–output value of every PLUM and BDEC verification predicate, and, under the black-box-transfer hypothesis, PLUM’s EUF-CMA reduction transfers unchanged. (The reduction’s reliance on (Q)ROM-replaceability of this Griffin instantiation is a separate, upstream assumption, recorded as an open upstream assumption in Section 6 and not discharged here.)

Proof sketch. The input–output equality is Lemma 6.4 applied to each PLUM and BDEC verification predicate: under hypotheses (1), (2) and (4) the precompiles return the reference value of every large-field multiplication and Griffin permutation, so the accept-or-reject decision is unchanged. Granting Assumption 6.5(iii), PLUM’s EUF-CMA reduction is in the random-oracle model and depends only on the input–output behaviour of the hash function instantiating the oracle, which the precompile reproduces *exactly* under Assumption 6.1; no random-oracle programming step in the reduction is affected, so it transfers unchanged. $\square$

The precompile interface, and the family that instantiates it. Although stated for Griffin, the theorem’s hypotheses are the *interface* any precompile for an algebraic primitive in this family must meet: (i) input–output equivalence to the reference primitive over $\mathbb{F}_p$, so the chip computes the same function the emulated code did; (ii) isolation, so the outer proof sees only the syscall’s committed inputs and outputs, never an intermediate limb (the black-box-transfer hypothesis together with lookup binding); and (iii) a constant, preprocessing-bound modulus, so a malicious prover cannot choose the field per row. To show that this interface is a reusable construction, we build *three* instances of it over the same 199-bit field, each meeting (i)–(iii) with a soundness argument of the same shape: the Griffin permutation AIR (§5.3); a dedicated FP192_MUL field-multiplication chip; and a FP192_POW_RES chip that composes the field multiplication into the power-residue-PRF symbol check $a^{(p-1)/t} \bmod p$ under a fixed square-and-multiply schedule, the steps bound row-to-row by a cross-table lookup. All three build and pass a core-STARK smoke proof (execute, prove, verify), and FP192_POW_RES additionally completes the multi-symbol (multi-shard) prove a real power-residue batch requires, after correcting an executor timestamp that had desynchronised its memory records across a shard boundary. Its row-to-row chain binding was also checked against an adversarial soundness review, and its user/mprotect path is disabled (it carries no page-protection columns), so it is validated at the same core-prove level as Griffin but not for an untrusted user path.

Measured versus demonstrated. Only the Griffin AIR is on the measured path. The measured PLUM-verify figure of Section 7 is produced by the Griffin chip together with the host UINT256_MUL for field multiplication, not by the FP192_MUL or FP192_POW_RES chips. Those two are built to demonstrate that the interface *generalises* across primitive classes (a permutation, a field multiplication, and a composed modular exponentiation) and they also locate where the criterion’s *positive* question lives. FP192_MUL re-implements arithmetic the host UINT256_MUL precompile already serves

(it is essentially that host chip retargeted), so the criterion predicts it does not pay. The chips’ committed dimensions sharpen rather than simply confirm this. The dedicated chip is in fact $\approx 18\%$ *narrower* per operation (304 versus 371 committed cells, since hardcoding the modulus deletes the modulus-from-memory columns the host chip carries; docs/measurements/keystone_chipprove_20260706). So on per-operation area alone it leans toward paying. The predicted not-pay verdict rests instead on the fixed whole-table overhead a dedicated chip adds to the recursion and verify shapes (the fixed layouts provisioned for the wrap’s recursion stages; Section 7), the same cost this thesis documents for Griffin. An 18% per-operation saving over an already-present host chip does not amortise that overhead. That whole-machine cost we do not measure end-to-end, so this verdict is a design-level prediction of the criterion, not a measured outcome. FP192_POW_RES is the component whose pay-or-not verdict is not settled by per-operation area alone. The power-residue symbol $a^{(p-1)/t} \bmod p$ is the one verifier component that is *irreducibly large-prime* for two independent reasons. Its PRF security is $O(\sqrt{p})$ key recovery (established for the Legendre ($t=2$) case [65, 66], on whose hardness the general t -th residue rests), so the prime cannot shrink to a matched field the way the hash can (the SHA-3 control of Section 7 shows the hash can); and the STIR low-degree test needs a smooth multiplicative subgroup of the same $\mathbb{F}_p$ [30]. Either bind makes it the natural target for a bespoke chip. Whether that chip *pays* turns on a control-overhead saving that a per-operation comparison hides. Per symbol the naive dedicated chip commits more field-operation area than the guest loop, evaluating both square-and-multiply branches unconditionally, so on the per-operation arithmetic axis alone it does not pay; but that axis omits the per-symbol control overhead of the square-and-multiply schedule the chip replaces (one syscall dispatch, memory access, and lookup for each of the ≈ 261 multiplications, against one for the chip). The deciding trace-area experiment, isolating exactly that saving (the chip against the guest loop on *total* committed area, field-operation plus control, at the honest baseline), is specified here and run in Section 7 (§7.5.3), where the criterion’s positive case is decided. Griffin, by contrast, is field-mismatched *and* unserved, the configuration in which the criterion expects a precompile to pay. Its verdicts under both baselines are likewise Section 7’s. The shared interface is what lets the three chips serve as evidence for one method rather than three separate constructions.

What is established. The field-substrate half rests on the reduction of §5.2. The suite includes the bespoke Griffin AIR, to which Assumption 6.1 applies. For the Griffin AIR, the per-family analysis (Appendix B) exhibits, family by family, the constraint *intended* to catch any deviation from reference Griffin, the forgery that deviation would enable, and the implementation status of each. Memory binding, output canonicity, parameter immutability, and row scheduling are implemented and constrained. What that analysis does *not* establish, and says so explicitly, is that the implemented constraint system is satisfied *only* by reference-equal witnesses. Assumption 6.1 names it the *chip–constraint equality*. It is empirically supported by the 256-vector cross-codebase agreement and the partial fault-injection coverage discussed below (per-vector false-pass probability $\approx 2^{-199}$ for the vector suite), but the vector agreement pins the *executor* against the reference, a weaker *chip–executor* equality, not the *constraints*. Assumption 6.1 is not verified at the constraint level.

Open question 1: Assumption 6.1 is empirically supported but unproved. The 256-vector agreement establishes that the executor’s native compute equals the reference permutation. It does *not* by itself establish that the *AIR constraints* admit only that computation. For the per-row layer (the per-row families) that stronger statement carries a written proof. Appendix B bridges the limb iden-

tities the chip constrains over its small base field to congruences modulo p and composes them into the reference round, conditional only on the byte-range lookups that the lookup-binding hypothesis already covers. A separate fault-injection suite corroborates this layer in the rejection direction via `debug_constraints`. Single-cell corruptions of a satisfying trace are rejected for each of the seven per-row families. The three lookup-borne families (round count, memory binding, and the cross-row threading of row scheduling) lie outside `debug_constraints`, which does not evaluate cross-table lookups; we reach them with a separate lookup-balance harness (`debug_interactions`) that computes the net multiplicity ($\sum \text{sends} - \sum \text{receives}$) of the Griffin cross-table lookup, a nonzero value being an imbalance the verifier rejects, conditional on the lookup-binding assumption. A short round chain (round count) and a broken cross-row state thread (row scheduling) each produce the predicted imbalance and are rejected; memory binding is rejected for the syscall-level state handoff at the committed clock and pointer, while its global-memory-consistency half, that the controller’s read equals historical guest memory, is a lookup to the global memory chip that needs the full machine and is not covered. Parameter immutability is argued by construction. What Assumption 6.1 still carries splits in two, and each needs a different tool. The code-to-constraint transcription fidelity of the per-row layer, that the constraints admit only the honest witness, would be closed by deriving the AIR witness from the same reference code and checking full-row constraint satisfaction directly, ideally by machine. The three lookup-borne families lie outside that per-row check, their cross-table binding not being a per-row property; the lookup-balance harness above exercises them in the rejection direction (round count and cross-row threading fully, memory binding for the syscall handoff), leaving the global-memory-consistency lookup uncovered and, for all three, the no-wrong-witness direction open and conditional on the lookup-binding assumption.

Open question 2: the classical security floor at our parameters and the QROM lift are upstream of the AIR. PLUM’s EUF-CMA argument under Fiat–Shamir requires Griffin to be replaceable by a (quantum) random oracle. The Griffin analyses cover the original instantiations. Whether they cover this $d = 3$, width-4, 199-bit-field, 14-round instantiation is not something the AIR can settle. If this instantiation were QROM-distinguishable, no amount of AIR engineering would recover security. We have surveyed the classical Griffin algebraic-attack frontier and placed this instance against the broken ones (Section 6, Table 2), and it is not among them. What that survey leaves open, and what we do not close here, is two things. The classical security floor at our exact parameters still needs the FreeLunch and Improved-Resultant estimator instantiated at $(r=14, d=3, t_{\text{state}}=4, \log_2 p \approx 199)$, and the QROM lift, whether this instantiation is quantum-random-oracle-indistinguishable and whether PLUM’s Fiat–Shamir reduction lifts to the quantum random-oracle model, is upstream of the AIR.

Further limitations. The EUF-CMA reduction is argued to transfer on the grounds that the reference reduction treats Griffin and field multiplication as black-box input–output maps, but we have not audited PLUM’s reduction line by line for a step that inspects an *intermediate* representation (such as a limb of a field product), which input–output equality alone would not cover. The zero-knowledge property required by BDEC’s anonymity guarantee is a property of the proof wrap, analysed as the anonymity–deployability tension of Section 6 and measured in Section 7, not closed here.

5.6 Integration into the Credential Relations (BDEC Instance)

The credential-generation relation $R_{\text{cre}}^{\text{Sig}}$ of Section 3, instantiated with PLUM as Sig, is the AND concatenation of two PLUM signature verifications, both under the user’s public key pk_U , with public statement $x_{\text{cre}} = (c_{U,TA}, h_{U,TA}, \text{ppk}_{U,TA})$ and witness $w_{\text{cre}} = (\text{pk}_U, \text{psk}_{U,TA})$. We implement this relation *in full* as an SP1 guest, with PLUM in place of Loquat and field multiplications routed through `UINT256_MUL`. The guest computes exactly the two $\text{Pred}_{\text{Verify}}$ conjuncts that define base CreGen, with no omitted predicate. (The optional Merkle-accumulator variant of Remark 3.7 is not part of this relation and performs its membership checks in `BDEC.ShowVer`. It is out of scope here.) The guest binds the proof to its public statement: it commits $x_{\text{cre}} = (c_{U,TA}, h_{U,TA}, \text{ppk}_{U,TA})$ to the proof’s public values, so an accepting receipt certifies *which* statement was proved rather than merely “two signatures verify under some witness”. This binding bears on security and is not a cosmetic completeness gap; it is discharged here and measured in Section 7, at essentially the cost of the outcome-only prove since the commitment is a few field elements and does not alter the proved relation. The public parameters, pk_U , and both messages enter the guest as private inputs, and $h_{U,TA}$ and the pseudonym are formed on the host and passed in as opaque messages. Committing x_{cre} binds the receipt to that specific $h_{U,TA}$. A fully attribute-bound verifier that also proved $h_{U,TA} = H(A)$ in-circuit would recompute these hashes there, a further step this relation does not take. In execute mode at $\lambda = 80$ the guest runs both verifications to acceptance, the two costing essentially the same, confirming that both pass through the same precompile-backed path (guest-cycle counts in Section 7). The transition from this execute-mode validation to a completed proof is the subject of Section 7. Both relations complete a succinct (non-zero-knowledge) prove on SP1, a receipt distinct from the zero-knowledge wrap. The prove times and the CreGen shard-configuration provenance are reported in Section 7.

The full presentation relation (`ShowCre`), the $k + 2$ -verification generalisation of this relation, is likewise implemented as an SP1 guest, validated in execute mode at $k = 1, 2$ and, like CreGen, statement-bound in prove mode by an end-to-end run committing x_{show} at $k = 2$ (Section 7). Its zero-knowledge wrap also completes and verifies on the target hardware at both $k = 1$ and $k = 2$ (the latter statement-bound), each within the memory budget, with the full timing, peak memory, and provenance in Section 7.

6 Security Analysis

This section answers one question. Which of the credential’s security properties survive the move to the zkVM substrate? The answer, established below, has three parts. Knowledge soundness (that only someone who genuinely holds a valid signature can produce an accepting proof) transfers, under Assumptions 6.1 and 6.6 and Conjecture 1. Classical zero-knowledge is delivered within the memory budget, by the pairing wrap. Deployable post-quantum anonymity is not delivered on this stack.

Section 5 described the precompile suite, which moves PLUM’s dominant field operations out of emulated guest code. A performance change to a credential system is meaningful only if it does not silently weaken the security the system provides. This section makes that requirement precise. We fix the security model and threat model in Section 6.1, prove in Section 6.2 a preservation theorem that bounds how far the move to a zkVM substrate and the precompile suite can shift the adversary’s advantage in each of BDEC’s security experiments, and then isolate in Section 6.3 the one property the analysis does not deliver on consumer hardware, namely *post-quantum* anonymity of the proof actually produced. The preservation theorem is conditional, and it rests on two essential premises, which we name here. The first is Assumption 6.1, the constraint-level chip-equivalence of the Griffin- $\mathbb{F}_p^{192}$ AIR, namely that its constraint system is satisfied only by a witnessed transition equal to the reference permutation. The second, for the knowledge soundness of the composite multi-shard receipt the prover actually emits, is the extractor conjecture stated with the analysis (Conjecture 1), which is stated, not proved. The former premise is empirically supported, by a 256-vector cross-codebase equivalence suite and rejection-direction fault injection on seven of its ten constraint families, and it is not yet verified at the constraint level for the three lookup-borne families; We carry each premise as a scoped open condition, not a defect. Closing the first is machine-checking rather than new theory. Closing the second is a composition proof. The anonymity finding, in turn, is not a weakness in the construction. A classically-secure zero-knowledge wrap of the receipt completes within the consumer memory budget (Section 6.3), so the obstruction is not that the anonymity-providing proof fails to run but that the only wrap the toolchain exposes is pairing-based, hence classically secure only. Given that the precompile suite is what lets PLUM verification run inside the zkVM at all (Section 5), this section fixes *which* of the credential’s security properties survive that move and which it leaves out of reach, the three-part answer stated at the outset. What the move costs is priced separately by the cost criterion of Sections 4 and 7.

6.1 Security Model and Threat Model

BDEC is proved secure in three experiments, formalised in Section 3. Unforgeability requires that an adversary with issuance and showing oracles cannot produce an accepting showing for a statement outside its honestly issued set, beyond negligible advantage. Anonymity and unlinkability are challenge-bit experiments in which the adversary cannot distinguish, beyond advantage $|\Pr[b' = b] - \frac{1}{2}|$, which of two honestly issued credentials produced a challenge showing (anonymity) or under which of two pseudonyms a single challenge credential was issued (unlinkability). We write $\text{Adv}_{\text{AC}}^{\text{unf}}$, $\text{Adv}_{\text{AC}}^{\text{anon}}$, and $\text{Adv}_{\text{AC}}^{\text{unlink}}$ for the respective advantages, all defined in Section 3.

Definition 6.1 (zkVM credential advantage). For $\bullet \in \{\text{unf}, \text{anon}, \text{unlink}\}$, write $\text{Adv}_{\text{zkVM-Cred}}^{\bullet}(\mathcal{A})$ for the advantage $\text{Adv}_{\text{AC}}^{\bullet}(\mathcal{A})$ of Section 3 evaluated on the BDEC scheme instantiated with PLUM as the signature scheme Sig and the zkVM as the zkSNARK Π .

Unfolding Definition 6.1, the experiments defining $\text{Adv}_{\text{zkVM-Cred}}^\bullet$ for $\bullet \in \{\text{unf}, \text{anon}, \text{unlink}\}$ are as follows. We first fix the proved object and the zkVM interface, then the oracles, then the three games. Each game restates the corresponding Section 3 experiment with the standard adaptive oracle interface and the zkVM receipt interface, the JBind conjunct being the receipt-model translation of “ π verifies against x ” rather than a change of the win predicate, evaluated on BDEC instantiated with PLUM and the zkVM, so BDEC’s Theorems 1–3 bounds still apply. For orientation: each assumption below is stated formally, most with a named error term (the knowledge-soundness and simulator games of Definitions A.1 and A.3 and the zero-knowledge game of Definition A.4, all stated formally in Appendix A.1, and the arithmetisation-soundness defect ϵ_{AIR} of Assumption 6.1; a few, such as the black-box-transfer property, are binary and carry none). The assembled advantage is the expanded per-experiment sum of Equations (5) and (6) together with the unlinkability specialisation and the separate issuance-anonymity bound, over six leaf error terms (the five named in Theorem 6.5 plus $\epsilon_{\text{keypriv}}$ for issuance) with the tightness factors given there. These *error terms* are a different list from the five open *assumptions* named below. An assumption does not always carry an error term. The *status* of each, established, assumed, or open, is set out term by term after the theorem.

A zero-knowledge proof of knowledge of a valid signature. The object the credential prover produces is not a signature verification but a *zero-knowledge argument of knowledge* of a witness that makes a signature-verification relation accept, with that witness hidden. Checking a signature is easy once the signature is in hand. The difficulty this thesis instruments is to certify that such a signature exists *without revealing it, or the key it verifies under*. Fix $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$ and recall the efficiently decidable credential relation of Section 3. Every value to be signed is first mapped into the message space by $\text{Encode}_{\text{Sig}} : \{0, 1\}^* \rightarrow \mathcal{M}_{\text{pp}_{\text{Sig}}}$, so $h_{U,TA} = H(A) \in \mathcal{M}_{\text{pp}_{\text{Sig}}}$ and the pseudonym is signed under the encoded message $m_{\text{nym}} := \text{Encode}_{\text{Sig}}(\text{ppk}_{U,TA})$; the signatures are produced honestly by $c_{U,TA} \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_U, h_{U,TA})$ and $\text{psk}_{U,TA} \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_U, m_{\text{nym}})$. With this encoding the credential relation reads

$$R_{\text{cre}}^{\text{Sig}}(x_{\text{cre}}; w_{\text{cre}}) = 1 \iff \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{pp}_{\text{Sig}}}(\text{pk}_U, h_{U,TA}, c_{U,TA}) = 1 \wedge \text{Pred}_{\text{Verify}}^{\text{Sig}, \text{pp}_{\text{Sig}}}(\text{pk}_U, m_{\text{nym}}, \text{psk}_{U,TA}) = 1,$$

with public instance $x_{\text{cre}} = (c_{U,TA}, h_{U,TA}, \text{ppk}_{U,TA})$ and *private* witness $w_{\text{cre}} = (\text{pk}_U, \text{psk}_{U,TA})$. Membership is decidable in time polynomial in $|x_{\text{cre}}| + |w_{\text{cre}}|$ by running Sig.Verify twice. The credential prover supplies a proof π for “ $\exists w_{\text{cre}} : R_{\text{cre}}^{\text{Sig}}(x_{\text{cre}}; w_{\text{cre}}) = 1$ ” that is zero-knowledge in w_{cre} . pk_U , though public *by type*, is used here as a hidden witness *by role*. Revealing it would tie the showing to the user’s identity, so anonymity requires $\text{pk}_U \in w_{\text{cre}}$, never in x_{cre} . The same reading carries to the showing relation $R_{\text{show}}^{\text{Sig}}$ with its $k + 2$ predicate conjuncts, where each pseudonym message is $m_{\text{nym}}^{(j)} := \text{Encode}_{\text{Sig}}(\text{ppk}_{U,TA}^{(j)})$ as in Section 3.

Modelling the zkVM interface and journal-binding inside the game. The proof is produced by a zkVM whose universal relation is $R_{\text{vm}}((\text{img}, \text{in}_{\text{pub}}, \text{out}); (P, \text{in}_{\text{priv}})) = 1$ iff $\text{img} = \text{digest}(P)$ and running guest P on $(\text{in}_{\text{pub}}, \text{in}_{\text{priv}})$ terminates emitting public output out ; the public instance is $x_{\text{vm}} = (\text{img}, \text{in}_{\text{pub}}, \text{out})$ and the witness $w_{\text{vm}} = (P, \text{in}_{\text{priv}})$. The host supplies inputs over a *read* channel and receives outputs over a *commit* channel. Values placed on the commit channel enter out and become public, values read but not committed stay in in_{priv} and remain hidden. In the BDEC instantiation sk_U , pk_U , and the credential signatures enter only via the read channel and are never committed, so they

remain in w_{vm} across the entire relation, which is what keeps w_{cre} hidden. We make the binding of the public statement an explicit predicate of the game:

$$\text{JBind}(\text{out}, x) = 1 \iff x \text{ is committed on the commit channel, i.e. } x \text{ is recoverable from out.}$$

An accepting receipt certifies only “ $\exists w : R(x; w) = 1$ for the x that JBind pins down”. The games below carry JBind as an explicit conjunct in the win condition, so that they model the committed system in which x_{cre} is committed to the journal, which both credential relations now implement (Section 7); the earlier boolean-only variant, which removes it, we state explicitly after the games.

The wrap’s public instance, and what it leaks. Instantiated on the deployed stack, the public instance $x_{\text{vm}} = (\text{img}, \text{in}_{\text{pub}}, \text{out})$ carries only two statement-bearing components, the read-channel inputs being private so that in_{pub} is empty: img , a commitment to *which* guest program was proved, and out , a digest whose preimage is exactly the statement $x - x_{\text{cre}} = (c_{U,TA}, h_{U,TA}, \text{ppk}_{U,TA})$ for issuance, $x_{\text{show}} = ((\text{ppk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V}, h_{U,V})$ for showing—together with the aggregate accept bit. Here $h_{U,V} := H(A^\downarrow)$ is the disclosed-attribute digest, the showing analogue of $h_{U,TA} = H(A)$: the receipt commits this constant-size digest in place of $A^\downarrow$ itself, which the verifier still checks in the clear (§3), so the committed showing statement and the anonymity game’s canonical $x_{\text{show}} = (A^\downarrow, (\text{ppk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V})$ name the same disclosure, $h_{U,V}$ being a deterministic function of the disclosed $A^\downarrow$.⁴ The wrapped proof *leaks nothing beyond* x_{vm} , which resolves into two obligations. First, the wrap is zero-knowledge: by Definition A.4 it hides w_{vm} , and with it pk_U , the signatures, and the entire execution trace, though only against a classical distinguisher, the wrap being pairing-based over BN254 (the second obstruction, §6.3). Second, out commits only the intended disclosures $\text{Leak}(x)$ and no witness residue, since pk_U , $\text{psk}_{U,TA}$, and every signature travel the read channel alone. Zero knowledge hides the witness. It does not hide what the public statement itself reveals. What the wrap therefore delivers is *pseudonymity*, the real identity pk_U staying hidden, together with unlinkability on the freshly sampled verifier-facing pseudonym $\text{ppk}_{U,V}$, scoped by the equal-Leak admissibility of the anonymity game below; it is not concealment of the teaching-authority pseudonym $\text{ppk}_{U,TA}$, a public ledger handle by design (§3).

Oracles. All three experiments give the challenger an honest-user registry populated at setup (so that the notion of an honestly generated user key pair is well defined, exactly as in Section 3) and an issuance log $\mathcal{L}$ of tuples $(\text{ipk}, \text{upk}, A, \text{cred})$, together with the following two oracles, to which the adversary $\mathcal{A}$ has *adaptive* access. There is no corruption oracle. The source model of Section 3 exposes only issuance and showing, and the cited BDEC bounds are proved against exactly this interface.

- $\mathcal{O}_{\text{Issue}}(i, A)$: for a registered honest user i , runs the honest issuance path. The teaching authority signs

$$\sigma_{\text{cred}} \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_{TA}, \text{Encode}_{\text{Sig}}(\text{pk}_U, \text{ppk}_{U,TA}, A, \text{aux})),$$

the ledger stores the record digest $\rho_{U,TA} = H(\text{pk}_{TA}, \text{ppk}_{U,TA}, A, \text{aux}, \sigma_{\text{cred}})$, the challenger appends $(\text{ipk}, \text{upk}, A, \text{cred})$ to $\mathcal{L}$, and returns the issued credential material.

⁴The concrete SP1 wrap proof additionally publishes three witness-independent constants, a guest halt code, a verifying-key-set root, and a prover nonce, that depend on no part of w_{vm} and carry no user information.

- $\mathcal{O}_{\text{Show}}(i, \phi, A^\downarrow)$: for a registered honest user i , runs honest BDEC.ShowCre and returns the receipt $(x_{\text{vm}}, \text{receipt})$, i.e. the public statement $x_{\text{show}} = (A^\downarrow, (\text{ppk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V})$ and the proof π ; the presentation predicate ϕ is adversary-chosen at show time.

Experiment $\text{Exp}_{\text{zkVM-Cred}, \mathcal{A}}^{\text{unf}}(\lambda)$ (unforgeability)

Model: classical PPT adversary, classical random-oracle model.

1. The challenger samples $\text{par} = (\text{pp}_{\text{Sig}}, \text{crs}, \text{img}) \leftarrow \text{BDEC.Setup}(1^\lambda)$, where img is the fixed guest-image digest of the credential guest, initialises $\mathcal{L} \leftarrow \emptyset$ and the honest-user registry, and gives par and the public ledger roots $(\text{rt}_{\text{cred},e}, \text{rt}_{TA,e}, \text{rt}_{LR,e})$ to $\mathcal{A}$.
2. $\mathcal{A}$ interacts adaptively with $\mathcal{O}_{\text{Issue}}$ and $\mathcal{O}_{\text{Show}}$.
3. $\mathcal{A}$ outputs $(\text{ipk}^*, x^*, \text{receipt}^*)$ with $\text{receipt}^* = (\text{out}^*, \pi^*)$ and public instance $x_{\text{vm}}^* = (\text{img}, \text{in}_{\text{pub}}^*, \text{out}^*)$, where in_{pub}^* is empty, as the deployed instance fixes (the wrap-instance paragraph above).
4. The experiment outputs 1 iff

$$\Pi_{\text{vm}}.\text{Verify}(\text{crs}, x_{\text{vm}}^*, \pi^*) = 1 \wedge \text{JBind}(\text{out}^*, x^*) = 1,$$

and the relying party's verifier-side checks of Section 3 accept: ipk^* is an authorised teaching authority (a member under the published root $\text{rt}_{TA,e}$) and the statement's teaching-authority pseudonym(s), $\text{ppk}_{U,TA}$ for issuance and $(\text{ppk}_{U,TA}^{(j)})_j$ for a showing, are *anchored*, each appearing in an accepted ledger record under $\text{rt}_{\text{cred},e}$ (the ledger the challenger maintains, so an anchored pseudonym was issued through $\mathcal{O}_{\text{Issue}}$ and its record binds a registered honest key; the freshly sampled verifier-facing pseudonym $\text{ppk}_{U,V}$ is not a ledger object and carries no anchoring requirement); and the forgery is *fresh*, in the win predicate of Section 3: no honestly produced material explains the statement, i.e. there is no witness under a registered honest user key, assembled from the challenger's honestly issued credentials (the tuples of $\mathcal{L}$) and honestly produced showing-time signatures (the $\mathcal{O}_{\text{Show}}$ material), that satisfies the relation on x^* . This in particular excludes a verbatim replay of an $\mathcal{O}_{\text{Show}}$ receipt and a re-proof of its statement. We abbreviate this freshness clause $x^* \notin \mathcal{L}$.

$\text{Adv}_{\text{zkVM-Cred}}^{\text{unf}}(\mathcal{A}) = \Pr[\text{Exp}_{\text{zkVM-Cred}, \mathcal{A}}^{\text{unf}}(\lambda) = 1]$. The experiment itself runs no extractor: knowledge soundness (ϵ_{KS}) is spent by the reduction (Appendix A), whose first hop extracts a witness from the accepting receipt. The JBind conjunct pins the committed statement x^* that the extracted witness must satisfy, and the anchoring conjunct ties the forgery to the challenger's registry, which the reduction's honest-key guess requires.

Experiment $\text{Exp}_{\text{zkVM-Cred}, \mathcal{A}}^{\text{anon}}(\lambda)$ (anonymity)

Model: classical PPT adversary, classical random-oracle model.

1. Setup as above; $\mathcal{A}$ receives par , the roots, and adaptive access to $\mathcal{O}_{\text{Issue}}$ and $\mathcal{O}_{\text{Show}}$.
2. $\mathcal{A}$ designates two entries of the issuance log $\mathcal{L}$, honestly issued credentials (cred_0, A_0) and (cred_1, A_1) of registered honest users whose secret keys $\text{usk}_0, \text{usk}_1$ remain with the challenger (there being no corruption oracle), together with a predicate ϕ with $\phi(A_0) = \phi(A_1) = 1$ and disclosed attributes $A^\downarrow$.
3. Well-formedness: the challenge is admissible only if $\text{Leak}(x_0) = \text{Leak}(x_1)$, where $\text{Leak}(\cdot)$ projects the statement onto its intentionally-disclosed components, namely $A^\downarrow$, the teaching-authority pseudonym multiset $\{\text{ppk}_{U,TA}^{(j)}\}$ (carried in the statement in the holder's canonical record order, so multiset equality equalises the ordered component), and any linking tag carried in ϕ ; the verifier-facing pseudonym $\text{ppk}_{U,V}$ is freshly sampled at showing time.
4. The challenger samples $b \leftarrow \{0, 1\}$ and returns $(x, \text{receipt}_b) \leftarrow \text{BDEC.ShowCre}$ for cred_b , run with the designated user's witness material, which the challenger holds.
5. $\mathcal{A}$, with continued oracle access, outputs b' .

$\text{Adv}_{\text{zkVM-Cred}}^{\text{anon}}(\mathcal{A}) = \left| \Pr[b' = b] - \frac{1}{2} \right|$. The bound established below for this advantage is made effective only when the receipt is zero-knowledge in w_{vm} ; on the shipped succinct-STARK path it is not (Section 6.3), so it is sound but not effective there. Non-cryptographic side channels (timing, ledger-access and revocation-update patterns, wallet metadata) are outside this model.

Experiment $\text{Exp}_{\text{zkVM-Cred}, \mathcal{A}}^{\text{unlink}}(\lambda)$ (**unlinkability**)

Model: classical PPT adversary, classical random-oracle model.

1. Setup and the *same* oracle interface as the unforgeability experiment ($\mathcal{O}_{\text{Issue}}$ and $\mathcal{O}_{\text{Show}}$), available adaptively to $\mathcal{A}$.
2. The challenger registers an honest user $(\text{sk}_U, \text{pk}_U)$ and creates two pseudonyms $\text{ppk}^0, \text{ppk}^1$, freshly and identically sampled, each bound to the user's key by its binding signature $\text{psk}^i \leftarrow \text{Sig.Sign}(\text{pp}_{\text{Sig}}, \text{sk}_U, \text{Encode}_{\text{Sig}}(\text{ppk}^i))$ (Section 3). The pseudonyms are internal to the challenger and are *not* given to $\mathcal{A}$, whose only view of them is through the challenge receipt below. The challenger creates an attribute set $A^\downarrow$, samples $b \leftarrow \{0, 1\}$, and issues a *single* credential $c^{(b)}$ with respect to $\text{ppk}^{(b)}$ on $A^\downarrow$.
3. The challenger runs ShowCre on $c^{(b)}$ under a freshly-sampled verifier-facing pseudonym $\text{ppk}_{U,V}$, holding the credential $c^{(b)}$ as a private witness, and forwards the resulting zero-knowledge showing receipt (with public statement x_{show} , not the raw credential) to $\mathcal{A}$.
4. $\mathcal{A}$ outputs b' .

$\text{Adv}_{\text{zkVM-Cred}}^{\text{unlink}}(\mathcal{A}) = \left| \Pr[b' = b] - \frac{1}{2} \right|$. When the statements intentionally carry a public linking tag, linkability is allowed only as that tag determines. The single-transcript structure is why the one-transcript simulation applies once, giving unlinkability the single-transcript ($k+2 \rightarrow 1$) specialisation of the anonymity bound.

The shipped-prototype game. Both credential relations commit their public statement, so $\text{JBind} = 1$ (Section 7). Where a guest commits only the verification outcome, a single boolean, $\text{JBind}(\text{out}^*, x^*) = 0$ for every nontrivial x^* ; modelling that case means deleting the JBind conjunct from the unforgeability win condition above. With that conjunct gone the extracted x^* is unpinned, and the adversary trivially satisfies statement-binding with any x^* consistent with the one committed boolean, so an accepting receipt attests merely “two signatures verify under *some* witness”. Such a receipt is a functional benchmark, not a statement-bound verifier; committing the public statement to the journal, as both credential relations now do, restores the conjunct and closes the gap.

Assumption 6.1 (Griffin-AIR constraint soundness). The Griffin- $\mathbb{F}_p^{192}$ AIR admits only the reference permutation: its constraint system is satisfied solely by a witnessed transition equal to reference Griffin- π , equivalently the arithmetisation soundness defect ϵ_{AIR} of Definition A.2 is zero. This assumption is *open*. It is argued at the specification level in Appendix B and empirically supported by the 256-vector cross-codebase equivalence suite together with rejection-direction fault injection on the seven per-row families (via `debug_constraints`) and, via a lookup-balance harness, the three lookup-borne families (memory binding partially); it is not established at the constraint level, that no wrong witness satisfies the constraints, for the lookup-borne families, and is not machine-checked in the sense of automated circuit-uniqueness checkers such as QED2 [67].

Assumption 6.2 (PLUM signature-transcript simulatability). PLUM admits a signature-transcript simulator with negligible error ϵ_{sZK} , in the sense of Definition A.3. This assumption is *open*: PLUM's

analysis establishes EUF-CMA security, not simulatability. It is required only by the anonymity and unlinkability bounds, whose BDEC reductions simulate the credential and pseudonym transcripts with the signature’s own simulator.

Assumption 6.3 (Credential-signature key-privacy). The PLUM credential signature $c_{U,TA}$, a public component of x_{cre} , hides its verification key (the signature analog of *key-privacy* [68], or *issuer-hiding* at the credential layer [69]): for honestly generated pk_0, pk_1 the distributions of $c_{U,TA}$ under pk_0 and under pk_1 on a common message are computationally indistinguishable, with distinguishing advantage $\epsilon_{keypriv}$. This assumption is *open* for PLUM, whose analysis establishes EUF-CMA but not key-privacy. BDEC’s Theorem 2 reduction simulates the credential under a random verification key, so key-privacy is the half of BDEC’s zero-knowledge-signature hypothesis that Assumption 6.2 does not capture: transcript simulatability hides the secret key, not the verification key, and does not imply key-privacy. Statement-binding of x_{cre} (Section 7) makes it essential rather than latent by making $c_{U,TA}$ provably public.

Assumption 6.4 (Griffin (Q)ROM instantiation). PLUM’s EUF-CMA argument under Fiat–Shamir models Griffin as a (quantum) random oracle [70, 71]; modelling a concrete sponge hash this way is justified by indifferentiability [72] only under an ideal-permutation assumption, which the algebraic-hash attack literature (Assumption 6.5(i)) bears on. The published Griffin analyses cover the original instantiations; whether they extend to the $d = 3$, width-4, 199-bit-field, 14-round instantiation used here is *open*, and the question is upstream of the AIR: were this instantiation (Q)ROM-distinguishable, no amount of AIR engineering would recover security.

Assumption 6.5 (Standard assumptions inherited from PLUM, Griffin, and SP1). The following are standard cryptographic assumptions carried unchanged from the components this thesis builds on; a reader who accepts PLUM, Griffin, and SP1 already accepts them, and they are not open problems introduced by this work. (i) *Griffin collision resistance*. Griffin at the deployed parameters (degree-3 S-box, width 4, 199-bit field, 14 rounds), used as PLUM’s Merkle-compression, hash-chain, and Fiat–Shamir hash, is collision-resistant; this is *open at the level of a bound*, no dedicated floor being established at these parameters, and the round-count margin is disclosed with Table 2. One output-width ceiling is unconditional: the deployed Merkle path truncates the two-lane Griffin digest to 32 bytes (one field element), so its collision resistance is birthday-capped at $\sqrt{p} \approx 2^{99.5}$ independently of round count, above the measured $\lambda = 80$ and below the $\lambda = 128$ headline. (ii) *Power-residue-PRF Q1 hardness*. The t -th power-residue PRF ($t = 256$, $\log_2 p \approx 199$) is key-recovery-hard against a quantum adversary restricted to classical (Q1) queries; the restriction is essential, since the Legendre ($t=2$) case falls to a polynomial-time Q2 attack [73], and the Q1 cost is analysed for $t=2$ [74] and inherited to $t = 256$. (iii) *Black-box transfer of PLUM’s reduction*. PLUM’s EUF-CMA reduction treats Griffin and field multiplication as black-box input–output maps (PLUM §3.2 [7], carrying over from Loquat [6]), so the value-level precompile substitution leaves it unchanged; the residual we carry is that no reduction step inspects an internal representation. The detailed status of each, including the best-known cryptanalysis, follows in the discussion below and in the round-margin disclosure; those references are cited there as *background*, not as open problems of this thesis.

Assumption 6.6 (Cross-table lookup-argument binding). The logarithmic-derivative (LogUp) cross-table lookup argument [75] binds the precompile tables to the main execution trace, forcing the range and multiset conditions on which every per-row lemma and the round-count, memory-binding, and

row-scheduling steps depend. This assumption is inherited from the SP1 implementation of its lookup argument and is not re-established here at the deployed parameters. It is jointly essential with Assumption 6.1: absent the binding a prover can leave every constraint polynomial vanishing while violating those conditions, so the chip accepts a non-reference Griffin trace, which through the unforgeability reduction’s AIR-mismatch hop is an accepted non-reference transition and a forged credential.

The power-residue PRF key-recovery cost at the deployed parameters. PLUM’s unforgeability rests on the hardness of recovering the PRF key from the public evaluations a signature exposes, and two figures bound that cost at the deployed parameters ($t = 256$, $\log_2 p \approx 199$, public-key length $L = 2^{12}$). They are not the same figure, and the difference is essential. The first is the generic collision ceiling. The best key-recovery strategies known for the Legendre ($t=2$) case run in time of order $\sqrt{p}$, so with $\log_2 p \approx 199$ the ceiling is $2^{99.5}$; this is the lowest cost any known attack reaches, and it clears $\lambda = 80$ while sitting below the $\lambda = 128$ headline. The second figure is the operative one, because that ceiling is attained only once the attacker holds about $p^{1/4}$ symbols. The data-dependent attack of Beullens, Beyne, Udovenko and Vitto [76] recovers the key from L available symbols in time of order p/L^2 up to polylogarithmic factors, using memory of order L , improving the linear-in-data bound of Khovratovich [65] (preprocessing variants trade offline for online cost [66]). At the deployed public-key length $L = 2^{12}$ this is

$$\frac{p}{L^2} \approx \frac{2^{199}}{2^{24}} = 2^{175}.$$

Here $L = 2^{12}$ is the *total* exposed symbol count and not a per-query figure: a signature and its verifier reveal the PRF only on the fixed public index set $I = (I_1, \dots, I_{2^{12}})$, so additional signing queries do not enlarge the symbol set available to the attacker (this same fixed-index structure is what pins the quantum access model discussed below). The polylogarithmic factors only raise the 2^{175} figure, to roughly 2^{183} with a single $\log_2 p$ factor. The two figures coincide only at $L = p^{1/4} = 2^{49.75} \approx 2^{50}$, where $p/L^2 = \sqrt{p}$; the signature exposes $L = 2^{12}$ symbols, far below that crossover, so the operative cost is the 2^{175} figure rather than the $2^{99.5}$ ceiling. Under the data the deployed scheme actually exposes, power-residue key recovery therefore clears both $\lambda = 80$ (the measured point) and $\lambda = 128$ (the headline); the unconditional $\sqrt{p}$ ceiling clears $\lambda = 80$ but not $\lambda = 128$, which is why the bound on the exposed data, and not the ceiling, is the statement that carries the headline security level. Both figures are established for the Legendre case. For $t = 256$ each symbol carries $\log_2 t = 8$ bits rather than one and the power-residue ($t > 2$) key-recovery literature is thin [76], so we inherit these costs by the assumption already carried in Assumption 6.5(ii), that the t -th residue PRF inherits Legendre’s cryptanalytic status, rather than by a $t = 256$ derivation.

BDEC’s Theorems 1 through 3 [5] already prove these properties, so this section does not re-prove them. It bounds how far the move to a zkVM substrate shifts each advantage.

The parties are the teaching authorities and the issuer, who hold signing keys, together with the holder and the relying party. The ledger roots $\text{rt}_{\text{cred},e}$, $\text{rt}_{TA,e}$, and $\text{rt}_{LR,e}$ are public and authenticated. The prover is the holder generating a showing, and the verifier is the relying party, which chooses the presentation predicate ϕ at presentation time. The adversary is the standard anonymous-credential adversary of Section 3, lifted to the zkVM execution setting: it may request issuance of credentials on attribute vectors of its choice, observe showings produced by honest holders, act as a malicious verifier choosing ϕ adaptively, and act as a malicious prover attempting to produce an accepting showing it should not be able to prove. Following PLUM, the unforgeability we inherit is established against

classical adversaries in the classical random-oracle model. Its post-quantum reading rests on the underlying power-residue PRF (whose Legendre, $t=2$, case is analysed in [74]) and Griffin’s collision resistance at the deployed parameters (Assumption 6.5(i)) being quantum-hard, whereas the lift of the EUF-CMA reduction itself to the quantum random-oracle model is a separate question that PLUM does not settle and that we do not re-establish here. That such a lift is *attainable* for this PRF family is shown by Pegasus [31], which proves (Q)ROM security for a power-residue-PRF signature; Pegasus is a different construction (a commit-and-open Σ -protocol, not PLUM’s STIR-based design), so it does not close PLUM’s own lift but it removes any presumption that the PRF family itself is the obstacle.

The quantum access model for the PRF. The post-quantum reading of unforgeability above assumes the power-residue PRF is quantum-hard, which requires pinning the query model. Assumption 6.5(ii) states it in the Q1 model, the adversary restricted to classical PRF queries: the signature and its verifier query the PRF only on the fixed public index set, so no superposition (Q2) query ever arises, and the restriction is essential because under Q2 access even the Legendre ($t=2$) case falls to a polynomial-time quantum key recovery [73].

Side-channel and timing attacks on the prover host, denial of service, and the physical security of signing keys are out of scope, since they are orthogonal to the substrate question this thesis studies. The trusted computing base of the zkVM regime is the RISC-V execution circuit, the precompile chips, and the lookup argument that binds them to the main execution trace.

The preservation theorem of Section 6.2 and the precompile-soundness theorem of Section 5 rest on three of the five named open assumptions, Assumptions 6.1 and 6.2 and the cross-table lookup-argument binding of Assumption 6.6 (a fourth, the quantum-random-oracle lift of Assumption 6.4, is upstream of the preservation theorem; the fifth, signature key-privacy of Assumption 6.3, is required only by issuance anonymity), together with one further named assumption, the black-box-transfer property (Assumption 6.5(iii), taken from PLUM’s reduction structure, stated with the inherited bundle above). The lookup-binding hypothesis, that the cross-table lookup argument binds the precompile chips to the execution trace, is assumed from the SP1 implementation (its logarithmic-derivative lookup argument [75]), not independently verified here. Assumption 6.1, that the Griffin- $\mathbb{F}_p^{192}$ AIR admits only the reference permutation, equivalently that the defect ϵ_{AIR} is zero, is partially discharged and partially open; its constraint-family breakdown and the test coverage it rests on are given with the discussion after Theorem 6.5, where the assumption is essential. The black-box-transfer hypothesis, that PLUM’s EUF-CMA reduction treats Griffin and field multiplication as black-box input–output maps, is taken from PLUM’s own reduction (§5), which programs the hash as a random oracle and computes field operations at the value level (PLUM §3.2 [7], whose building blocks are treated as black boxes so the proofs carry over from Loquat [6]), which opens no internal representation. It is distinct from Assumption 6.1, which separately asserts that the precompile computes the reference permutation. Assumption 6.2, that PLUM admits a signature-transcript simulator, equivalently that ϵ_{sZK} is negligible, is open and is required only by the anonymity and unlinkability bounds, PLUM’s analysis establishing EUF-CMA security but not simulatability. The zkVM is additionally assumed knowledge-sound (for the composite multi-shard receipt this rests on Conjecture 1 below) and the modular-multiplication precompile sound, the latter established for SP1 (§5.2); the quantum-random-oracle lift of PLUM’s EUF-CMA reduction and of the nested Fiat–Shamir transform is open, the analysis being in the classical random-oracle model.

Remark 6.2 (Field-boundary non-malleability). The one representation change the precompile substi-

tution introduces is the decomposition of each $\mathbb{F}_p$ element of the statement and witness into the prover field’s limbs. This crossing adds no malleability of the relation beyond the assumptions already named. The prover does have latitude in the limb witness, the chip admitting non-canonical representatives in intermediate cells (Appendix B, where the carry and witness columns vary with the representatives chosen), but that latitude is value-preserving: every admitted representative is congruent modulo p , and each output lane is canonicalised to its representative in $[0, p)$ (Lemma B.5), so a re-encoding changes the trace without changing the represented $\mathbb{F}_p$ value, the permutation output, or the statement x that JBind commits. PLUM’s reduction moreover operates on $\mathbb{F}_p$ values, not on their limb encodings (Assumption 6.5(iii)), so the encoding is transparent to it. The output canonicalisation is proved per row (Lemma B.5) conditional on the byte-range lookups whose binding is Assumption 6.6; the field boundary therefore rests only on assumptions already named (Assumptions 6.5(iii) and 6.6, and the per-row soundness of Assumption 6.1) and introduces no new hypothesis.

What a deployment must deliver. The three experiments fix *what must be guaranteed*. Before asking what the substrate preserves, we name the target as a single predicate.

Definition 6.3 (Deployable post-quantum anonymity). A deployment delivers *deployable post-quantum anonymity* for the credential system if the holder’s machine can produce, within its resource budget, a proof of the credential relation that is simultaneously (i) knowledge-sound, (ii) zero-knowledge, and (iii) post-quantum-sound: soundness and the zero-knowledge simulator resting on no assumption a quantum adversary breaks. Section 6.2 establishes which of these conditions transfer to the zkVM substrate; Section 6.3 establishes which does not.

6.2 Security Preservation

The analysis rests on one structural observation. The zkVM regime changes how the credential relation $R_{\text{show}}^{\text{Sig}}$ (and the structurally analogous credential-generation relation $R_{\text{cre}}^{\text{Sig}}$) is proved; the relation itself, what it states, is unchanged. The relation is fixed in Section 3, and both the static-circuit and the zkVM regimes are proof systems for that same relation. Preservation therefore reduces to two claims: that the substrate computes the same relation, and that its proof system supplies the soundness and zero-knowledge that BDEC’s reductions consume. We treat the two in turn.

Lemma 6.4 (Relation preservation). *Suppose the host modular-multiplication precompile is sound, the cross-table lookup argument binds the precompile tables to the main execution trace (Assumption 6.6), the zkVM is a knowledge-sound argument of knowledge for RISC-V execution, the Griffin- $\mathbb{F}_p^{192}$ AIR admits only the reference permutation (Assumption 6.1), and the guest evaluation of R^{Sig} decomposes into a finite sequence of operations, each a large-field modular multiplication, a Griffin permutation, or a native RISC-V instruction. Then the zkVM guest program with the precompile suite active accepts a pair (x, w) if and only if $R^{\text{Sig}}(x; w) = 1$, where R^{Sig} denotes either of the credential relations $R_{\text{cre}}^{\text{Sig}}, R_{\text{show}}^{\text{Sig}}$ of Section 3. The lemma is stated at the level of the intended guest specification; the code-level claim additionally requires that the zkVM journal be bound to the public statement x . For both credential relations $R_{\text{cre}}^{\text{Sig}}$ and $R_{\text{show}}^{\text{Sig}}$ this binding is now implemented (Section 7), so the code-level reading is discharged: an accepting receipt certifies which statement was evaluated.*

Proof sketch. A guest evaluation of the credential relations invokes only large-field multiplications and Griffin permutations, which the precompiles return identically under the lookup binding, together

with native control flow covered by the zkVM’s knowledge-soundness; composing these per-operation equalities along the finite, input-fixed execution yields equality of the accept-or-reject decision at the program level. The Griffin step’s constraint-level soundness rests on the AIR argument of Appendix B, so the equality holds modulo Assumption 6.1; the execute-mode runs of Section 7 confirm the guest program’s I/O but do not exercise the AIR constraints. The step-by-step argument, including the $k + 2$ presentation predicates, is given in Appendix A. $\square$

Theorem 6.5 (Security preservation, conditional under classical ROM). *Suppose PLUM is EUF-CMA secure in the classical random-oracle model and, for the anonymity and unlinkability bounds only, additionally admits a signature-transcript simulator with error ϵ_{sZK} (Assumption 6.2, an open assumption parallel to Assumption 6.1: PLUM’s analysis establishes only EUF-CMA, whereas BDEC’s Theorems 2–3 simulate the credential and pseudonym transcripts with the signature’s own simulator). Suppose the zkVM is a knowledge-sound argument of knowledge for RISC-V execution, the host modular-multiplication precompile is sound, and the cross-table lookup argument binds the precompile tables to the trace (Assumption 6.6). Suppose further, as the black-box-transfer hypothesis (Assumption 6.5(iii)), that PLUM’s EUF-CMA reduction treats Griffin and field multiplication as black-box input–output maps, so that the precompile substitution leaves that reduction unchanged. Suppose further, as the unverified Assumption 6.1, that the Griffin- $\mathbb{F}_p^{192}$ AIR admits only the reference permutation; this assumption is argued in Appendix B but not established at the constraint level. Then for every classical adversary $\mathcal{A}$ against the PLUM-instantiated, zkVM-executed BDEC system, in the random-oracle model, the advantage is bounded by five leaf error terms (a list distinct from the five open assumptions): ϵ_{EUF} (PLUM’s EUF-CMA error), ϵ_{KS} (the zkVM argument’s knowledge-soundness error), ϵ_{AIR} (the Griffin-AIR constraint-level soundness defect), ϵ_{ZK} (the produced proof’s zero-knowledge error), and ϵ_{sZK} (PLUM’s signature-transcript-simulation error), assembled as the bounds in Equations (5) and (6) below. The anonymity bound carries a factor $k + 2$ for the showing’s $k + 2$ signature transcripts, and the unforgeability bound the factors N (an honest-key guess) and N_{G} (Griffin permutations per verification). The unlinkability bound is $\epsilon_{\text{sZK}} + \epsilon_{\text{ZK}}$, the single-transcript ($k + 2 \rightarrow 1$) case of (6), since its game (BDEC Theorem 3) forwards a single challenge credential. These bounds do not map one-to-one onto BDEC’s three theorems [5]. Equation (5) re-instantiates Theorem 1, and the unlinkability bound, its single-transcript case, re-instantiates Theorem 3, both with PLUM as the signature scheme and the zkVM as the zkSNARK. Equation (6) bounds anonymity of the showing. BDEC’s Theorem 2, by contrast, states anonymity on the issuance proof (verified against its Appendix A.2, whose simulator reconstructs the credential-generation transcript); its re-instantiation is the separate bound $2\epsilon_{\text{sZK}} + \epsilon_{\text{ZK}} + \epsilon_{\text{keypriv}}$ derived in the paragraph “Anonymity of the issuance proof” below, under the additional open Assumption 6.3. The honest count is thus four advantage expressions across the three theorems, unforgeability, showing-anonymity, unlinkability, and issuance-anonymity. The first three are assembled from the five leaf errors just named; issuance anonymity additionally carries a sixth, $\epsilon_{\text{keypriv}}$ (Assumption 6.3), entering no other bound. Their sources and status are set out below.*

Proof sketch. By Lemma 6.4 the zkVM proves the same relation as BDEC’s static circuit, so we re-instantiate BDEC’s three reductions with PLUM as the signature scheme and the zkVM as the zkSNARK, each as a labelled game sequence $G_0 \rightarrow G_1 \rightarrow G_2$ on a single probability space; all bounds are for classical $\mathcal{A}$ in the random-oracle model.

Unforgeability is two failure-event hops followed by a signature-forgery terminal, each hop bounded by Shoup’s Difference Lemma. The first hop (F_1 , extractor failure) runs the knowledge extractor of

Definition A.1 on the accepting receipt and aborts if it returns no witness, at cost ϵ_{KS} . The second hop (F_2 , AIR mismatch) aborts if any one of the $N_G \approx 1052$ committed Griffin transitions differs from the reference permutation, at cost $N_G \epsilon_{\text{AIR}}$ by a union bound over the committed permutations and the open Assumption 6.1. On the surviving path the extracted signature is a genuine PLUM signature, which the terminal embeds into PLUM’s EUF-CMA experiment under an honestly guessed key, at cost $N \epsilon_{\text{EUF}}$ for the $1/N$ guess. Summing the chain gives Equation (5).

Anonymity is a hybrid of indistinguishability hops with an information-theoretic terminal, and extracts no witness, so neither ϵ_{KS} nor ϵ_{AIR} enters. The first hop is a hybrid of $k + 2$ sub-hops that replace the showing’s $k + 2$ PLUM signature transcripts, one at a time, by the signature-transcript simulator of Assumption 6.2, at total cost $(k + 2) \epsilon_{\text{sZK}}$. The second hop replaces the one proof by the zkVM’s zero-knowledge simulator, at cost ϵ_{ZK} . The terminal is information-theoretic: with every transcript and the proof simulated, the challenge statement is equalised across the two credentials by the well-formedness condition $\text{Leak}(x_0) = \text{Leak}(x_1)$ and the freshly sampled verifier-facing pseudonym, so the adversary’s view is independent of b and its success probability is exactly $\frac{1}{2}$. This gives Equation (6).

Unlinkability is the single-transcript specialisation of the anonymity sequence. Its game forwards one challenge credential, so the hybrid collapses to a single simulator hop at cost ϵ_{sZK} , one zero-knowledge hop at cost ϵ_{ZK} , and the same information-theoretic terminal, giving $\epsilon_{\text{sZK}} + \epsilon_{\text{ZK}}$, the $k + 2 \rightarrow 1$ case of Equation (6).

Because BDEC proves its three theorems for a generic (signature, zkSNARK) pair [5], substituting PLUM and the zkVM changes nothing beyond these leaf errors. The step-by-step reductions, with the explicit interpolating distinguishers $\mathcal{D}_\sigma^{(i)}$ and $\mathcal{D}_\Pi$, the forger $\mathcal{B}$ with its honest-key guess, and the freshness argument, are given in Appendix A. On the target hardware the bare receipt does not make anonymity and unlinkability effective, since ϵ_{ZK} is not driven small there; the pairing wrap that completes within budget drives it small only against a classical distinguisher, not a quantum one (Section 6.3). $\square$

Remark 6.6 (Deployment reading of the anonymity bound). Equation (6) bounds anonymity by the zero-knowledge error ϵ_{ZK} of the produced proof. On the bare succinct receipt that error is not made small, since that receipt has no established zero-knowledge simulator; the wrap that would supply one completes within the consumer memory budget, but it is pairing-based, so it drives ϵ_{ZK} small only against a classical distinguisher and not against a quantum one. The anonymity and unlinkability bounds are therefore guarantees the construction inherits from BDEC under a zero-knowledge argument, and which distinguisher the substrate makes them effective against is settled in Section 6.3: classical, via the completing pairing wrap, but not post-quantum.

The assembled bounds of this chapter are sums of leaf error terms. We state, term by term, what *form* of bound each source delivers. None of the five is a reproduced concrete or parameterised expression. Every one resolves to “negligible”, and three of them (ϵ_{AIR} , ϵ_{sZK} , and ϵ_{ZK} on the shipped stack) are *open or not-met* rather than established, as is the issuance-only $\epsilon_{\text{keypriv}}$. In particular, the $2^{-199}/2^{-191}$ figures are executor-equivalence test *coverage*, explicitly *not* a soundness bound on ϵ_{AIR} ; see the discussion of Assumption 6.1 below.

With these forms, the assembled bounds read, in the classical random-oracle model,

$$\text{Adv}_{\text{zkVM-Cred}}^{\text{unf}}(\mathcal{A}) \leq N \epsilon_{\text{EUF}} + \epsilon_{\text{KS}} + N_G \epsilon_{\text{AIR}}, \quad (5)$$

$$\text{Adv}_{\text{zkVM-Cred}}^{\text{anon}}(\mathcal{A}) \leq (k + 2) \epsilon_{\text{sZK}} + \epsilon_{\text{ZK}}, \quad (6)$$

where N is the number of honestly generated signing keys in the registry (the reciprocal of the forger’s honest-key guess probability $1/N$), $N_G \approx 1052$ is the number of Griffin permutations a PLUM verification commits (over which the AIR-mismatch event union-bounds), and $k+2$ is the number of PLUM signature transcripts the challenge showing carries under pk_U (the k teaching-authority pseudonym signatures, the verifier-facing pseudonym signature, and the shown credential), each simulated in one hybrid sub-hop. For unlinkability, whose game forwards a *single* challenge credential, the hybrid collapses to one transcript and

$$\text{Adv}_{\text{zkVM-Cred}}^{\text{unlink}}(\mathcal{A}) \leq \epsilon_{\text{sZK}} + \epsilon_{\text{ZK}},$$

the $k+2 \rightarrow 1$ specialisation of (6). Unforgeability is negligible whenever its three terms are (the polynomial factors N, N_G preserve negligibility), of which ϵ_{AIR} is the open term. The anonymity and unlinkability bounds are negligible only when ϵ_{ZK} is (the polynomial factor $k+2$ preserves this), which the shipped bare receipt does not deliver and the completing pairing wrap delivers only against a classical distinguisher (Section 6.3); on the bare path Equation (6) is sound but not made effective.

Anonymity of the issuance proof. Equation (6) and the anonymity experiment above are stated on the *showing*. BDEC’s Theorem 2 states anonymity on the *issuance* proof instead: its challenge forwards the credential-generation output $(\text{ppk}_{U,TA}, c_{U,TA}, \varepsilon_{c_{U,TA}})$. In base BDEC that output is published; the variant of Section 3 anchors only the record digest $\rho_{U,TA} = H(\cdot)$, so whether the raw proof $\varepsilon_{c_{U,TA}}$ is itself exposed is a deployment choice. Wherever it is exposed, issuance anonymity requires $\varepsilon_{c_{U,TA}}$ to be zero-knowledge, because its witness $w_{\text{cre}} = (\text{pk}_U, \text{psk}_{U,TA})$ carries the same long-term key pk_U that anchors every showing: a non-zero-knowledge $\varepsilon_{c_{U,TA}}$ exposing pk_U would let an adversary join a user’s issuance records under that key and, through the intentionally-disclosed teaching-authority pseudonyms $\{\text{ppk}_{U,TA}^{(j)}\}$ that every showing statement carries, link the user’s showings, defeating Equation (6) and its unlinkability specialisation regardless of the showing proof’s own zero-knowledge. This is precisely why the produced system wraps CreGen and not only ShowCre (§7.4.1). The transfer is *not* the identical hybrid, however. Unlike the shown credential $c_{U,V}$, which is a witness (§3) hidden by the proof simulator, the credential $c_{U,TA}$ is a *public* component of x_{cre} , and once x_{cre} is committed it is provably so (Section 7); replacing it by the transcript simulator of Assumption 6.2 leaves it a signature under the challenge key pk_b . Equalising the terminal across the two challenge identities therefore needs the credential signature to *hide its own verification key* pk_U , a signature key-privacy property that the transcript-simulatability of Assumption 6.2 does not provide, since that simulator hides the secret key and not the verification key; we carry it as Assumption 6.3. Under Assumptions 6.2 and 6.3 the issuance terminal equalises, $h_{U,TA} = H(A^*)$ under a common disclosed set, $\text{ppk}_{U,TA}$ freshly uniform, and $c_{U,TA}$ key-private, and Theorem 2 gives the two-transcript issuance-side bound $2\epsilon_{\text{sZK}} + \epsilon_{\text{ZK}} + \epsilon_{\text{keypriv}}$, the key-privacy term entering only here, conditional on Assumption 6.3 (signature key-privacy) *in addition to* Assumption 6.2 and the zero-knowledge wrap. One modelling point is inherited, not introduced: base BDEC’s issuance already has the teaching authority sign σ_{cred} over $\text{Encode}_{\text{sig}}(\text{pk}_U, \dots)$ (§3), so the issuing authority learns pk_U regardless of any proof’s zero-knowledge; issuance anonymity is therefore against every party other than the issuing authority, already implicit in base BDEC and named here for clarity.

The hypotheses of Theorem 6.5 are not all equally settled, and the honest reading of the theorem turns on which are established and which remain open. PLUM’s EUF-CMA security is taken from its specification and the Loquat analysis it builds on [6], which establish it in the classical random-oracle model against classical adversaries; the lift of that reduction to the quantum random-oracle model is

open, so the unforgeability bound holds against classical adversaries and its quantum case inherits that open lift. The soundness of the modular-multiplication precompile is established for SP1 by the modular-multiplication reduction of §5.2, while the knowledge-soundness hypothesis is obtained by applying the STARK and BCS soundness analyses [57] to SP1’s concrete argument; the cross-table lookup-argument binding (the lookup-binding hypothesis [75]) is instead assumed from the SP1 implementation of its logarithmic-derivative lookup argument, whose published soundness analysis we do not re-establish here. These last are themselves obtained through the Fiat–Shamir compilation [77], whose soundness is a random-oracle property invoked twice and nested, PLUM inside the zkVM, and whose quantum case is again open.

The soundness parameter B at the measured security level. The measured point is $\lambda = 80$, at which the implementation sets $B = 16$ challenged residuosity symbols, with $m = 4$ and $n = 4$ so that the stored symbol count $m \cdot n$ equals B . PLUM tabulates its residuosity parameters only at $\lambda = 128$, where §3.3 fixes $(m, n, B, \kappa_0) = (4, 7, 28, 26)$ with $\beta = 0.961$ and $L = 2^{12}$. The $\lambda = 80$ profile is therefore not read from the paper. It keeps the paper-fixed sumcheck dimension $m = 4$ and sets $n = 4$, and the value $B = 16$ follows the implementation’s own rule, an integer buffer over the floor $\lceil \lambda / \log_2 t \rceil$. That floor is the point at which a cheating prover who guesses each residuosity symbol with probability about $1/t = 2^{-8}$ is caught except with probability $t^{-B} = 2^{-\lambda}$: at $\lambda = 80$ it is $\lceil 80/8 \rceil = 10$, and the implementation applies a $1.6\times$ buffer to reach 16. This buffer is the implementation’s, not the paper’s. PLUM does not select B by any buffer rule: at $\lambda = 128$ it derives $B = 28$ from its own residuosity-soundness expression in β and L (§3.3), and 28 only happens to sit at $1.75\times$ the floor $\lceil 128/8 \rceil = 16$ when that ratio is formed after the fact. Because the descriptive ratio at $\lambda = 128$ ($1.75\times$) differs from the implementation’s $1.6\times$ at $\lambda = 80$, no single buffer rule reproduces both $B = 28$ and $B = 16$, and the paper tabulates only $\lambda = 128$. The $B = 16$ choice is therefore an implementer extrapolation rather than a value calibrated by PLUM’s own soundness expression. Any statement that the deployed configuration is “80-bit secure” at the identification level rests on this extrapolation, and we state it as such.

The open hypotheses among these each deserve their necessity, their best-known cryptanalysis, and their honest tightness stated plainly; the discussion below takes them in turn.

The lookup-binding hypothesis. SP1 realises the binding of Assumption 6.6 with a logarithmic-derivative (LogUp) lookup argument [75], deployed in the GKR-accelerated variant [78], whose soundness is a multiset-equality check that degrades with the number of lookup entries relative to the size of the Fiat–Shamir challenge field. We inherit that generic analysis and do not re-derive it for these chips at these parameters; generic LogUp soundness is not the same statement as the soundness of the byte-table and cross-table arguments as instantiated here. We record its status as negligible by the cited analysis, with no concrete bound established at the deployed parameters, assumed from the SP1 implementation rather than verified here.

A concrete bound for the lookup argument. The lookup-binding hypothesis above is carried without a concrete error at the deployed parameters, and the generic bound is fillable. The soundness of a logarithmic-derivative (LogUp) lookup argument [75] is that of a rational-identity check over the

Fiat–Shamir challenge field: it fails with probability at most

$$\epsilon_{\text{lookup}} \leq \frac{M \cdot d}{|K|},$$

where M is the number of lookup entries, d is the degree of the fractional-sum identity, of order the number of tables, and $|K|$ is the size of the challenge field. On the SP1 stack the challenge field is the degree-4 extension of KoalaBear (`SP1ExtensionField = BinomialExtensionField<KoalaBear, 4`), so $|K| = (2^{31} - 2^{24} + 1)^4 \approx 2^{124}$. The entry count M is bounded by the trace, the byte-range and cross-table sends over the $N_G \approx 1052$ Griffin permutations together with the RISC-V shard lookups, which for a PLUM verification stays well under 2^{30} . Taking $M \cdot d \leq 2^{34}$ generously,

$$\epsilon_{\text{lookup}} \leq 2^{34-124} = 2^{-90},$$

negligible at both security levels. What this fills is the generic bound only. It is a different statement from the soundness of SP1’s specific byte-table, cross-table, and memory-consistency arguments as instantiated, which is a wiring property of those particular multiset arguments and needs machine-checking rather than a formula. We therefore report $\epsilon_{\text{lookup}} \lesssim 2^{-90}$ as the generic figure and keep the as-instantiated soundness of these chips flagged open. This lookup error is not a separate leaf in the assembled bounds. The lookup argument is a sub-protocol of SP1’s STARK, so a lookup failure is one route by which its knowledge extractor could be fooled, and ϵ_{lookup} is therefore one contributor already inside the knowledge-soundness error ϵ_{KS} of Definition A.1, not a separate leaf term of its own in the assembled unforgeability bound.

The knowledge-soundness error ϵ_{KS} . The $G_0 \rightarrow G_1$ hop spends ϵ_{KS} to run the extractor of Definition A.1 and recover the signature witness from the accepting receipt. Absent extraction there is no signature to embed into PLUM’s EUF-CMA experiment, so the unforgeability reduction fails at its first hop. SP1 compiles its interactive oracle proof to a non-interactive argument through the BCS and Fiat–Shamir transforms, and we take its knowledge-soundness in the IOP-and-extractor sense [57]; the deployed commitment is multilinear (Basefold over KoalaBear), not univariate FRI, as the next paragraph reads off the code. We do not reproduce the concrete knowledge error at $\lambda = 80$ from first principles for the shipped blow-up, query count, and grinding parameters; at this point in the argument the status is generic, “up to a negligible knowledge error”, inherited from SP1’s released argument. The proximity regime in which that error is claimed, and the concrete target of the deployed configuration, are read from the code in the next paragraph; a formal re-derivation of the Basefold bound at those parameters remains future work.

The knowledge-soundness regime and parameters, named. The paragraph above left ϵ_{KS} at the deployed parameters unnamed. The forked SP1 v6.2.1 that the measurements run fixes them in code. The core and recursion arguments are a Basefold and jagged multilinear commitment over the KoalaBear field ($p = 2^{31} - 2^{24} + 1$), not univariate FRI, and the deployed core and recursion verifiers construct it (`from_basefold_parameters`) at *log-blowup* 2 (a blowup *factor* of 4), that is code rate $1/4$, with the query count set in the proven unique-decoding regime, where a word at the unique-decoding radius passes a single query with probability at most $(1 + \rho)/2$ at code rate ρ [79]: at rate $1/4$ this is $\lceil (100 - 16)/(-\log_2(0.5 + \frac{1}{4} \cdot \frac{1}{2})) \rceil = \lceil 84/0.678 \rceil = 124$ queries, with 16 grinding bits,

targeting a fixed 100 bits of security. The measured core commitment therefore sits in the proven unique-decoding regime [80]. A lower-rate configuration does exist in the fork, *log-blowup* 1 (factor 2), rate 1/2, 94 queries, whose own source comment records that it “relies on a conjecture for achieving 100 bits of security” (the query count raised from 84 to 94 to absorb Gruen and Diamond’s proximity-gap bound [81]); but it is the toolkit’s `default_fri_config`, exercised only in test harnesses, dummy-proof scaffolds, and the unused `slop-veil` crate, never on the production core, recursion, shrink, or wrap prover path. Two points refine the estimate above. First, the soundness target is a fixed 100 bits and is decoupled from the signature security level: $\epsilon_{KS} \approx 2^{-100}$ on this stack whether $\lambda = 80$ or $\lambda = 128$, so “ ϵ_{KS} at $\lambda = 80$ ” mislocates it. Second, because the measured path is a Basefold multilinear commitment over KoalaBear, its soundness lineage is the Basefold analysis [82] in the unique-decoding regime, rather than the univariate-FRI ethSTARK analysis [83]. The honest statement is therefore that the measured core and recursion path sits in the proven unique-decoding proximity regime, rate 1/4, 124 queries, 16 grinding bits, targeting 2^{-100} ; the outer prover’s ϵ_{KS} does not rest on a proximity-gaps conjecture, and anonymity does not depend on ϵ_{KS} in any case. The 2^{-100} figure is classical, and the quantum-random-oracle status of the extractor is a separate open question.

Conjecture 1 (Knowledge extraction through the SP1 shard and recursion tree). *Let the core receipt for a PLUM verification consist of S shard proofs bound by SP1’s global cross-shard permutation and logarithmic-derivative argument, and, for a wrapped receipt, let the succinct proof be the recursion and compression tree of depth D (recursion Basefold commitment at log-blowup 2, rate 1/4, with unique-decoding queries, terminating in the shrink and wrap layers at blow-up 3 with 22 grinding bits). Then there is a straight-line random-oracle extractor $\mathcal{E}$ for the whole receipt with*

$$\epsilon_{KS} \leq \sum_{i \leq S} \epsilon_{\text{shard},i} + \epsilon_{\text{xshard}} + \sum_{j \leq D} \epsilon_{\text{rec},j},$$

where $\epsilon_{\text{shard},i}$ is the per-shard Basefold knowledge error at the core commitment configuration (log-blowup 2, rate 1/4, unique decoding, about 2^{-100}), ϵ_{xshard} is the cross-shard consistency soundness, and $\epsilon_{\text{rec},j}$ is the recursion-verifier circuit’s knowledge-soundness at layer j , which subsumes that circuit’s own arithmetisation fidelity. Each term is negligible and the sum is negligible for the deployed shard count S and depth D .

This conjecture is stated, not proved, and the scope distinction matters. The multi-shard term is not a property of a hypothetical wrap: the measured succinct receipt is already a multi-shard core receipt, and the unforgeability reduction spends ϵ_{KS} at hop F_1 on exactly that composite object. Definition A.1 treats the argument as a single monolithic extractor and the cited per-shard analysis (Basefold [82]) is a single-proof extractor, so the extractor for the composite multi-shard receipt, per-shard extraction together with cross-shard consistency, is neither constructed nor cited here. It therefore touches the current core claim and not only a wrapped one. If the conjecture fails, unforgeability is what is lost, at hop F_1 : no witness is extracted and there is no PLUM signature to embed into the EUF-CMA experiment. The status is classical random-oracle, the straight-line Merkle-commitment extractor being classical, and quantum-random-oracle composition of the whole tree is a further open question. No published end-to-end extractor for the SP1 shard-and-recursion stack exists; here S is the measured shard count of the run and D the recursion tree depth, and the proof of ϵ_{xshard} and of the composite extractor is left as an open problem.

The black-box-transfer hypothesis. The freshness step of the unforgeability reduction (Appendix A) hands the extracted pair to BDEC’s own EUF-CMA embedding and relies on the precompile substitution leaving every input of that reduction unchanged (Assumption 6.5(iii)). Absent this, a reduction step that inspected an intermediate representation, a limb of a field product or an internal Griffin state that the value-level substitution does not reproduce, would break, and BDEC’s freshness argument would not transfer.

Assumption 6.2 (PLUM signature-transcript simulatability). The $G_0 \rightarrow G_1$ hybrid of the anonymity reduction produces the challenge showing’s $k+2$ transcripts without the challenge user’s sk_U by calling the simulator of Definition A.3. Absent such a simulator the challenger cannot produce the challenge showing without sk_U , and the anonymity and unlinkability reductions have no first hop. PLUM’s analysis establishes EUF-CMA security, which does not by itself furnish a transcript simulator; the gap is precisely that a simulator is required and none is exhibited. The property is not without support, since PLUM and Loquat build on a sumcheck-and-low-degree-test IOP that is honest-verifier zero-knowledge in its masked variant, which is the mechanism a simulator would use, but PLUM does not carry that through to a signature-transcript simulator. The status is asserted negligible with no formula, and open.

AIR completeness (Lemma B.7). This is the one property here whose failure is not an adversarial advantage. A completeness failure aborts an honest prover and costs the 14.24-minute feasibility figure, it does not let an adversary win any of the three experiments. It is the wrong direction to attack, and we discharge it operationally, by the single-permutation smoke proof and the fault-injection suite’s positive control, following the practice of the verified Cairo work [84]. It stays a lemma supported by tests rather than a named assumption, and would need elevation only if a completeness edge case, such as a non-canonical intermediate cell causing an honest abort, were suspected; none is.

The one hypothesis below the level of an established or cited result is Assumption 6.1, the Griffin AIR’s constraint-level soundness. The chip’s compute matches the reference permutation on the 256-vector equivalence suite, with per-vector false-pass at most $1/p \approx 2^{-199}$ and aggregate at most 2^{-191} under a uniform-wrong-output model (Appendix B; a deterministic bug agreeing on the sampled inputs is not bounded by these figures). This figure is the test *coverage* of a positive-direction equivalence suite, run on valid inputs, and is not a soundness bound: it does not show that the constraint system *rejects* a malformed Griffin trace, which is the direction Assumption 6.1 actually asserts. The equality of the constraint system with the reference is argued rather than machine-checked in Appendix B. Two further points require disclosure. The implementation fixes a concrete 199-bit prime $p = 2^{64}p_0 + 1$, so any security-bit claim that depends on the exact value of p is conditional on this instantiation matching PLUM’s intended parameters. And whether Griffin at the instantiation used here, a degree-3 S-box over a width-4 state of the 199-bit field with 14 rounds, may be replaced by a random oracle in PLUM’s EUF-CMA argument is a question the AIR cannot settle and that we carry as open. The reduction of Theorem 6.5 is therefore sound *modulo its hypotheses*, but three of its inputs are not yet closed: the constraint-level AIR equality (Assumption 6.1), PLUM’s signature-transcript simulatability (Assumption 6.2), and the quantum lifts of PLUM and of Fiat–Shamir. The black-box-transfer property is argued separately from PLUM’s reduction structure, which programs the hash as a random oracle and never opens its internals (PLUM §3.2), a reduction we have not audited line by line. We state the theorem conditionally on them rather than assert guarantees the cited proofs do not

establish. The conservative reading is explicit: absent a PLUM transcript simulator (Assumption 6.2), only the EUF-CMA-derived *unforgeability* of the credential transfers, and anonymity and unlinkability are conditional on Assumption 6.2; absent the quantum lift, every guarantee holds against classical adversaries only. For each open premise we therefore state which guarantee survives it, and we make no unconditional claim.

A sharper disclosure is owed for the Griffin round count itself, since the hash makes up roughly 91% of the verification’s R1CS constraints (the PLUM-128 figure of Section 4). Neither PLUM nor Loquat prints a Griffin round count for our parameters. The 14 rounds are our own instantiation, derived by the Griffin round-count methodology [9] (a margin over the Gröbner-resistance floor) as implemented in Loquat [6] and scaled for our S-box degree, at our parameters as $\lceil 1.2 \times 11 \rceil$, a 20% margin over the floor of 11 at which the monomial count reaches $2^{\kappa/2} = 2^{64}$, so an FGLM-type CICO solver costs $\approx 2^{128} = 2^\lambda$ (Griffin, eprint 2022/403, Eq. 8). Griffin’s own Table 2 tabulates 15 rounds for a width-4 instance of this field size, so our formula-derived 14 sits one round below that tabulated recommendation and a conservative instantiation would take 15. That methodology is no longer the state of the art. The FreeLunch attack [85] solves the CICO problem below the FGLM model from which the count is derived, and the Improved Resultant Attack [86] reports that most variants of Griffin fail to reach their claimed security level, with CICO solutions found in practice for 8 of 10 rounds of a Griffin instance (7 for FreeLunch; the line continues in [87], which does not by itself threaten a full-round instance). A subsequent result [88] reports full-round (10/10) CICO solutions against Griffin in the CICO-2 setting, reaching the full recommended round count on the wide, small-field parameters ($t \geq 12$, ≈ 55 -bit) of the FreeLunch and resultant benchmarks; this strengthens our refusal to claim a 128-bit floor at 14 rounds. The instances broken in practice use an $\alpha = 3$ S-box, the same degree as our instantiation, and were solved at 7–8 of their 10 rounds on reduced or smaller-field parameters. Our instance is the standard Griffin- π of [9] at width-4, $d=3$, 199-bit, 14-round parameters (Section 3); it is not among the broken ones (Table 2), separated from them, which use a wider state ($t=12$) at their own recommended round count, by its state width and round count rather than by its S-box degree or field size, the last of which enters the algebraic attack only polylogarithmically. We therefore do not claim the 14-round count establishes a 128-bit floor against the current attack frontier; the frontier now prices our instance directly. The Improved Resultant model tabulates the standard $\alpha = 3$ (the attack literature’s name for the S-box degree $d = 3$), width-4 Griffin at $2^{87} \mathbb{F}_q$ -multiplications at the design’s 15 rounds and $\approx 2^{82}$ at our 14 ([86], Table 1), so the standard width-4 Griffin clears $\lambda = 80$ by roughly two bits but does not reach $\lambda = 128$ at any round count near its design, a width-4 instance needing $r \approx 23$ to price CICO at 128 bits. This is a theoretical attack cost, not a demonstrated break: the practical solutions above are at $t = 12$, ≈ 55 -bit, with none run at $t = 4$, so what remains for future work is a practical demonstration at $t = 4$ and a re-derivation at our exact 199-bit prime. This bears directly on Assumption 6.4: a reduced-round CICO solution is precisely the non-random structure that replacing Griffin by a random oracle assumes absent, so the algebraic-attack question and Assumption 6.4 are two readings of one open problem.

Instance	d	t	field (bits)	rounds	rounds req.	CICO cost	practical CICO
Griffin, FreeLunch [85]	3	12	≈ 55	10	10	2^{55}	7/10 rounds
Griffin, Resultant [86]	3	12	≈ 55	10	10	2^{55}	8/10 rounds
This work	3	4	199	14	15	2^{82}	—

Table 2: Griffin round margin versus broken CICO instances.

The case Assumption 6.1 must rule out is silent *acceptance* of an invalid Griffin trace by the AIR. The equivalence suite exercises only valid inputs, and a separate fault-injection suite tests the rejection direction for seven of the ten constraint families (the per-row families) via `debug_constraints`: single-cell corruptions of a satisfying trace are rejected in each case (Appendix B). The three lookup-borne families (round count, memory binding, and the cross-row threading of row scheduling) lie outside `debug_constraints`, but a separate lookup-balance harness rejects them in the rejection direction (round count and cross-row threading fully, memory binding for the syscall handoff, the global-memory-consistency half excepted); parameter immutability is argued by construction. Resolving Assumption 6.1 fully, which needs the no-wrong-witness direction and not only these rejection-direction tests, by closing the global-memory-consistency half, by metamorphic testing, or by constraint-level verification of the chip against the reference permutation (tractable because the chip is a fixed, bounded AIR), is left as future work; the metamorphic and fault-injection approach is that of Arguzz [44], which targets zkVM soundness and completeness bugs by that technique.

One structural point about Assumption 6.1 governs how the measurements of Section 7 should be read. Only the Griffin-precompile arms exercise the chip at all: the precompile-free baseline, the SHA-3 control arm, and the Aurora legs never touch it, so their figures carry no dependence on Assumption 6.1. Where the chip is exercised, every run does so on the honest path: the guest supplies a valid Griffin trace and the prover pays the cost of the constraint system as built. If that system admitted a non-reference trace, the defect would not have made any measured run cheaper; for any repair that strengthens the constraint set, the measured cost is then a cost floor for the corrected chip. Assumption 6.1 is therefore a premise of the positive results alone, namely the 14.24-minute feasibility figure, the 13.0-second single-permutation smoke proof, and the unforgeability bound of Theorem 6.5. The negative findings either do not exercise the chip at all or are untouched by its constraint soundness: the pairing wrap’s classical-only anonymity is a primitive limit no chip change removes, and the SHA-3 control’s faster wall-clock could only widen against a corrected and at-least-as-costly chip.

What remains open. Several premises of Theorem 6.5 are open in a sense that no rewriting closes, and we name the specific sub-problem each leaves. Assumption 6.1 is open in the no-wrong-witness direction: a lookup-balance harness now exercises the rejection direction of the three lookup-borne families (round count and cross-row threading fully, memory binding for the syscall handoff, the global-memory-consistency half excepted), which `debug_constraints` does not reach, but that tests specific corruptions rather than establishing that no wrong witness satisfies the constraints, and the code-to-constraint transcription fidelity is not machine-checked; closing it needs constraint-level verification of the chip against the reference permutation (tractable because the chip is a fixed, bounded AIR), which is machine-checking, not writing. Assumption 6.2 is open because PLUM establishes EUF-CMA security, not simulatability; closing it means exhibiting a PLUM signature-transcript simulator, or deriving one from the masked IOP’s honest-verifier zero-knowledge, and bounding ϵ_{SZK} ,

which is new cryptanalysis. The quantum-random-oracle lift of PLUM’s EUF-CMA reduction and of the nested Fiat–Shamir compilation is open; Pegasus [31] shows the lift attainable for the power-residue-PRF family through a different construction but not for PLUM’s own STIR-based design. The Griffin round margin (Assumptions 6.4 and 6.5(i)) is now priced by the current frontier: the Improved Resultant model tabulates the standard $\alpha = 3$, $t = 4$ Griffin at $\approx 2^{82}$ at our 14 rounds (2^{87} at the design’s 15; round-margin disclosure above, Table 2), clearing $\lambda = 80$ but not $\lambda = 128$, so what remains open is a practical demonstration at $t = 4$ and a re-derivation at the exact 199-bit prime, not the security level itself. The concrete ϵ_{KS} is read from the code’s fixed 100-bit target rather than re-derived at the deployed parameters (log-blowup 2, rate 1/4, unique decoding); closing it means evaluating the Basefold unique-decoding bound at those parameters, not the univariate-FRI ethSTARK bound. The lookup-binding hypothesis carries no concrete ϵ_{lookup} for these chips at these parameters; closing it means a soundness bound for the byte-table and cross-table arguments as instantiated, not the generic LogUp analysis.

6.3 The Dual Obstruction, Reduced to One Remaining Obstruction

The preservation theorem leaves exactly one condition of Definition 6.3 unmet on the studied stack. We state it as the chapter’s negative result and justify each obstruction below.

What this chapter delivers is two results: the conditional preservation theorem of Section 6.2 (Theorem 6.5), which bounds how far the move to a zkVM substrate and the precompile suite can shift the adversary’s advantage in each of BDEC’s security experiments, and establishes that knowledge soundness, unforgeability, and the showing-anonymity and unlinkability bounds transfer to the substrate under the named assumptions (AIR constraint soundness, transcript simulatability, lookup binding, and black-box transfer), with issuance anonymity requiring key-privacy (Assumption 6.3) in addition; and a *dual obstruction* to deployable post-quantum anonymity on the studied stack, the *anonymity–deployability tension* under which later chapters cite it. The cost criterion that decides *when* a precompile pays, and the with/without-precompile inversion it postdicts, are developed and measured separately (Sections 4 and 7); this chapter takes the feasibility they establish as given and asks which security properties survive the move. The dual obstruction (Proposition 6.7) is likewise real and has two independent sides, now that the feasibility question is resolved: the feasible, post-quantum-sound bare-STARK receipt is not zero-knowledge, and the zero-knowledge wrap of it the toolchains expose is pairing-based, hence not post-quantum. Both proofs run within the 24 GB budget, so neither side is a memory or provisioning wall; what no proof the stack produces achieves is zero-knowledge and post-quantum soundness at once. The first obstruction is a property of the bare receipt. The second is a primitive limit no memory or time budget removes.

We do *not* claim a delivered *post-quantum* anonymity or unlinkability guarantee on the evaluated stack. The guarantees are conditional and sound; on the bare receipt their bound is non-informative because ϵ_{ZK} is not made small, the receipt having no established zero-knowledge simulator, and the pairing wrap that completes as the toolchain ships (below) drives ϵ_{ZK} small only against a classical distinguisher (the second obstruction). Aligning with Proposition 6.7, the construction’s anonymity is therefore “sound, and made effective against a classical distinguisher by the completing pairing wrap, but not post-quantum: no proof the stack produces is at once zero-knowledge and post-quantum-sound.” Two further honesty caveats bound the claim. First, both credential relations now commit their public statement to the journal (Section 7), so each accepting receipt binds to a specific statement rather

than attesting merely that its verifications hold “under some witness”; statement-binding and the zero-knowledge wrap combine in a single measured run for both credential relations, credential generation and the $k = 2$ showing (Section 7), within run-to-run variance of the plain wrap. This statement-binding requirement is disclosed in Section 5, discharged in Lemma 6.4, and made an explicit win-condition conjunct (JBind) in the experiments above. Second, “the user’s long-term key and signatures enter only via the read channel and are never committed” is a statement about the witness channel, not an unlinkability result; it is the mechanism that *could* deliver anonymity under a zero-knowledge wrap, and nothing more absent that wrap. The anonymity guarantee is stated as what it is, a conditional that the measured substrate does not discharge.

Proof stack (this work-load, $\lambda = 80, 24 \text{ GB target}$)	Consumer- HW feasible	Knowledge- soundness	ZK / anonymity	Post-quantum soundness
Bare succinct STARK (measured)	✓	✓	×	✓
Groth16-wrapped (supported, not measured)	(✓)	✓	✓	×
PLONK-wrapped (BN254, measured)	✓	✓	✓	×
PQ-sound recursive ZK wrap (hypothetical)	<i>open</i>	✓	✓	✓

Table 3: Property-survival matrix across proof configurations. The parenthesised entry is expected from the measured PLONK wrap, not itself measured.

Proposition 6.7 (The dual obstruction, as a falsifiable prediction). *On the studied stack (SP1 on the 24 GB target, $\lambda = 80$), no proof of the credential relation that the stack can produce satisfies all three conditions of Definition 6.3 at once. The stack produces two proofs, both within the memory budget, and each falls on one of two independent sides.*

First obstruction (the feasible post-quantum proof is not zero-knowledge). *The bare succinct STARK receipt is generated within budget, and its soundness rests on hash-based assumptions alone, no quantum-broken primitive entering (the quantum-random-oracle status of its extractor being the separate open lift recorded in Section 6.2); but it is not zero-knowledge: SP1’s specification states that its STARK proofs are not zero-knowledge [89], no simulator being established, so the error ϵ_{ZK} of Equation (6) is not made small and the anonymity bound is sound but not effective on this receipt.*

Second obstruction (the feasible zero-knowledge proof is not post-quantum). *Wrapping the receipt for zero-knowledge completes within the memory budget (the measured PLONK/BN254 wrap proves in 46.83 min for PLUM-verify at peak $\approx 15.3 \text{ GiB}$ and 4.20 h for credential generation at peak $\approx 15.6 \text{ GiB}$; Section 7), but the only wraps the toolchains expose are pairing-based, hence classically secure only, violating condition (iii) of Definition 6.3.*

The feasibility question is resolved: both proofs run within 24 GB, and what no proof the stack produces achieves is zero-knowledge and post-quantum soundness at once. Read across Table 3, the claim is that no stack we could run fills all four properties at once: the intersection of consumer-hardware feasibility, knowledge-soundness, zero-knowledge, and post-quantum soundness is empty on the toolchains available to us. This is an empirical, toolchain-contingent finding, not an impossibility in the standard model, the setting that assumes no idealised oracle. It is falsified by exhibiting a single post-quantum-sound zero-knowledge recursive wrapping of the STARK receipt that proves within the consumer resource budget on this workload, that is, by populating the hypothetical bottom row of Table 3. The bottom-row cell that would carry all four properties is empty on any stack we could run, so its feasibility is open, not excluded. We do not claim the second obstruction is impossible in the standard model: two hash-based routes already exist as code (masked FRI and the VEIL compiler, Section 9), and whether either proves within the memory budget is the open resource question this thesis leaves for future work. The first obstruction is resolved for a deployer who accepts classical security, since the pairing wrap runs. The second obstruction is a present toolchain gap, the absence of a post-quantum-sound zero-knowledge wrap, not a proven barrier.

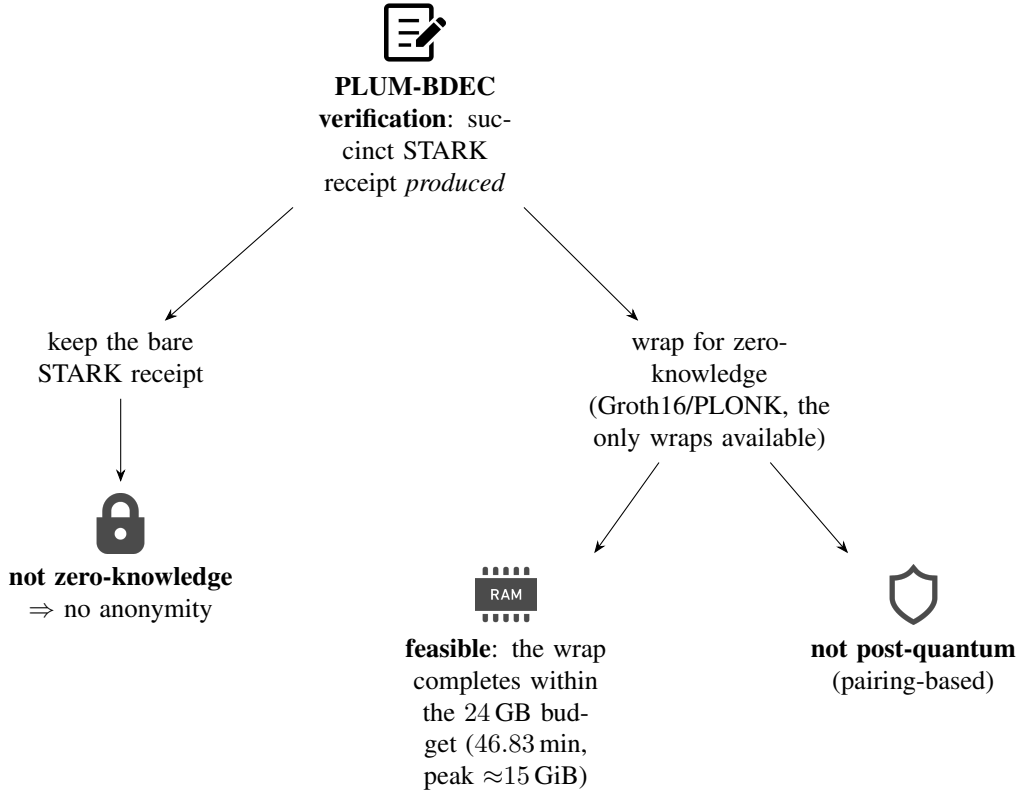


Figure 2: The dual obstruction to post-quantum anonymity.

BDEC’s anonymity and unlinkability require the underlying argument to be zero-knowledge. The succinct STARK receipt produced by the zkVM is, in the configurations we run, an argument of knowledge but not a zero-knowledge one. SP1’s specification states plainly that its STARK proofs are not zero-knowledge [89]. We therefore treat the bare receipt as not dependably zero-knowledge, a concrete instance of the caution that a system may be called a zkVM without delivering zero-knowledge.

Zero-knowledge is obtained only by wrapping the receipt in a succinct proof, and the wraps SP1 exposes, Groth16 and PLONK, are pairing-based and therefore only classically secure. The obstruction splits into two independent sides, set out below and summarised in Figure 2. Both block *post-quantum* anonymity for separate reasons, and naming each one precisely is the security-level result of this analysis.

The first obstruction is that the feasible, post-quantum-sound proof is the bare receipt, which is not zero-knowledge; zero-knowledge is recovered only by wrapping. That wrap completes within the consumer memory budget. The deployable PLONK/BN254 wrap of the PLUM verification proves end-to-end (core, compress, shrink, wrap, and native gnark PLONK, then verify) in 46.83 min at peak 15.3 GiB, and the credential-generation wrap in 4.20 h at peak 15.6 GiB, both with roughly 8 GiB of headroom under 24 GB. What clears the recursion-shape provisioning wall (the static verifying-key tables of Section 7) is building a reduced verifying-key map from the guest’s own shard shapes, a one-time amortised setup of some 19–26 verifying-key setups completing in tens of seconds, together with the stock v6.1.0 gnark circuit on the sound path (verifying-key checking on), so neither the ≈ 51 h regeneration nor a gnark rebuild is needed. The credential-generation wrap’s longer wall is a compress tail that scales with the credential’s shard count (1018 shards against PLUM-verify’s 104), not a memory bound; peak memory stays bounded near 15 GiB throughout. This completing wrap runs gnark PLONK with the zero-knowledge flag enabled, so it is zero-knowledge by gnark’s analysis of that configuration. A dedicated verification run confirms the configuration is active, with two proofs of one witness differing under randomized blinding, and the post-quantum obstruction below holds regardless. The classical anonymity this wrap supplies rests further on the gnark PLONK stage’s own knowledge-soundness and on its universal SRS, which the measured runs generate locally rather than through a trusted-setup ceremony. We carry these as benchmark-level properties of the wrapped artifact, not as established guarantees of a deployment. The second obstruction is one of primitives and is the operative one. Wrapping a post-quantum STARK receipt in a pairing-based PLONK or Groth16 proof re-introduces a quantum-vulnerable assumption into the credential’s post-quantum soundness, the very class of assumption the post-quantum motivation rules out (Section 2). Such a wrap remains acceptable for a classically-secure deployment, and PLONK and Groth16 are not unsuitable in general. They are ruled out here only by the thesis’s post-quantum-anonymity goal. Recovering post-quantum anonymity requires a post-quantum-sound zero-knowledge wrap, for instance a STARK-based or lattice-based recursive argument, which the toolchains we run do not yet provide.

The two obstructions differ in kind. The first is not a resource cost at all. The bare receipt is feasible and the pairing wrap that supplies zero-knowledge completes within 24 GB (Section 7), so a deployer who accepts classical security is already served. The second is a primitive limit within the post-quantum goal, since a pairing-based wrap is not post-quantum at any memory or time budget. The first is cleared on this stack; the second bounds what this stack can deliver.

The classically secure PLONK/BN254 wrap therefore gives a deployable *classically*-anonymous configuration, but not a deployable post-quantum-anonymous one. The two obstructions together leave no post-quantum option on the zkVM stack on commodity hardware: the proof that is post-quantum-sound is not anonymous, and the proof that is anonymous is, with the available wraps, only classically secure, whereas a zero-knowledge-enabled Aurora run on the same machine proves the Loquat single-verification building block in about 22 minutes (mean of three runs, 21.4–22.9; Section 7). Within the scope of the literature we surveyed (Section 2.5), we did not find this conjunction stated explicitly for post-quantum anonymous credentials, and surfacing it is a contribution of this thesis: we measure that

the zero-knowledge wrap runs within budget and identify the remaining obstruction (the second), the absence of a post-quantum-sound wrap, as a toolchain gap.

The post-quantum gap is scoped to the substrate and hardware studied: an Apple M5 Pro with 24 GB of memory, the CPU-bound prover of SP1, and the parameter set $\lambda = 80$. We do not claim it is fundamental to the primitive or that it persists on server-class hardware. Characterising how it moves with the memory budget and the security level is left as future work.

What would move this gap can be named as three levers. Reaching post-quantum zero-knowledge for this workload requires (i) a post-quantum-sound zero-knowledge wrap in place of the pairing-based ones the toolchains currently expose, for which two hash-based routes already exist as code (masked FRI and the VEIL compiler; Section 9); (ii) that this post-quantum-sound wrap prove within the memory budget, the resource question this thesis leaves open now that the classical pairing wrap is measured to fit; and, for everlasting in place of merely computational anonymity, (iii) a non-interactive, standard-model, statistically-hiding commitment to the witness-bearing trace (Proposition 6.12). Levers (i) and (ii) are engineering questions, the techniques for (i) already implemented and (ii) reducing to whether the masked prover fits the budget; lever (iii) is an open construction problem.

A separate question is the strength of the anonymity guarantee where the argument is zero-knowledge. Two points bound it. The first concerns quantum adversaries. The bound of Equation (6) is stated in the classical random-oracle model, so it constrains a classical distinguisher. Lifting it to the quantum random-oracle model is attainable in principle: the compilation of a round-by-round sound interactive oracle proof into a non-interactive argument is post-quantum secure in the quantum random-oracle model [90], and this is argued for the STIR-class proof systems PLUM uses in recent work [91]. The lift is not settled here. It remains conditional on a PLUM transcript simulator (Assumption 6.2) and on a post-quantum analysis of PLUM’s own Fiat–Shamir step, which the scheme’s published analysis does not provide. We therefore claim computational anonymity against a classical distinguisher and record the quantum lift as attainable but open.

The missing lemma for the quantum lift. The quantum-random-oracle lift of PLUM’s Fiat–Shamir compilation, recorded above as attainable but open, has a single identifiable missing input: round-by-round, equivalently state-restoration, soundness of PLUM’s own multi-round STIR interactive oracle proof at the deployed parameters. The general compilers assume exactly this hypothesis. Chiesa, Manohar and Spooner [90] compile any round-by-round sound IOP into a non-interactive argument secure in the quantum random-oracle model, and the post-quantum BCS-compilation analysis of [91] delivers adaptive post-quantum soundness from classical round-by-round soundness for the hash-based interactive-oracle proof systems PLUM uses. Neither result asserts that PLUM’s particular IOP meets the round-by-round hypothesis at its parameters, and that assertion is the missing lemma.

Conjecture 2 (Round-by-round soundness of PLUM’s STIR IOP at the deployed parameters). *PLUM’s multi-round STIR-based interactive oracle proof is round-by-round sound with error ϵ_{RBR} at the deployed parameters ($\eta = 4$, $R = 1$, $\kappa_0 \in \{16, 26\}$, $|U| = 2^{12}$, $d^* = 128$ the degree bound of the committed polynomial oracle, $\log_2 p \approx 199$), in the list-decoding (up-to-capacity) proximity regime. The dominant query-repetition term is fixed by the code rate $\rho = d^*/|U| = 128/4096 = 2^{-5}$: in the capacity regime each of the κ_0 first-round queries fails with probability about ρ , so the query soundness is $\rho^{\kappa_0} = 2^{-5\kappa_0}$, giving 2^{-80} at $\kappa_0 = 16$ ($\lambda = 80$) and 2^{-130} at $\kappa_0 = 26$ ($\lambda = 128$). The full ϵ_{RBR} additionally carries STIR’s proximity-gap and single folding-round ($R = 1$) terms [30], which we do*

not reproduce here. Under this conjecture the Fiat–Shamir compilation is quantum-random-oracle secure by [90, 91], lifting EUF-CMA to a quantum forger.

The conjecture is not established for PLUM’s specific IOP, and if round-by-round soundness fails at these parameters only the quantum unforgeability lift is lost, while classical EUF-CMA and all anonymity claims are unaffected. The deployed κ_0 values are worth reading against the regime: at the proven unique-decoding radius each query fails to catch a cheating word with probability about $(1 + \rho)/2 \approx 0.516$, so $\kappa_0 = 16$ would yield about 2^{-15} , far short of $\lambda = 80$. The parameters therefore presume the conjectured, up-to-capacity proximity regime, and the $2^{-5\kappa_0}$ figure holds only in that regime; this reliance is specific to PLUM’s own STIR IOP, unlike the outer SP1 prover’s ϵ_{KS} , whose measured configuration was read above as the proven unique-decoding regime. The up-to-capacity proximity-gaps conjectures this regime rests on were, moreover, recently disproven in their strongest form [92], with an unconditional soundness bound above the Johnson bound established only subsequently and at parameters other than ours [93]; the conjectured regime these parameters presume therefore stands on weaker ground than when they were set, reinforcing that the $2^{-5\kappa_0}$ figure is conjectural rather than proven.

Remark 6.8 (The anonymity guarantee is computational). The guarantee is computational, and it is not everlasting: a recorded transcript hides the witness only while the underlying commitment stays hiding. The showing relation itself contains no commitment. The witness enters the zkVM’s execution trace, and the prover commits that trace under an algebraic-hash Merkle root, so the receipt binds w through salted evaluations of the schematic form $c = H(w \parallel r)$, applied by the proof system rather than by the proved relation. A salted hash leaf hides its input statistically only in the random-oracle model. Once H is a fixed function the hiding rests on a computational assumption on that function, and the random-oracle methodology gives no assurance that the hiding survives the instantiation [94]. An adversary who stores a transcript and later acquires quantum computing power attacks the instantiated function, against which the hiding is at best computational. Recovering everlasting anonymity would require the trace commitment to be statistically hiding in the standard model and to be non-interactive, since the receipt is a single message. A non-interactive statistically-hiding commitment is not available from general assumptions: any black-box construction from one-way permutations needs $\Omega(n/\log n)$ rounds of interaction [95, 96]. Statistical hiding and statistical binding also cannot hold together, so binding must remain computational. The everlasting variant is therefore not currently establishable (the obstruction is formalized as Theorem 6.10 below).

Definition 6.9 (The hash-committed showing class $\mathcal{C}$). A credential-showing system belongs to the class $\mathcal{C}$ if its receipt satisfies the following four structural conditions.

(C1) Witness carried by a trace hash commitment. The only component of the receipt that depends on the private witness w and survives the proof’s zero-knowledge masking is an algebraic-hash trace root R , computed by salted evaluations $\text{com} = H(w \parallel r)$ of a fixed public compressing function H over trace data containing w , with prover randomness r , applied by the proof system rather than by the proved relation. No separate statistically-hiding commitment is applied to w before it enters H . The masked opened values are statistically independent of w by the honest-verifier statistical zero-knowledge of the underlying interactive oracle proof [47], so R is the sole carrier of w .

- (C2) **Binding for soundness.** The receipt is knowledge-sound, so H is instantiated as a collision-resistant compression and R binds the committed trace; an accepting receipt therefore certifies a specific authorised witness w rather than a lossy equivalence class of witnesses.
- (C3) **Harvest-now adversary.** Everlasting anonymity is anonymity against an adversary who records the single-message receipt and is thereafter computationally unbounded.
- (C4) **Random-oracle instantiation.** H is modelled at best as a random oracle; in deployment it is a fixed public function whose description and outputs the adversary of (C3) holds, and no standard-model statistical-hiding property (a leftover-hash-lemma regularity, say) is proven for it.

A *masked-STARK* showing, the strongest variant the hash route offers [47], lies exactly in $\mathcal{C}$: its masked opened values are statistically independent of w by honest-verifier statistical zero-knowledge, its witness is bound through the salted trace evaluations $c = H(w \parallel r)$ of Remark 6.8 under the prover’s algebraic-hash Merkle root over its native field, and no statistically-hiding commitment is interposed. The two variants the toolchain actually produces sit below this best case and fail earlier: the bare STARK receipt leaks w through unmasked opened values, and the pairing-based PLONK wrap hides the inner openings only computationally, so neither attains even the statistical masking (C1) presumes. The theorem below therefore bounds the hash route at its strongest point.

Theorem 6.10 (Non-establishability of everlasting anonymity for $\mathcal{C}$).

Everlasting anonymity cannot be established in the standard model for a system in the class $\mathcal{C}$ of Definition 6.9 by the routes the class affords: neither H ’s idealised hiding nor an interposed commitment yields a standard-model statistical-hiding guarantee for the receipt, so the witness-hiding provable for $\mathcal{C}$ is at most *computational*. The proof invokes no computational hardness assumption and grants the adversary unbounded power; it claims no universal impossibility beyond the class structure (Remark 6.11).

Proof. Everlasting anonymity requires that the recorded receipt be statistically independent of w , so that the unbounded distinguisher of (C3) gains no advantage. By (C1) the receipt’s only w -dependent surviving component is the trace root R , a commitment computed by the fixed function H through $\text{com} = H(w \parallel r)$; establishing everlasting anonymity for a system in $\mathcal{C}$ therefore requires exhibiting a standard-model statistical-hiding guarantee for R . Such a guarantee must be a property either of H itself or of a structure interposed before H ; these are the only two routes, and both are closed. The first is closed by (C4): H ’s statistical hiding is an artefact of the idealisation, and no standard-model hiding property is proven for the deployed function (a fixed H with, say, a proven leftover-hash-lemma regularity would fall outside (C4)). The second is closed by (C1) and the one-message form of the receipt, as the next two paragraphs detail.

The statistical hiding that H does enjoy is an artefact of the random oracle. A salted leaf $H(w \parallel r)$ hides its input statistically when H is an ideal random oracle, with distinguishing advantage at most $q/(2^\lambda - q)$ for q queries; this is the elementary random-oracle collision-and-hit bound, holding only in the idealised model. By (C4) the deployed H is a fixed public function, and the random-oracle methodology gives no assurance that a property proved relative to the oracle survives the instantiation [94]. The harvest-now adversary of (C3) does not query an oracle; it holds the recorded digest

R and the public function H , and attacks the instantiation, against which the random-oracle hiding argument transfers nothing.

The remaining route to statistical hiding, a non-interactive standard-model statistically-hiding commitment applied to w before it reaches H , is excluded from $\mathcal{C}$ by (C1). It is moreover not available from general assumptions for a single-message receipt: any black-box construction of a statistically-hiding commitment from one-way permutations requires $\Omega(n/\log n)$ rounds of interaction [95, 96], whereas the receipt is one message, so no commitment obtained from these general-assumption constructions can be interposed.

No route open to $\mathcal{C}$ yields a standard-model statistical-hiding guarantee for R : the random-oracle hiding of (C4) does not transfer to the instantiation by the argument of [94], and by (C1) the receipt interposes no separate statistically-hiding commitment, and none is available to a one-message receipt from general assumptions. Consequently $\mathcal{C}$ affords no standard-model argument that its receipt is statistically independent of w , so any provable witness-hiding for a system in $\mathcal{C}$ is at most computational, and reading Equation (6) in the everlasting setting its ϵ_{ZK} term is not statistical and is not established small against the unbounded distinguisher of (C3). No step assumes any function is hard to invert; the obstruction is that the only hiding $\mathcal{C}$ provably affords in the standard model is computational, and against the unbounded adversary of (C3) computational hiding carries no established guarantee. Everlasting anonymity therefore cannot be established in the standard model for any system in $\mathcal{C}$; whether some system in $\mathcal{C}$ nonetheless happens to be everlastingly anonymous is left open. $\square$

Remark 6.11 (The computational boundary). Theorem 6.10 does *not* claim that computational anonymity fails. Under the zero-knowledge assumption (Definition A.4) the commitment $\text{com} = H(w \| r)$ is computationally hiding, so the showing is anonymous against a bounded classical distinguisher, at advantage $\text{Adv}^{\text{anon}} \leq (k + 2) \epsilon_{\text{sZK}} + \epsilon_{\text{ZK}}$ of Equation (6), once a zero-knowledge wrap makes ϵ_{ZK} small. The non-establishability is confined to the everlasting, statistical strengthening for the class $\mathcal{C}$. The harvest-now, de-anonymise-later harm therefore attaches to transcripts *in* $\mathcal{C}$ because their hiding is computational; it is not a claim that every recorded zero-knowledge proof must be post-quantum, only that a receipt whose witness-hiding is a bound hash commitment to the execution trace inherits a computational, not everlasting, guarantee.

The theorem is a non-establishability result *for the class*, not a universal impossibility, and the boundary of $\mathcal{C}$ is the escape route. Interposing a non-interactive standard-model statistically-hiding commitment to w before H leaves $\mathcal{C}$ and reaches everlasting anonymity, which is exactly the hypothesis of Proposition 6.12. This route is realisable as shown by the everlasting anonymous rate-limited tokens of Chairattana-Apirom, Döttling, Lysyanskaya and Tessaro [54], which achieve everlasting anonymity through a statistically-hiding pairing-based commitment, at the cost of classical, non-post-quantum soundness and only within a rate limit. Their construction is consistent with the theorem rather than a counterexample to it: everlasting anonymity is attained precisely by leaving the hash-only class $\mathcal{C}$, and on this evidence the achievable route is pairing-based rather than post-quantum. This is the same seam the dual obstruction of Proposition 6.7 records, read now at the information-theoretic level.

Proposition 6.12 (Conditional everlasting anonymity). *Suppose there is a non-interactive, post-quantum, statistically-hiding commitment to the witness columns of the trace that composes with the low-degree test, so that the witness is committed before it enters the algebraic-hash root and the proximity test does not re-expose it. Then, in a masked variant of the proof and under the hypotheses of Theorem 6.5, with Assumption 6.2 taken in its statistical form (the transcript-simulation error ϵ_{sZK} a statistical distance; Definition A.3 as stated quantifies only over efficient distinguishers), the zkVM-executed BDEC*

system is everlastingly anonymous and unlinkable: against an unbounded distinguisher the advantage is at most $\epsilon_{\text{ZK}}^{\text{stat}} + (k + 2) \epsilon_{\text{sZK}}$ for anonymity and $\epsilon_{\text{ZK}}^{\text{stat}} + \epsilon_{\text{sZK}}$ for unlinkability, where $\epsilon_{\text{ZK}}^{\text{stat}}$ is statistical. Binding, and hence unforgeability, remains computational.

Proof sketch. The anonymity argument of the preservation theorem is unchanged except at the step that hides the witness in the trace commitment. In the construction analysed that step is computational, because the witness columns sit under a collision-resistant Merkle root (Remark 6.8). Under the hypothesised commitment the witness columns are statistically hidden before they reach the root, so the root carries no statistical information about the witness, and the low-degree test operates on the committed columns without re-exposing it. The masked interactive oracle proof is statistically honest-verifier zero-knowledge by randomisation, independently of the hash [47], so the transition incurs only the statistical distance $\epsilon_{\text{ZK}}^{\text{stat}}$, and the transcript-simulation step is unchanged, its ϵ_{sZK} now the statistical distance of the strengthened assumption, at cost $(k + 2) \epsilon_{\text{sZK}}$ for anonymity and ϵ_{sZK} for unlinkability. An unbounded distinguisher therefore gains at most $\epsilon_{\text{ZK}}^{\text{stat}} + (k + 2) \epsilon_{\text{sZK}}$ for anonymity, and $\epsilon_{\text{ZK}}^{\text{stat}} + \epsilon_{\text{sZK}}$ for unlinkability. Binding stays computational, since a commitment cannot be both statistically hiding and statistically binding, so the unforgeability bound is unaffected. $\square$

A concrete candidate for the hypothesised commitment is the Module-SIS-based commitment of Baum, Damgård, Lyubashevsky, Oechsner and Peikert [97], which is statistically hiding in a parameter regime that trades off against binding. It is post-quantum, but its proof of opening is interactive, so it does not by itself meet the non-interactive requirement above. Closing that gap is the open problem behind Proposition 6.12.

This closes the chapter on the terms its opening set. Of the three conditions of Definition 6.3: knowledge soundness transfers conditionally (Theorem 6.5, modulo Assumptions 6.1 and 6.6 and Conjecture 1); zero-knowledge is delivered within the memory budget by the pairing wrap, against a classical distinguisher; and post-quantum anonymity remains blocked by the second obstruction alone, the toolchain gap to which this section has reduced the dual obstruction.

7 Evaluation

This section reports what the precompile-enabled zkVM regime can and cannot do for PLUM-based BDEC on consumer hardware, and reads the measurements back against the cost framework of Section 4. Four findings answer the thesis’s question, one per contribution. *Feasibility*: the Griffin precompile turns PLUM verification from a non-terminating run into a 14.24-minute proof within the 24 GB envelope (Table 5). *The cost criterion*: that precompiled hash nonetheless proves slower than a software SHA-3 control (14.24 versus 13.24 min), an inversion whose sign the trace-area model postdicts. The measured evidence gives three results (the design-level field-multiplication verdict is Section 5’s). Griffin removes $58.5\times$ cycles yet loses on committed area. The matched-field control confirms the necessity direction ($164\times$ tax, execute mode). The power-residue symbol pays $10.33\times$ on total committed area (§7.3, §7.5.3). *The credential characterisation*: both credential relations run and prove on SP1 (Table 7). *The dual obstruction*: the zero-knowledge wrap completes in 46.83 minutes but is pairing-based and so classical, not post-quantum (§7.4.1). Within the scope of the literature we surveyed these are among the first such figures; the configurations that do not complete are reported alongside those that do, and everything that follows is the evidence for these four.

7.1 Setup and Methodology

We measure PLUM verification and the credential relations built on it with and without the precompile suite, attribute where the zkVM cost falls and how much the suite removes, locate the resource boundary for the configurations that do not complete, and compare the zkVM and static-circuit regimes on update churn (Definition 4.7).

Two measurement modes. *Execute mode* counts RISC-V cycles without producing a proof; its cost is machine-independent and always terminates. *Prove mode* produces the receipt, is the deployment-relevant cost, and does not always complete on the target hardware. We therefore report prove wall-clock where proving completes and fall back to execute-mode cycles where it does not, never comparing the two across that boundary; each table states its mode. *Feasible* means the prove completes within the 24 GB envelope; whether a completed time is *acceptable* is a separate judgment this chapter does not make: the deployment trade-offs it turns on are weighed in Section 8, and the operational bar the thesis does and does not state is fixed in Section 9.

What we do not claim. We report no single “ $N\times$ speedup.” The with/without-precompile contrast at the prove level is an *infeasible-to-feasible* transition, not a ratio of two finite times (§7.2); the finite reductions we report are cycle-level, at a stated security level and mode.

Platform. Table 4 lists the evaluation platform; all prove-mode measurements are local single-machine runs with no network or dev-mode acceleration unless stated.

Component	Specification
Hardware	Apple M5 Pro, 18-core CPU, 20-core GPU, 24 GB unified memory, 1 TB SSD
OS	macOS 26.5 (Darwin 25.5.0), Apple Silicon (arm64)
Prover	CPU-bound; no CUDA/network prover (Apple Silicon has no NVIDIA GPU path)
zkVM	SP1 v6.2.1 (forked)
Signature	PLUM over $\mathbb{F}_p$, $p = 2^{64}p_0 + 1$ (199-bit; see §5.2)
Security levels	$\lambda = 80$ (prove-mode runs; Griffin-delta and credential execute figures), $\lambda = 128$ (cost-attribution analysis,

Table 4: Evaluation platform and measurement setup.

7.1.1 Settings Justification

Each fixed parameter is one a deployer would plausibly choose; we disclose where a setting is tuned.

Consumer hardware (M5 Pro, 24 GB). The research question concerns deployability on a personal machine, so this hardware is the thesis’s explicit scope. Every feasibility claim is scoped to this envelope and is not asserted in general.

Security level: $\lambda = 80$ for prove-mode, $\lambda = 128$ for cycle attribution. We disclose this seam. Prove-mode runs use $\lambda = 80$ because $\lambda = 128$ proving is untested and expected to exceed the memory budget, the precompile-free arm already failing at $\lambda = 80$ (Table 5). $\lambda = 80$ is the largest level at which prove-mode wall-clock is observable on this hardware, which is why we use it and not because it is favourable. The cost-attribution analysis (§7.3) is reported at the audience-relevant $\lambda = 128$ because it is machine-independent (execute mode) and so remains measurable where proving is not. No number mixes the two levels silently. Each table states λ .

Prover memory tuning. At SP1’s default settings the Cell 2 prove is OOM-killed after ≈ 3 min on the 24 GB machine. A documented reduced-memory configuration (`SHARD_SIZE` = 2^{22} , default 2^{24} ; `ELEMENT_THRESHOLD` = 2^{26} ; `HEIGHT_THRESHOLD` = 2^{20} ; `TRACE_CHUNK_SLOTS` = 2; `RAYON_NUM_THREADS` = 8, default 18; §7.1.2) trades smaller, more numerous shards and fewer threads for fitting the envelope, so the headline timings are tuned-configuration times (Cell 2 14.24 min, SHA-3 control 13.24 min under identical tuning; the zero-knowledge wrap ≈ 15 GiB peak, §7.4.1). One contributing factor to the Griffin arm’s memory pressure: the fork’s shard-budgeting table prices the Griffin chip at 348 trace columns against its compiled 3,819, so a Griffin-bearing shard is budgeted at roughly a tenth of its committed area and split later than the trace-area threshold intends; correcting the entry is a one-line change whose effect on Cell 2 is untested and left to future work.

Micro-benchmark isolation. Cost-attribution runs isolate single primitives (one Griffin permutation; one verification) so that per-operation cost can be attributed rather than inferred from an aggregate.

7.1.2 Reproducibility

Prove-mode and attribution runs use fixed RNG seeds and fixed messages. The SP1 submodule version is pinned in the lockfile. The memory-tuning environment variables, host binaries, and the adversarial-probe harness are recorded in the project’s measurement notes, together with the per-run commit hashes

and, where applicable, guest-image identifiers. The exception is an earlier, retired pair of CreGen prove runs (the 26.98/27.45 min pair; §7.4): their shard configuration was recorded only in a human-written summary rather than machine-echoed in the run log, and does not reproduce on the shipped fork. The credential-relation prove figures of Table 7 are the config-verified runs of §7.4.

7.2 PLUM Verification With and Without the Precompile

Table 5 reports prove-mode results for a single PLUM signature verification at $\lambda = 80$. The central comparison is between the precompile-free baseline and the precompile-enabled arm on the same hardware. Its rows label Cell 1/2/3 as no precompile / Griffin precompile / SHA-3 control, and the control is not a deployable variant.

zkVM / mechanism	λ	Prove time	Peak mem	Outcome
Cell 1: SP1, rv64im Griffin (no precompile)	80	n/a	≈ 13.6 GiB	DNF (stopped at 5 h 07 m)
Cell 2: SP1, Griffin-$\mathbb{F}_p^{192}$ precompile (this work)	80	14.24 min[†]	≈ 13.7 GiB	receipt verified
Cell 3: SP1, SHA-3 hasher (control)	80	13.24 min [†]	≈ 14.4 GiB	receipt verified

Table 5: PLUM verification, prove mode ($\lambda = 80$; the Griffin instance runs at 14 rounds, one below the security-conservative 15 of Table 2).

A run of the precompile-free arm under the documented reduced-memory tuning (§7.1.1) ran for 5 h 07 m without terminating and without exhausting memory before we stopped it by hand, yielding no receipt. Under that tuning the binding resource is time rather than memory, and the prove still does not complete within five hours on this hardware.

The honest reading. The with/without-precompile contrast is not a ratio of two finite times. Without the precompile the workload *does not complete* on this hardware, and with it the workload completes in 14.24 minutes (mean of five runs, 14.16–14.44). The defensible claim is therefore qualitative at the prove level (*the precompile moves PLUM verification from infeasible to feasible on consumer hardware*) and quantitative only at the cycle level (§7.3).

The SHA-3 control does not isolate the hash family (it differs from the Griffin arm in two things at once, the hash (bit-oriented SHA-3 against $\mathbb{F}_p^{192}$ Griffin) and the field, matched only on the security level λ , so it is a deliberately confounded control rather than a hand-picked favourable setting). The verification harness is identical except that every Griffin- $\mathbb{F}_p^{192}$ sponge call is replaced by a SHA3-256 call computed in software, so that the SHA-3 arm fires *no* hash precompile and only the shared `UINT256_MUL` field-multiplication precompile remains active. It shows that a workload using a *standard* hash in software is cheaper, by the margin the next paragraph quantifies, than one using the algebraic hash PLUM requires through a bespoke chip, which is the quantitative form of the field-mismatch-tax argument of Section 4.

The inversion is direct. The bespoke Griffin- $\mathbb{F}_p^{192}$ precompile proves *slower* at prove than the software SHA-3 control at matched $\lambda = 80$ in every run we took. The sign is robust. At a fixed tuned config on a clean machine, five Cell 2 runs and five Cell 3 runs give means of 14.24 min (range 14.16–14.44) and 13.24 min (range 13.02–13.38), a magnitude of $\approx 1.08\times$ in wall-clock and $\approx 1.11\times$ in cycles, with each arm’s prove-time spread under 5%. An earlier single run measured 32.53 versus

13.28 min; its high Cell 2 figure was not reproduced under fixed-config replication and is attributed to machine-state contamination rather than an intrinsic build difference, so the reported value is the replicated 14.24 min.

Why the model postdicts the sign. We read the inversion as a genuine property of the pipeline rather than a measurement artefact, and the trace-area accounting of Section 4 (Equation (1)) postdicts its sign from the chips’ committed dimensions. The Griffin chip emits 14 rows of 3,819 columns per permutation (each row carries 26 modular-arithmetic gadgets of 127 cells each, beside range checks and state limbs), so the 1,052 permutations of one verification commit $14 \times 3,819 \times 1,052 \approx 5.6 \times 10^7$ cells, plus one controller row per permutation, that the SHA-3 arm never pays. The shared `UINT256_MUL` chip is common-mode (69,396 versus 69,385 events) and cancels, and the Griffin arm additionally executes 11% more guest cycles (1.23×10^8 versus 1.11×10^8 , execute mode). Both terms of Equation (1) are therefore larger on the Griffin arm, so the model places it behind the control for every non-negative choice of the per-cell rates, granted a common per-cycle core area across the two arms. The calibration of the model’s constants on these two arms is reported with the cost model (§7.5). The feasibility recovery and the inversion are the same cost model (Equation (1)) evaluated at two operating points. With the precompile the run completes and can be measured, and the cost model explains why it is still no faster than the control. The control’s hash costs the substrate no chip area at all, while the Griffin arm pays for its chip and saves nothing in return.

Not all of that width is irreducible. Of the twenty-six field operations per row, about ten are genuine 192-bit multiplications, the S-box arithmetic that no small field avoids. The other sixteen are additions and small linear combinations. They are charged at the full multiplication price only because the chip reuses one circuit for both, so a dedicated addition path would cut about a quarter of each row. Even so trimmed, every remaining cell still computes over the 199-bit field, so a leaner chip narrows the gap toward parity but does not reverse its sign against a hash whose cost is field-invariant.

The inversion is moreover conservative twice over. The SHA-3 control runs in software with no chip of its own, so a native bit-hash precompile would only widen the gap, and the Griffin instance is arithmetised at 14 rounds against the 15 its parameters call for (Section 6, Table 2), so a correctly-secure instance adds roughly another fourteenth of the Griffin cells and widens the gap further.

Proof size, and the wrap that produces it. Receipt size in SP1 is a property of the proof *type*, not of the workload. The runs above use SP1’s default *core* proof, whose size *scales with execution size* [50]. The compact, on-chain-deployable proof is instead the constant-size SNARK *wrap* of that receipt, about 260 bytes for Groth16 over BN254, or 868 bytes for PLONK [50]. Producing that wrap completes on the target hardware: the full sound PLONK/BN254 chain wraps one PLUM verification into a verified constant-size proof in 46.83 minutes at a 15.29 GiB peak, inside the 24 GB budget and without the default path’s multi-day regeneration (§7.4.1). On a 24 GB machine the deployer can therefore generate either the large, shard-scaling core receipt or the 260-to-868-byte SNARK wrap of it; the post-quantum gap that wrap leaves is taken up in §7.4.1. We report the wrapped figure as measured and do not serialise the workload-dependent core receipt, whose byte count is not the deployment-relevant number.

An empirical witness leak on the non-zero-knowledge core path. The core receipt’s workload-dependence is not merely formal. Fixing the relation and varying only the witness across $N = 20$

standalone PLUM verifications, the Griffin permutation count is witness-invariant at 1,052, while the field-multiplication count and the total cycle count vary from witness to witness (execute mode). The variation reaches the verifier-observable proof. Three prove-mode runs on distinct witnesses ($N = 3$) produce core receipts of 154.67, 156.12, and 153.21 MB, all distinct, a $\approx 1.9\%$ spread. Because the shipped core STARK is not zero-knowledge (§7.4.1), this leak is on a path where no anonymity is claimed. A constant-size zero-knowledge SNARK wrap removes it, its byte count being independent of the witness (about 260 bytes for Groth16 over BN254, 868 for PLONK; Figure 3). That wrap completes on this hardware (§7.4.1), so on the wrapped path the leak closes. What it does not close is post-quantum security, because that wrap rests on a BN254 pairing (§7.4.1): the leak’s fix and the anonymity gap are the same wrap seen from two sides.

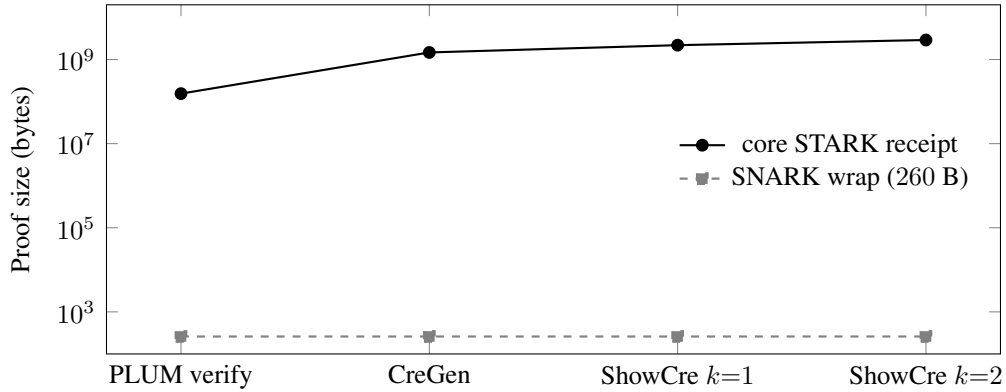


Figure 3: Within-pipeline proof-size collapse (same scheme, same runs).

7.3 Cost Attribution

To attribute the cost we use execute-mode cycle counts at $\lambda = 128$, the audience-relevant security level, where prove-mode is infeasible but cycle counts remain machine-independent. Table 6 reports the cumulative effect of routing PLUM’s field arithmetic through the host precompile, as an ablation over the SHA-3-hashter verification path.

Configuration (cumulative)	SP1 cycles	Δ vs. baseline
Baseline (BigUint emulation)	289,778,709	n/a
+ field mul via host precompile	276,643,994	−4.5%
+ power as square-and-multiply	240,857,140	−16.9%
+ skip redundant reduction	179,952,920	−37.9%

Table 6: Cycle-level cost attribution of the field-arithmetic optimizations.

Where the cost lives, and the parameter-set caveat. At $\lambda = 128$ a single PLUM verification on the SHA-3-hashter arm issues about 112,902 $\mathbb{F}_p^{192}$ multiplication syscalls through the host precompile, as counted in our own SP1 run log. No Griffin permutation executes on that arm, so this figure measures the verifier’s *non-hash* field arithmetic (PLUM §4.2 reports $\approx 10^4$ non-hash algebraic R1CS constraints, 10,333 of 116,285). The Griffin-internal multiplication cost appears instead in

the with/without-AIR contrast at $\lambda = 80$. The software-Griffin arm fires 4,870,724 `UINT256_MUL` syscalls against 69,396 with the Griffin AIR active, the hash-internal field operations either absorbed by the `GRIFFIN_FP192_PERMUTE` syscall or inflating the multiplication count $70\times$. The contrast is therefore consistent with the field-mismatch cost model. The cost is dominated by the algebraic hash, whose internal field operations leave the syscall stream only when the bespoke chip absorbs them. We caution explicitly against transferring the PLUM paper’s circuit-SNARK figure (that 91% of *RICS constraints* are hash work) directly to the zkVM cost model. That figure is a constraint count in a different proving system, whereas Table 6 is a RISC-V cycle count. The qualitative conclusion (hash/field arithmetic dominates) agrees. The exact share differs by cost model and is reported here in the zkVM’s own units.

Execute-mode delta for the Griffin precompile (PLUM-80). For the bespoke Griffin AIR specifically, the precompile-free versus precompile-enabled execute-mode cycle counts differ by a factor of $58.5\times$ at $\lambda = 80$, the ratio of precompile-free to Griffin-AIR-enabled cycles on the Griffin arm. This is close to the field-mismatch tax of Proposition 4.2. It exceeds the schoolbook $\ell^2 \approx 49$ per-multiplication estimate because the whole Griffin permutation collapses into a single precompile call, rather than being emulated one multiplication at a time. This is distinct from the 37.9% of Table 6, a separate field-multiplication-reduction ablation reported at $\lambda = 128$. We report this as an *execute-mode cycle ratio* rather than a wall-clock speedup. It is the machine-independent counterpart of the infeasible-to-feasible prove transition of Table 5, and it is the figure that should be cited when a single number is wanted, in place of any prove-time “speedup,” which is undefined when the baseline does not complete.

7.4 The Credential Relations and the Consumer-Hardware Frontier

Moving from a single signature to the credential-generation relation $R_{\text{cre}}^{\text{Sig}}$ instantiated with PLUM (two signature verifications under a hidden user key; §5.6), Table 7 reports the application-level results at $\lambda = 80$. These are execute-mode cycle counts and SP1 succinct-prove-mode wall-clock for both credential relations.

Workload	Mode	Cost	Outcome
CreGen	execute	9.12×10^{10} cyc	both verifications accept
CreGen, SP1	prove (succinct)	68.96 min (1.47 GB proof, $n=1$)	succinct receipt verified
ShowCre $k=1$, SP1	prove	103.34 min (2.2 GB proof, $n=1$)	succinct receipt verified
ShowCre $k=2$, SP1	prove (succinct)	136.82 min (2.9 GB proof, $n=1$)	succinct receipt verified

Table 7: Credential relations: execute and prove-mode cost ($\lambda = 80$).

The execute-mode result validates the honest path of the BDEC credential relation end-to-end inside the zkVM. Both signature verifications accept, at 9.12×10^{10} cycles. The CreGen prove figure of Table 7 is the config-verified reproducible completing prove, 68.96 min (1.47 GB proof) at

HEIGHT_THRESHOLD = 2^{16} ; an earlier 26.98/27.45 min (296 MB proof) pair was faster but its shard configuration was recorded only in a human-written summary, not machine-echoed, and does not reproduce on the current fork, so it is not used. The ShowCre figures are likewise the config-verified reproducible completing proves at $k=1, 2$. Rejection behaviour was probed only at single-verification scope (Section 5). On SP1 both credential relations complete a succinct (non-zero-knowledge) prove (Table 7); the credential-generation wrap and both the $k=1$ and $k=2$ showing wraps complete and verify within the memory budget, the $k=2$ wrap statement-bound.

Both relations bind their public statement. The proves of Table 7 are outcome-only. A statement-bound variant additionally commits each relation’s public statement to the receipt journal and recovers it on verification. A statement-bound CreGen prove completes in 66.05 min, accepting, with the committed $x_{\text{cre}} = (c, h, \text{ppk})$ recovered and matched against the input; a full end-to-end run binds the showing as well, with two cross-issuer CreGen issuances (66.14 and 65.59 min) and a $k=2$ ShowCre presentation (127.94 min) each accepting, its committed statement recovered and matched, the presentation binding $x_{\text{show}} = ((\text{ppk}_{\text{TA}})_j, \text{ppk}_{UV}, h_{UV})$ while keeping the shown credential private. Binding composes with the zero-knowledge wrap in a single run: a statement-bound CreGen wrap (the jbind guest of Section 5 through the full chain of §7.4.1) completes and verifies in 4.18 h at a 15.2 GiB peak, and the showing wrap likewise (the statement-bound $k=2$ figure in §7.5), so both credential relations carry a measured statement-bound zero-knowledge wrap. The commitment cost is not separately resolvable: the bound CreGen prove (66.05 min) and wrap (4.18 h) each land within a few percent of their outcome-only counterparts (68.96 min, Table 7; 4.20 h, §7.4.1); both sides are single runs, so this is closeness between two runs rather than a variance-resolved bound, and binding is indistinguishable from free at single-run resolution.

7.4.1 The post-quantum frontier

The zero-knowledge wrap of a PLUM proof completes on the target hardware. The full sound chain (core, compress, shrink, wrap to BN254, and a native gnark PLONK proof of the fixed wrap circuit, with verifying-key verification on throughout) produces a verified constant-size proof of one PLUM single-signature verification (Cell 2 of Table 5, the 14.24-min core prove) in 46.83 minutes at a peak of 15.29 GiB, inside the 24 GB envelope with ≈ 8.7 GiB to spare. The credential relation wraps too. A full CreGen zero-knowledge wrap completes in 252 minutes (4.20 h) at a 15.63 GiB peak. Neither is memory-bound.

The wrap avoids the multi-day artifact regeneration the toolchain’s default path implies. On the default path the wrap is killed at the recursive-compression step, because SP1’s static recursion-shape and verifying-key tables are provisioned for smaller relations than PLUM verification, and rebuilding those tables in full is a one-time ≈ 51 h compute over 185,862 entries (projected from a measured per-setup slope of ≈ 1 s across contiguous slices at the start, middle, and end of the table, docs/measurements/zkwrap_vkmap_slope_20260628; the rebuild itself was never run, being exactly what the reduced table avoids). We circumvent that rebuild rather than pay it. The wrap verifies against a *reduced* verifying-key table built from only the guest’s own shard shapes (19 setups in 31 s for PLUM verification, 26 setups in 41 s for CreGen) together with SP1’s stock wrap circuit, which reuses the shipped BN254 gnark circuit unchanged. The sound path is preserved (verifying-key verification stays on), so no recursion-shape rebuild and no circuit rebuild is needed. The gnark

PLONK stage runs with its zero-knowledge blinding enabled, confirmed by a dedicated check in which two proofs of one witness differ under randomized blinding (`docs/measurements/wrap_zk_check_20260707`). The pipeline’s completion and its verifier acceptance are likewise measured. The post-quantum gap below holds independently of that check.

What the wrap does *not* provide is post-quantum anonymity. The wrap rests on a BN254 pairing, so the zero-knowledge it confers is classical-computational, sound against a classical distinguisher but not against a quantum one. A post-quantum-anonymous proof would need a post-quantum-sound recursive zero-knowledge wrap, which this toolchain does not offer. The provisioning wall therefore does not bind here; what does is the dual obstruction, whose analysis is Section 6’s and whose deployment reading is Section 8’s. The substrate can produce a zero-knowledge proof (over a classical pairing) or a post-quantum proof (the non-zero-knowledge core receipt), and not, as it ships, one proof that is both: the anonymity–deployability tension takes its measured form here.

A second and separate cost governs the wrap’s *wall-clock*, and it grows with the credential. The compress stage dominates the wrap tail and scales with the core proof’s shard count. PLUM verification’s 104 shards compress in ≈ 19 min, whereas CreGen must be proved at `HEIGHT_THRESHOLD = 216` to avoid a core-prove overflow, which splits it into 1,018 shards and stretches the compress tail to ≈ 166 min (≈ 0.16 min per shard, near-linear). Peak memory stays bounded at ≈ 15 GiB throughout, so this is a time cost, not a memory cost. The BDEC wraps fit the 24 GB envelope, but their wall grows about linearly with credential size, projecting past any practical bound for large disclosure sets (§7.5). Memory is not the constraint here; the compress-tail cost is.

The precompile enlarges the recursion shape the wrap must verify. The default-path overflow, and the enlarged recursion shape the reduced verifying-key table must cover, attribute entirely to the Griffin- $\mathbb{F}_p^{192}$ precompile. Removing only the Griffin chip, with every other fork edit intact, reproduces the pre-Griffin compose shape byte-for-byte (795,264 provisioned ExtAlu events, md5 `f97a8b2f`). With the chip the provisioned shape is 1,072,032, a marginal +276,768 ExtAlu (+34.8%) and +38,816 PrefixSumChecks, while the fork’s non-Griffin recursion edits move the shape by exactly zero. That enlargement (an actual required height of 1,064,022, just under the provisioned shape, against the default 795,264 budget) is what overflows the arity-4 compose shape on the default path, which is why the wrap is reached instead through a reduced table matched to the guest’s own shard shapes. The precompile that lowers the core-layer cost is therefore the same chip that enlarges the recursion shape the wrap must verify, and hence the compress tail it must pay.

Separately, a *non-zero-knowledge* baseline proof of a standardised-lattice signature verification (an ML-DSA proxy, the 17 NTT-domain polynomial multiplications of 256 coefficients modulo $q = 8,380,417$ that dominate ML-DSA-65 verification, proved as a baseline succinct STARK on SP1, not a zero-knowledge wrap) was jetsam-killed after 249 s with peak RSS 12.59 GB and a virtual-memory footprint of 82.7 GB (compressed memory plus swap, hence exceeding physical RAM). The measurement is a substrate base cost that *strengthens but does not instantiate* the tension and supports the frontier claim, since it bears on the cost of proving any PQ verification on this substrate, not on the zero-knowledge wrap specifically. The positive feasibility anchors are the PLUM single-verification core prove of Table 5 (14.24 min) and its zero-knowledge wrap (46.83 min, above). Table 8 summarises, for every proof mode attempted, whether a receipt is produced and whether it is zero-knowledge and post-quantum.

Substrate	Configuration	ZK	PQ	Time	Peak
SP1	PLUM verify (core)	no	yes	14.24 min	—
	PLUM verify, SHA-3 (core)	no	yes	13.24 min	—
	PLUM verify, BN254 wrap	yes [‡]	no	46.83 min	15.3 GiB
	CreGen (core)	no	yes	68.96 min	—
	CreGen, BN254 wrap	yes [‡]	no	252 min	15.6 GiB
	ShowCre $k=1, 2$ (core)	no	yes	103–137 min	—
	ShowCre $k=1$, BN254 wrap	yes [‡]	no	706 min [¶]	14.93 GiB
Aurora	Loquat verify	no [§]	yes	3.76 min	—
	Loquat verify, ZK	yes	yes	≈ 22 min ($n=3$)	—

Table 8: Proof-mode feasibility: produced, zero-knowledge, post-quantum. [‡] Zero-knowledge only over a BN254 pairing (classical, not post-quantum); [§] zero-knowledge supported for this circuit but disabled in this run; [¶] includes a one-time 2.96 h core-shape pass (decomposition in §7.5, where the statement-bound $k=2$ wrap, 10.21 h, is also reported).

7.5 Calibrating the Cost Model

The cost criterion this chapter develops (*when does a precompile pay?*) is decided on a single declared axis. The trace-area cost model of Section 4 (Equation (1)) prices a primitive by the chip cells a precompile commits against the core cycles it removes, as a function of the field-mismatch width. We make that model the comparison axis explicitly, rather than comparing raw prove times across primitives, so that the matched and mismatched cases are commensurable on one quantity. The measurements above are point observations on this axis. To answer “what would this cost at a configuration we did not run,” we calibrate the model on them, validate it against the points we have, and state the prediction it makes for a point we have not yet run; the deployment decision this calibration feeds is taken up in Section 8. The model is fitted and validated in *execute-mode* cycle counts. Carrying it to prove-mode wall-clock assumes the additive form survives recursion, memory pressure, segment aggregation, and the zero-knowledge wrap, any of which can introduce nonlinearity, so the prove-mode reading is an extrapolation, not a measured fit.

7.5.1 Validation against the measured points

The additive credential-cost model of Section 4 (Equation (3)), linear in the per-verification cost C_{ver} , is falsifiable against the two BDEC points we measured. The strongest internal check is CreGen. Equation (3) predicts the two verifications should each cost $\approx C_{\text{ver}}$ and together dominate the trace. The measurement bears this out directly. CreGen’s software-Griffin execute cost is 9.12×10^{10} cycles for its two verifications (Table 7), i.e. $\approx 4.56 \times 10^{10}$ cycles per verification, and the per-verification cost holds to within 2% across CreGen and both ShowCre instances (below), so the verifications account for essentially the whole CreGen trace and C_0 is small. For the Griffin hasher at $\lambda = 80$, then, $C_{\text{ver}}(80, \text{Griffin}) \approx 4.6 \times 10^{10}$ cycles. This per-verification figure counts a credential-relation verification, about 6,603 Griffin permutations once the relation’s Merkle and pseudonym hashing is included, larger than the 1,052 of a standalone signature verification (Table 5), and consistent with it: both arms

cost the same $\approx 6.9 \times 10^6$ cycles per permutation ($4.56 \times 10^{10}/6,603$ here, $7.21 \times 10^9/1,052$ for the standalone arm below). We caution that this single relation validates the *linearity* and the *verification-dominance*, not the absolute constant at other (λ, h) . The $k = 14$ projection below is labelled as such.

The hasher gap measures the tax. A single PLUM verification is markedly cheaper with the SHA-3 hasher control than with the Griffin algebraic hash, an order-of-magnitude separation that is the field-mismatch tax of Section 4 quantified at the verification level. Matched- λ figures exist on SP1. At $\lambda = 80$ in execute mode, at matched standalone single-verification scope (1,052 Griffin permutations, execute mode), the software-Griffin verification (Cell 1, no precompile) costs 7.21×10^9 cycles against 1.11×10^8 for the SHA-3 control (Cell 3), an order-of-magnitude ($\approx 65\times$) separation; the credential-relation per-verification figure of 4.56×10^{10} cycles is larger only because it counts 6,603 permutations and is not the matched comparison. The qualitative conclusion (that the algebraic hash is the tax-bearing component while the field arithmetic is not, so C_{ver} must be reported per hasher) holds regardless of the exact multiplier. The precompile narrows this gap (Table 6). Closing it entirely is what a native Griffin AIR on the proving path would buy.

ShowCre execute-mode cost (measured at $k = 1, 2$). The ShowCre relation is realised by the guest `bdec_showcre_plum_griffin`, the $k + 2$ generalisation of the CreGen guest, and measured in execute mode at the two running examples. At $k = 1$ (the ϕ_1 workload) ShowCre executes in 1.36×10^{11} cycles, and at $k = 2$ (the ϕ_2 workload) in 1.82×10^{11} cycles, each with all $k + 2$ verifications accepting. Both agree with Equation (4) at $C_{\text{ver}}(80, \text{Griffin}) \approx 4.6 \times 10^{10}$. The per-verification cost is constant at $\approx 4.6 \times 10^{10}$ cycles across CreGen and both ShowCre instances (spread under 2%), so the additive $(k + 2) C_{\text{ver}}$ form is measured rather than assumed. Applying the same measured constant to a stress-test disclosure-set size, well beyond the running examples ($k = 1, 2$) and the $k \leq 4$ design target, projects $\approx 7.3 \times 10^{11}$ cycles at $k = 14$. A direct $k = 14$ execute run does not reach this projection. The 16 accumulated verifications exhaust the guest’s default non-reclaiming bump allocator before the cycle count completes, an in-guest memory limit distinct from the prover-side wrap cost of §7.4.1 and clearable only by the reclaiming allocator at additional cycle cost. The $k = 1$ and $k = 2$ executions are correct and their cost is measured. The zero-knowledge *proof* runs through the same wrap pipeline that completes for PLUM verification and CreGen (§7.4.1). The $k = 1$ wrap completes and verifies on the target hardware in 11.76 hours end-to-end (8.79 hours for the per-proof wrap chain plus a one-time 2.96 hour core-shape pass) at a 14.93 GiB peak, within the 24 GB envelope. The $k = 2$ wrap likewise completes and verifies, and is statement-bound (the jbind guest of Section 5 put through the same chain): 10.21 hours end-to-end (7.91 hours for the per-proof wrap chain plus a one-time 2.30 hour core-shape pass) at a 15.66 GiB peak, again within the envelope. Across the two runs the quantities that grow with the verification count are the core-proof shard count (1516 at $k = 1$, 2011 at $k = 2$) and the peak memory (14.93 $\rightarrow$ 15.66 GiB), both staying within the budget; these are the configurable readings of the compress-dominated model. The per-proof wall-clock is a single-run figure, not controlled for machine state across the two runs (see the thermal-throttling limitation below), so we rest the growth-in- k statement on shard count and peak memory rather than on wall-clock. As with those workloads, the wrap it produces is zero-knowledge only over a BN254 pairing (§7.4.1).

Calibrating the trace-area constants. The trace-area model carries two constants. On the replicated runs of Table 5 (Cell 2 Griffin 14.24 min, Cell 3 SHA-3 13.24 min; §7.2) the Griffin arm’s prove-time residual over the control is only about a minute, too small to pin a positive per-cell rate against the chip’s 5.6×10^7 cells, so the additive prove-time decomposition is near-degenerate on the trustworthy data. We therefore report an *illustrative* calibration on an earlier run pair whose Cell 2 is the 32.53-minute figure not reproduced under replication (§7.2): there the control prices core work at $\kappa_{\text{core}} \bar{a}_{\text{core}} \approx 7.2 \mu\text{s}$ per guest cycle, and attributing the Griffin arm’s residual 17.5 minutes to its 5.6×10^7 cells gives $\kappa_{\text{Griffin}} \approx 19 \mu\text{s}$ per committed cell (about 2.6 guest cycles bought per committed cell). This magnitude is illustrative only, and nothing downstream rests on it. The model’s out-of-fit content is the *sign*, which is fixed by the committed cell and execute-mode cycle counts of §7.2 (5.6×10^7 Griffin cells and 11% more guest cycles, both machine-independent), and is therefore unaffected by the retired pair. Those orderings are consistent with the rest of Table 5. The precompile-free baseline’s 7.21×10^9 cycles price at upwards of 14 hours of core work alone (before the chip area of its 4,870,724 `UINT256_MUL` events is charged), consistent with its DNF, and against that baseline the precompile still pays by orders of magnitude, removing about 6.6×10^6 cycles per permutation against 5.3×10^4 cells added.

A deductive forecast: the matched-field point. The illustrative calibration above fixes two constants, both on the *mismatched* side of the axis, where the primitive’s native field (199 bits) is far wider than the prover’s (31 bits). The model’s out-of-fit content is the prediction it makes for a *matched-field* operating point, at which a primitive native to the prover field needs essentially one limb, so a precompile commits chip area without removing a comparable multi-limb emulation from the core. On that point the precompile-pays inequality (Proposition 4.4, Equation (2)) predicts the precompile does *not* pay. The removed core area shrinks toward zero while the added chip area does not, so the sign that the mismatched measurements postdict should *reverse*. This forecast is, however, a *deductive* consequence of the trace-area model rather than an independent test of it. As $\ell \rightarrow 1$ the removed core area shrinks toward a single native operation while the added chip area does not, so the model cannot but predict no-pay there. An independent, out-of-sample test needs an operating point the two-constant calibration did not fix, and within PLUM that point is the already-served `FP192_POW_RES` break-even (§7.5.3), a large-prime chip on the construction’s own primitive whose verdict the model forecasts and whose deciding measurement we specify there. A matched-field ($\ell = 1$) point would be a further test, but every primitive in PLUM is $\ell = 7$, so carrying it in prove mode would require building and proving a chip for an operation outside the construction, which we do not do; the execute-mode necessity test of the next subsection needs no chip and does use an out-of-construction $\ell = 1$ multiplication. We therefore report the criterion as *postdictive* on its two calibration points, with its out-of-fit content deductive at $\ell = 1$ and tested at the `FP192_POW_RES` point, where the measured pay decides the forecast in its favour (§7.5.3).

7.5.2 Matched-field control

Distinct from the deductive matched-field forecast above (the keystone control of Section 4, now a deductive consequence of the model), the necessity claim of the criterion (Proposition 4.5), that the field-mismatch benefit scales with the width ℓ and vanishes at $\ell = 1$, admits a direct execute-mode test, forecast before the run (`docs/measurements/keystone_matched_field_20260701`). We measure the per-multiplication emulated cost of a field-matched primitive (a KoalaBear single-limb

multiplication, $\ell = 1$, native to the prover field) against the mismatched $\mathbb{F}_p^{192}$ software multiplication ($\ell = 7$), giving 19 guest cycles per matched multiplication against 3,121 for the mismatched one, a $164\times$ field-mismatch tax. The emulated cost a precompile removes is therefore large at $\ell = 7$ and essentially nothing at $\ell = 1$, confirming the necessity direction. The ratio exceeds the schoolbook $\ell^2 = 49$ because the $\mathbb{F}_p^{192}$ software path is a general bignum (`num_bigint`) rather than a limb-aligned routine, an implementation datum consistent with the ℓ lower bound of Proposition 4.2, not a violation. The naive KoalaBear reduction makes $164\times$ a conservative estimate. This establishes the necessity direction in execute mode. The prove-mode corollary, that the precompile does not pay at $\ell = 1$ (Proposition 4.4), follows deductively from the trace-area model, and PLUM has no matched-field primitive on which to test it independently.

A field-width control. The two measured widths bracket the effect. A single field multiplication (the operation a precompile removes) costs 19 cycles at $\ell = 1$ and 3,121 at $\ell = 7$, a $164\times$ span. A general bignum library (`num_bigint`) is unsuited to a finer shape measurement, its allocation and division overhead masking the scaling at these limb counts, so we read these as conservative endpoint magnitudes rather than a scaling law.

7.5.3 The already-served case, and the break-even that decides it

The matched-field control probes one reason a precompile does not pay, $\ell = 1$, where there is no multi-limb emulation to remove. A second, distinct case arises at full mismatch ($\ell = 7$) when a primitive’s multi-limb arithmetic is *already* served by an existing precompile, and it is where the criterion’s positive question lives. The `FP192_POW_RES` chip (§5.5) makes it concrete. It composes the power-residue-PRF symbol $a^{(p-1)/t} \bmod p$ as a 191-step square-and-multiply, every step a 199-bit modular multiplication the host `UINT256_MUL` already carries inside the guest loop. On the arithmetic axis a dedicated chip does not pay. As a naive square-and-multiply chip it commits 382 `FieldOpCols` per symbol against the guest loop’s 261, *adding* field-multiplication area rather than removing it (the chip evaluates both the square and the multiply branch at every one of the 191 steps, $382 = 2 \times 191$, whereas the guest loop multiplies only at the exponent’s set bits, 191 squarings plus ≈ 70 multiplications ≈ 261). But that comparison is incomplete. It counts field-op columns only, and omits the guest loop’s per-symbol *control* overhead, namely one syscall dispatch, memory access, and cross-table lookup for each of the ≈ 261 multiplications, against one for a fused chip. A fused fixed-exponent chip (a preprocessed addition chain, not naive square-and-multiply) would pay that control width once. The break-even therefore turns on a quantity the field-op measurement did not isolate, the $\approx 261:1$ control-overhead ratio. The deciding measurement, the chip against the guest loop on *total* committed area (field-op plus control) at $\ell = 7$ on the honest `UINT256_MUL` baseline: the chip commits 11.37M total trace cells against the baseline’s 117.46M over 28 symbols ($n=3$, deterministic; docs/measurements/onspec_powres_settle_20260711), a $10.33\times$ pay. The $\approx 261:1$ syscall-count collapse dominates the baseline’s shards, so the chip pays even in naive square-and-multiply form; a fused fixed-exponent chip would only widen the ratio. The already-served case does still sharpen why the criterion is priced on trace area, not guest cycles. In execute mode the chip costs 365 cycles against the guest loop’s 6,549 (a $17.9\times$ apparent saving), the opposite of the field-op-column comparison (382 against 261), so a criterion read off execute cycles alone would mislead. To sum up, there are two paying cases and one losing case. Griffin pays against its own emulated base-

line (the infeasible-to-feasible transition of §7.2, whose machine-independent counterpart is the $58.5\times$ execute-cycle collapse), because it is mismatched *and* unserved, yet it loses the security-matched comparison. The matched-field control confirms the necessity direction ($164\times$). And the power-residue chip, the criterion’s irreducibly-large-prime target, is measured to *pay* $10.33\times$ on total committed area. These are two paying instances, each against its own declared baseline.

7.6 The Circuit-SNARK Reference Point

To situate the zkVM costs against the static-circuit regime they are meant to escape, we measured the Aurora prover end-to-end on the same hardware over a single Loquat-verification R1CS. This is the single-verification building block the BDEC construction composes [5]. The full credential relation merges two such verifications under a shared hidden public key, so it is roughly twice this figure, and we did not run the composed relation statically. We measure it directly rather than cite it, so that the comparison is on our hardware rather than across machines. The reported values are means of three runs with ranges in parentheses over the R1CS ($\mathbb{F}_{p^2}$, 127-bit), and the prover time includes Aurora’s parameter setup but not per-change recompilation (§7.7).

Quantity	Value
Constraints (padded)	524,288
Prover time (setup + prove)	225.4 s (3.76 min; 223.2–226.6)
Verifier time	41.3 s (41.0–41.6)
Proof size	704,416 B (688 KiB, identical across runs)
Achieved security	107.5 bits (target 128)
Zero-knowledge	disabled in this run; supported
Prover time, zk enabled	≈ 22 min ($n=3$)

Table 9: Aurora reference: one Loquat verification, three runs.

The zero-knowledge-enabled run. The runs of Table 9 have zero knowledge disabled. We additionally ran the same 2^{19} R1CS (524,288 constraints, RS extra dimensions 5) with zero knowledge *enabled*, three solo runs. The prover takes 1317.7 s at the mean (21.96 min; 1285.7–1376.1 s), the verifier 40.8–41.8 s, and the proof is 876,768 B (856 KiB), identical across the three runs, against 704,416 B with zk disabled. Achieved security is 107.5 bits (target 128, as in the zk-disabled legs), peak RSS spans 9.2–11.0 GiB, comfortably inside the 24 GB envelope, and the verifier returns ACCEPT in every run. Against the zk-disabled prover mean of 225.4 s, the measured zero-knowledge premium on this circuit is about $5.8\times$ prover time at the means.

What this reference does and does not license. Two readings of this point are legitimate and one is not. *Legitimate*: on this hardware the static-circuit regime proves a single Loquat verification in minutes, against the zkVM’s tens-of-minutes-to-infeasible (Tables 5, 7), so for a *fixed* predicate the static circuit is the faster prover, as one would expect from a specialised encoding. *Also legitimate*: the zk-disabled instance of Table 9, like the zkVM succinct receipt, does not by itself discharge BDEC’s anonymity requirement. The zero-knowledge-enabled run reported above supplies that property for the same circuit, at the measured premium. *Not licensed*: reading Table 9 *against the PLUM zkVM*

numbers (Tables 5, 7) as a same-scheme verdict, since that pairs Loquat over a 127-bit field with PLUM over a 199-bit field. A same-scheme verdict would require running one scheme on both substrates at matched security and rate, which this thesis does not measure and identifies as future work. Either reading is timed without the predicate-change cost that is the separate subject of the update-churn comparison, developed next.

7.7 Update Churn

Table 10 enumerates the regeneration footprint of Definition 4.7 for the three localised verification-predicate changes (Definition 4.6) of §4.4, using $\phi_1 \rightarrow \phi_2$ (par. 1) as the presentation-change instance. Here $\phi_1 \rightarrow \phi_2$ is a presentation-*structure* change. It moves the credential count from $k=1$ to $k=2$, altering the relation $R_{\text{show}}^{\text{Sig}}$ and hence the static R1CS. A presentation *policy* change at fixed structure (e.g. retuning a threshold the verifier checks on the disclosed attributes) is handled verifier-side and triggers no regeneration on either substrate, so the rows below measure structural changes.

Localised change	Static circuit (Aurora)		zkVM	
	N_{circ}	N_{param}	N_{circ}	N_{param}
Presentation change ($\phi_1 \rightarrow \phi_2, k:1 \rightarrow 2$)	≥ 1	≥ 1	0	0
Signature swap (Loquat $\rightarrow$ PLUM)	≥ 1	≥ 1	$\geq 1^*$	$\geq 1^*$
Security retune ($\lambda=80 \rightarrow 128$)	≥ 1	≥ 1	0	0

Table 10: Regeneration footprint per localised relation change (Def. 4.7). *Loquat $\rightarrow$ PLUM regenerates the field-specific Griffin AIR.

Structural separation, and the measured recompile cost. The structural separation in Table 10 is established by construction. A zkVM’s relation is fixed to the ISA, so a presentation-structure or signature change is absorbed as a source edit ($N_{\text{circ}} = N_{\text{param}} = 0$) with no recompilation. *Both* components of the static side are measured, and recompilation is sub-second. R1CS *construction* from the credential relation is sub-second (≈ 0.2 s of CPU time per emission at Loquat- $\lambda=80$ and $\lambda=128$, single timed runs; cached wall-clock 0.48 s). The Aurora *parameter setup* that a relation change forces to recompute is, measured directly, about $3 \mu\text{s}$ (the runner reports $t_{\text{setup}} = 0.003$ ms at the 2^{19} -padded single-Loquat-verification R1CS, the BDEC building-block size), because Aurora is *non-preprocessing*. It has no per-circuit indexer. The prover’s low-degree-encoding FFTs (its dominant step) run *per proof*, not per relation change, and so do not enter R_{static} . Hence $R_{\text{static}} \approx 0.3$ s at the BDEC building-block size. A $\phi_1 \rightarrow \phi_2$ change alters only the credential count and does not enlarge the microsecond parameter setup, so this bound also holds for the per-presentation delta. A k -parameterised emitter would refine the R1CS-construction term but cannot move the dominant conclusion, namely that for a transparent non-preprocessing argument the per-change recompile is sub-second. The zkVM therefore gains no measurable wall-clock recompile advantage over Aurora; what separates the regimes on this dimension is the re-audit footprint of Table 10, and the deployment weighing of that trade is Section 8’s.

The source-edit and audit components. The remaining tuple components, promised in prose by Section 4.4, are read off this project’s own repository record. For the security retune ($\lambda = 80 \rightarrow 128$)

the zkVM side is the clean case. The guest binary is parameterised over λ , so $N_{\text{src}} = 0$ and $N_{\text{audit}} = 0$. The static side keeps its generator source ($N_{\text{src}} = 0$) but its regenerated matrices re-enter the review boundary ($N_{\text{audit}} \geq 1$). For the presentation change ($k:1 \rightarrow 2$) the zkVM edits one guest source (the presentation relation’s credential count), giving $N_{\text{src}} = 1$ and $N_{\text{audit}} = 1$ (the changed guest and its image identifier). The static side regenerates the merged R1CS, with the regenerated circuit as the re-audited component. For the signature swap (Loquat $\rightarrow$ PLUM) the static side replaces the constraint generator for the verification predicate and re-audits the full regenerated circuit. The zkVM side is the honest headline of this dimension. The guest edit itself is small, but because the swap moves the hash field, it crossed the precompile boundary, and in this project’s record the crossing cost was the bespoke Griffin- $\mathbb{F}_p^{192}$ chip, approximately 3,900 lines of chip and executor code (§5), with the chip and its soundness argument (Appendix B) entering the audit boundary as new components. The absorb-as-software property therefore held for every benchmark change except the one that touched a precompiled primitive, where the zkVM regime paid a circuit-engineering cost of the same kind it was adopted to avoid.

The advantage is conditional on the baseline’s preprocessing model. The negligible recompile cost above is a property of Aurora’s *non-preprocessing* structure. It does not generalise to static circuits at large. A *preprocessing* (holographic) transparent argument such as Fractal [33] must instead build a per-circuit index, and that index is exactly what a relation change forces to regenerate. Timing Fractal’s indexer (its `fractal_snark_indexer` stage) in isolation on the same 127-bit-field harness as §7.6 (≈ 107 -bit achieved, zero-knowledge disabled; a Loquat proxy for PLUM) gives the contrast of Table 11. The 2^{10} – 2^{14} rows are synthetic R1CS instances emitted on the same harness to expose the indexer’s scaling. Only the 2^{19} row is the single-Loquat-verification relation at the BDEC building-block size. Aurora’s parameter setup is microseconds and flat in circuit size, whereas Fractal’s holographic indexer grows superlinearly and, at the 2^{19} -padded size, is killed under memory pressure mid-indexer (during a 2^{27} -element FFT) before returning.

R1CS size (constraints)	Aurora param. setup (one component of R_{static})	Fractal indexer (one component of R_{static})
2^{10} (1,024)	0.0021 ms	0.86 s
2^{12} (4,096)	0.0023 ms	3.74 s
2^{14} (16,384)	0.0025 ms	18.5 s
2^{19} (524,288; Loquat single-verify)	0.0029 ms	OOM-killed at ≈ 33 min

Table 11: Per-change recompile cost: Aurora versus Fractal.

The flexibility reading is therefore *conditional on the static baseline’s preprocessing model* (its deployment reading is Section 8’s). The zkVM’s wall-clock recompile advantage is negligible against a non-preprocessing argument (Aurora) and large against a preprocessing one (Fractal), where at the BDEC circuit size the baseline’s per-circuit indexing is itself infeasible on the target hardware. The same-scheme seam of §7.6 applies here too, and a smaller folding parameter than the Aurora-matched $RS = 5$ might let the Fractal indexer fit and yield a finite but still large R_{static} .

7.8 Threats to Validity

- **Run counts and variance.** The Cell 2 and Cell 3 wall-clocks (Table 5) are means of five fixed-config runs each (ranges reported) and the Aurora reference a mean of three ($\pm 0.8\%$); every other prove-mode and zero-knowledge-wrap figure is a single run, for which fixed-config replication was not undertaken. Wall-clock is not controlled for thermal state, which moves laptop-class timings by factors of 1.4–1.6, so conclusions rest on order-of-magnitude separations, the memory-exhaustion walls, and shard-count and peak-memory monotonicity rather than on single wall-clocks (this is also why the single-run wrap times are not read as a k -series). The Cell 2-versus-Cell 3 inversion is confirmed sign-robust across all five runs per arm (Griffin slower in every run, $\approx 1.08\times$, each arm’s spread under 5%); its sign is model-postdicted and its magnitude we do not over-read.
- **Thermal throttling.** Runs are on a laptop-class M5 Pro. Sustained multi-minute proving may throttle, which would inflate wall-clock relative to a thermally-unconstrained host.
- **Hardware/OS-string provenance.** The reported platform string is from `sysctl`. We have not cross-checked every sub-component against an independent inventory.
- **Parameter-set seam.** Prove-mode wall-clock is at $\lambda = 80$. The cost-attribution analysis of §7.3 is at $\lambda = 128$, while the Griffin-precompile cycle ratio ($58.5\times$) and the credential-relation cycle counts are execute-mode figures at $\lambda = 80$. No figure combines the two levels, but a reader must track which is which. The tables are labelled accordingly.
- **Constraint-level soundness of the Griffin AIR is empirically supported, not proved** (the chip–constraint equality, Assumption 6.1, of Section 6). The positive feasibility figures, namely the 14.24-minute Cell 2 prove, its 46.83-minute zero-knowledge wrap, the 13.0-second smoke proof, and any prove-level reading of the $58.5\times$ cycle ratio, assume the chip computes the reference permutation: proved at the per-row layer (Appendix B, conditional on the byte-range lookups under the lookup-binding hypothesis), tested on 256 random vectors and by single-cell fault injection on seven of the ten constraint families, and not verified at the constraint level for the lookup-borne families or the code-to-constraint transcription. The negative findings of this chapter do not carry this premise. An unsound AIR only understates the Griffin-arm cost, so the non-terminating precompile-free baseline, the SHA-3 inversion, and the wrap’s credential-size-scaling compress tail survive unchanged or strengthen (the argument is developed in Section 6).
- **Control vs. deployable.** The SHA-3 arm is a diagnostic control. It is not a secure PLUM variant and its timing is not a deployment claim.

7.9 Summary

The measurements above establish the four points of this section’s opening spine; Section 8 takes up what they mean for the deployment decision and which questions they leave open.

8 Discussion

The measurements of Section 7 and the security analysis of Section 6 together answer the question this thesis set out to ask. That question is whether a zkVM is a viable substrate for post-quantum anonymous credentials, and at what cost, not how fast the zkVM runs in isolation. We begin with the main finding, the anonymity–deployability tension, then give the cost results as guidance for a deployer.

8.1 The Anonymity–Deployability Tension

The anonymity–deployability tension of Section 6 matters more than any single timing figure. Two things hold together on the target hardware at $\lambda = 80$. First, the proof that provides anonymity both runs and binds. Both credential relations commit their public statement to the receipt, the full BDEC lifecycle runs end to end (two cross-issuer issuances and a $k = 2$ showing), and the zero-knowledge wrap that BDEC’s anonymity requires completes within the 24 GB budget. The standalone verification wraps in 46.83 minutes and the *statement-bound* issuance receipt in 4.18 hours, each near 15 GiB (Section 7). So binding and zero-knowledge combine in one measured proof, not two separate demonstrations. Second, that wrap is pairing-based, so the anonymity it delivers is classical only. A deployer who needs *post-quantum* anonymity on this stack still cannot get it, and the reason is not a shortage of memory or time. A reduced verifying-key table built from the guest’s own shard shapes (§7.4.1) closes the chain without a multi-day rebuild, so the memory cost of that rebuild is not what blocks anonymity. What blocks it is the primitive. The only wrap the toolchain provides uses a pairing again. This is the single remaining side of the dual obstruction, the second obstruction (Section 6), a limit of primitives rather than of resources. Recovering post-quantum anonymity needs a post-quantum-sound zero-knowledge wrap, a STARK- or lattice-based recursive argument the toolchain does not yet provide.

The obstruction is specific to the zkVM stack. The static-circuit regime does produce a zero-knowledge post-quantum proof of the Loquat single-verification building block in about 22 minutes (mean of three runs, 21.4–22.9) on the same machine (`docs/measurements/audit5_campaign_20260612`), at the per-change regeneration cost weighed below. And the classical anonymity the zkVM stack does deliver is itself conditional. It rests on two open assumptions of Section 6: PLUM’s signature-transcript simulatability (Assumption 6.2) and, for issuance, its signature key-privacy (Assumption 6.3), now essential because statement-binding makes the issuance credential a public input, together with the zero-knowledge wrap that BDEC’s anonymity requires. The tension holds independently of how favourable the timing numbers are and of the open AIR-soundness Assumption 6.1, which conditions the feasibility figure rather than the obstruction.

8.2 A Substrate-Selection Rule

The two regimes do not dominate each other. Each trades a different cost. The right choice depends on one property of the deployment: how often the verification rule changes compared with how often proofs are generated.

- **When the predicate is fixed at deployment time** and proofs are generated frequently, the static-circuit regime is preferable: its specialised RICS amortises across many showings, and it pays

its update cost only once. The zkVM’s per-proof overhead (Section 7) is not justified when there is nothing to update.

- **When the predicate is chosen at presentation time, or the signature or security parameters evolve over the deployment’s lifetime**, the static-circuit regime pays its update-churn cost (Table 10) *repeatedly*, and the zkVM regime, which absorbs each such change as a guest-program edit, becomes the structurally appropriate substrate, at the proving cost quantified in Section 7. The exception is a change to the precompiled primitive itself, which regenerates its chip (below). For a non-preprocessing static argument such as Aurora this update-churn cost is a regeneration-and-re-audit (governance and assurance) burden rather than a large wall-clock penalty, which the next paragraph makes precise.

This rule can be made precise. Over a deployment lifetime of P proof generations and U localised predicate changes, the amortised cost of each regime is, schematically,

$$\text{static: } U \cdot R_{\text{static}} + P \cdot t_{\text{static}}^P, \quad (7)$$

$$\text{zkVM: } U \cdot 0 + P \cdot t_{\text{zkVM}}^P, \quad (8)$$

where R_{static} is the per-change recompilation cost and t^P the per-proof time; the zkVM absorbs each predicate change as a guest-program edit, so its update term is zero. The static regime is cheaper only while the predicate-change rate stays below the crossover

$$\left(\frac{U}{P}\right)^* = \frac{t_{\text{zkVM}}^P - t_{\text{static}}^P}{R_{\text{static}}}, \quad (9)$$

above which the zkVM is the cheaper substrate: the threshold is the ratio of the zkVM’s per-proof penalty to the static circuit’s per-change recompilation cost. The recompilation cost is measured (Section 7): for a transparent non-preprocessing argument such as Aurora it is $R_{\text{static}} \approx 0.3$ s (R1CS construction plus a microsecond parameter setup, single timed runs), and it sits outside the multi-minute prover, which is paid per proof. At that magnitude, and with the zkVM dearer per proof on the cross-scheme reading available to us, the wall-clock crossover would favour the static circuit in any realistic churn regime. Against Aurora, then, the zkVM has no wall-clock flexibility advantage on that reading. The same-scheme sign of the crossover is settled below as a conjecture, not a measurement. The flexibility claim we can defend is instead the *qualitative* regeneration-and-re-audit footprint of Table 10 (zero on the zkVM side, ≥ 1 on the static side per change), an engineering-and-assurance cost paid by humans, not prover time.

A wall-clock advantage appears only against a *preprocessing* baseline, which Section 7 measures for the transparent preprocessing argument Fractal (Table 11) and which would hold in principle for Groth16 [49], Marlin [34], or circuit-specific PLONK [98]; the non-preprocessing Aurora that BDEC uses does not offer one.

This zero-regeneration property is partitioned rather than absolute. A change confined to the verification predicate is a guest-program edit that reuses the existing universal relation, but a change to the hash family or to the field width still regenerates and re-audits the Griffin- $\mathbb{F}_p^{192}$ precompile, so the decision rule applies to predicate churn and not to substrate-level changes of the signature itself.

What the cost model licenses is limited: it fixes the *form* of the rule and the *sign* of every term, but not the numeric location of the crossover for PLUM. So the rule gives directional guidance, telling

a deployer which side a high- or low-churn deployment lies on without yet handing them a numeric threshold to evaluate.

One asymmetry the rule abstracts away: the zkVM’s per-proof cost is flat only on the issuance side. A k -credential presentation’s cost grows with the credential count: execute-mode cycles rise from 1.36×10^{11} at $k = 1$ to 1.82×10^{11} at $k = 2$, and the zero-knowledge wrap’s core-shard count and peak memory grow with them (both the $k = 1$ and $k = 2$ wraps measured, within the 24 GB budget; Section 7), so for a large number of shown credentials the zkVM showing cost projects past a practical bound even where the churn rate favours the zkVM. The credential count is therefore a second axis the deployer weighs alongside the churn rate the rule captures.

The contribution of this thesis to that rule is the *quantification of both sides*: the zkVM’s per-proof cost is measured (Tables 5–7), and the structural form of the static circuit’s update cost is enumerated (Table 10). The static circuit’s recompile *time* is measured ($R_{\text{static}} \approx 0.3$ s, §7.7) and is sub-second because Aurora is non-preprocessing; the remaining term is a *same-scheme* per-proof prover time on both substrates, and this we do not measure: neither Loquat nor PLUM is run on both substrates at matched security and rate. We expect the specialised static circuit to be the cheaper per-proof prover, since a native constraint system avoids the zkVM’s emulation of the signature’s field arithmetic, but the sign of that crossover is a conjecture rather than a measurement. Settling it requires an $\mathbb{F}_p$ (199-bit) Aurora harness to run PLUM statically (the current harness is 127-bit only), which we identify as the required follow-up experiment.

8.3 What Generalises and What Does Not

The field-mismatch tax (Section 4) and the update-churn dimension (Definition 4.7) are substrate-independent: they apply to any large-field algebraic-hash post-quantum signature of this form verified in any small-field zkVM that exposes application-defined precompiles, not only to PLUM in SP1. The specific magnitudes (Section 7) do not generalise: they are tied to PLUM’s 199-bit field, the M5 Pro’s 24 GB envelope, and the CPU-bound prover of the SP1 zkVM studied. We drew the boundary deliberately, so that the shape of the cost carries to other pairings of a large-field signature with a small-field zkVM while the numbers stay pinned to the configuration measured here.

8.4 Comparison to Prior Evaluations

As anticipated in Section 2, the prior-evaluation record omits two axes: the cost of a *change* to the verification predicate, and the consumer-hardware proving frontier for an algebraic-hash post-quantum signature. We do not restate that gap here; instead we report what our measurements add to the record. For predicate-change cost, Table 10 fixes its structural form (zero regenerations on the zkVM side against ≥ 1 per change on the static side) and §7.7 bounds its wall-clock magnitude; for the frontier, §7.4.1 locates the binding constraint at the zero-knowledge wrap rather than the base proof: as built and measured the wrap completes within budget, so what binds is its post-quantum soundness and not its feasibility (the base-proof cost is the honest-path cost of the chip and carries Assumption 6.1). Where our per-proof timings overlap the existing record, they agree in order of magnitude with the hitherto-qualitative observation that algebraic-hash workloads are expensive inside small-field zkVMs, now quantified at the verification level (Tables 5–6). The next section combines these into the thesis’s overall conclusion.

9 Conclusion

This thesis asked whether a post-quantum anonymous credential can run inside a zero-knowledge virtual machine (zkVM) on consumer hardware without breaking the security properties it must provide, with BDEC instantiated with the PLUM signature as the concrete workload on a personal computer (Apple M5 Pro, 24 GB). The answer has several parts. Precompile-free PLUM verification does not run, because the field-mismatch tax of its large-prime-field algebraic hashing leaves it non-terminating on the target machine (over five hours without terminating; Section 7). A custom precompile suite makes it run, turning the non-terminating emulation into a finite proof whose cost can be attributed and bounded. That clears the feasibility bar we can state operationally: completion within the target machine’s 24 GB envelope in bounded wall-clock. Sub-second interactive-showing latency, orders of magnitude below the tens of minutes to hours these proofs take, stays out of reach and is left to future work. With the credential running, a conditional preservation theorem bounds the shift to its unforgeability, anonymity, and unlinkability advantages, and finds them preserved under five explicitly named assumptions and, for the knowledge soundness of the multi-shard receipt, one named extractor conjecture (Section 6). One property the solution still leaves out of reach is deployable post-quantum anonymity, because the only zero-knowledge wrap the toolchain ships is pairing-based. The cost criterion, the hash-cost inversion, and the deployment decision rule fill in that answer, and the following subsections take them in turn. We built the construction on SP1 and measured it against an Aurora static-circuit baseline.

9.1 Main Findings

The cost criterion. A precompile lowers core-proving cost only on the field-mismatch axis, and whether it pays is a trace-area inequality read against two baselines. Against a precompile’s own emulated baseline the criterion measures *feasibility*. Against a security-matched control it measures whether the custom chip is worth building. The three shipped modules fall out differently under the two. The Griffin- $\mathbb{F}_p^{192}$ hash pays for feasibility, recovering a finite proof from an emulation stopped unfinished after five hours, yet loses the security-matched comparison, proving slower than a software SHA-3 at matched security (14.24 versus 13.24 min, $n = 5$) in an inversion whose sign the trace-area model postdicts. The field-multiplication chip is predicted not to pay, a design-level trace-area comparison rather than an end-to-end measurement, because a dedicated chip’s whole-table overhead is not amortised when the host precompile already serves its arithmetic (a dedicated chip is in fact narrower per operation; §5). And the power-residue symbol, irreducibly large-prime yet already-served, still pays $10.33\times$ on total committed trace area. That saving comes from the $\approx 261:1$ control-overhead collapse rather than from any arithmetic saving, and that overhead exists only because the primitive is multi-limb, so this saving too is a field-mismatch effect (Sections 4 and 7).

The dual obstruction. The sharpest *finding*, on the security axis rather than the cost axis, is the anonymity–deployability tension (Section 6): the dual obstruction to post-quantum anonymity reduces to its single remaining side, the second obstruction. Feasibility is not the issue. The full zero-knowledge wrap chain completes within the memory budget, on consumer hardware and without the default path’s multi-day regeneration, so neither the multi-day rebuild the default path would need nor the memory budget is what blocks anonymity (the figures and the reduced verifying-key table are in

Section 7). What blocks it is a fact about the primitive. The wrap the toolchain ships is pairing-based, so the anonymity it delivers is classical rather than post-quantum, and no proof on this stack is at once zero-knowledge and post-quantum. Recovering post-quantum anonymity needs a post-quantum-sound recursive zero-knowledge wrap, a STARK- or lattice-based argument the deployed toolchain does not yet provide. This obstruction holds independently of any timing number and of the open AIR-soundness Assumption 6.1 (the dependency direction is recorded in Section 6 and restated under Limitations below). Pushed to its information-theoretic limit, the boundary is proved: for the hash-committed showing class this construction uses, *everlasting* anonymity cannot be established in the standard model (Theorem 6.10), and the conditional route out is a non-interactive statistically-hiding trace commitment (Proposition 6.12).

Feasibility, and two falsified predictions. Around that finding, the measurements establish the surrounding cost picture. The custom Griffin- $\mathbb{F}_p^{192}$ precompile turns a non-terminating emulation into a finite measurement: without it, standalone PLUM verification does not complete on the target hardware within the hours attempted (over five hours without terminating; Section 7); with it, the verification proves in 14.24 minutes (mean of five fixed-config runs, 14.16–14.44; Section 7) on SP1 at $\lambda = 80$, under the documented reduced-memory configuration, default settings exhausting memory (Section 7). Two of the measurements falsify natural predictions. First, the algebraic-hash precompile is *slower* than a software SHA-3 control at matched security in every run we took, on both builds we tested. The sign is robust; under fixed-config replication ($n = 5$ per arm) the magnitude is $\approx 1.08\times$ in prove wall-clock (14.24 versus 13.24 minutes). Arithmetising a 199-bit-field algebraic hash through a custom chip costs more than a hash the substrate already supports. The trace-area accounting of Section 4 postdicts the sign of this inversion from the chips’ committed dimensions: the custom chip adds roughly 5.6×10^7 committed trace cells per verification (a count derived from the chip’s committed dimensions, not a logged trace readout) that the control never pays, a term the guest-cycle accounting alone does not carry. Second, against the transparent, non-preprocessing Aurora baseline the zkVM’s update-churn advantage never shows up in wall-clock. The static-side recompile is sub-second ($R_{\text{static}} \approx 0.3$ s, Section 7.7), so what separates the two regimes is the regeneration-and-re-audit *footprint* (Table 10), a cost in engineering and assurance rather than in proving time. A wall-clock flexibility advantage would appear only against a *preprocessing* baseline, which is not what BDEC uses.

The credential relations. The credential relations themselves run correctly inside the zkVM. Credential generation executes both of its signature verifications to acceptance in 9.12×10^{10} guest cycles, and the presentation relation in 1.36×10^{11} (one credential shown) and 1.82×10^{11} (two) cycles, consistent with the additive cost model. On SP1 both relations complete a succinct, non-zero-knowledge prove (credential generation in 68.96 min (1.47 GB) under the config-verified reproducible configuration, presentation in 103.34 min (2.2 GB) at $k=1$ and 136.82 min (2.9 GB) at $k=2$; Section 7). The zero-knowledge wrap of the credential relation is produced on consumer hardware as well: credential generation wraps end-to-end in 4.20 hours within the memory budget, peaking near 15 GiB, so the credential-generation proof, like the standalone verification, completes. The classical anonymity it then provides is conditional on the open key-privacy assumption (Assumption 6.3). Both relations moreover bind their public statement to the receipt, demonstrated end to end, and for issuance that binding and the zero-knowledge wrap combine in one measured proof (a bound CreGen wrap in 4.18 h; Section 7), so the deployable object is a statement-bound zero-knowledge credential rather than two

separate demonstrations. The presentation wrap completes and verifies at both $k = 1$ (11.76 hours end-to-end, plain wrap) and $k = 2$ (10.21 hours, statement-bound; the two are different configurations, not a matched k -series), each within the memory budget and peaking near 15 GiB (14.93 and 15.66 GiB). The core-proof shard count ($1516 \rightarrow 2011$) and peak memory grow with the credential count while staying within the 24 GB budget. What blocks post-quantum anonymity is separate, and independent of k . The only zero-knowledge wrap the toolchain ships is pairing-based (the second obstruction).

9.2 Answer to the Research Question

The narrower, mechanism-level question this thesis poses is *when a precompile pays*, and the answer is a trace-area inequality read against two baselines. Against a precompile’s own emulated baseline it pays exactly when the core trace area it removes exceeds the chip area it adds; by that feasibility test the Griffin chip pays, recovering a finite proof from an emulation stopped unfinished after five hours. Against the best available alternative it need not, and here it does not: the Griffin precompile proves slower than a software SHA-3 control that commits no chip area at all. The research question asks the larger thing, whether a post-quantum anonymous credential runs inside a zkVM without breaking its security, and its answer is *not yet*, for a reason more specific than “too slow.” Standalone PLUM verification becomes feasible on consumer hardware, at $\lambda = 80$ on SP1 under tuned memory settings and conditional on Assumption 6.1, once the precompile absorbs the field-mismatch tax. Deployable post-quantum *anonymity* is not reached, because it requires a zero-knowledge wrap that is at once feasible to generate within 24 GB and post-quantum-sound. The wrap the pipeline exposes is now feasible, running to completion within budget, but it is pairing-based and therefore not post-quantum-sound, for the reason of the previous subsection. The substrate choice is therefore governed by the deployment-lifecycle decision rule of Section 8: where the verification predicate is fixed and proofs are frequent, the specialised static circuit is preferable; where the predicate, signature, or security parameters evolve over the deployment’s lifetime, the zkVM’s fixed universal relation confines each change to a guest-program edit, provided the change leaves the precompiled primitive suite intact, since a hash-family or field-width change regenerates the Griffin AIR. Against a non-preprocessing baseline its advantage there is a smaller re-audit footprint, not a faster runtime.

9.3 Limitations

The results are bounded in ways we state explicitly.

Parameter level. Prove-mode wall-clock is measured at $\lambda = 80$, the largest level at which it is observable within the memory budget and not a deployment security level; the field-multiplication ablation is reported at $\lambda = 128$, while the Griffin-precompile cycle ratio ($58.5\times$) and the credential relation cycle counts are execute-mode figures at $\lambda = 80$.

The static baseline. The static baseline is Loquat over $\mathbb{F}_{p_{127}}$ rather than same-scheme PLUM, with the per-proof figure measured at zero knowledge disabled and the zero-knowledge-enabled counterpart proving in about 22 minutes (mean of three runs, 21.4–22.9) (Section 7), so the per-proof comparison is indicative and the lifecycle crossover is bounded in form but not located in wall-clock.

Run counts. The standalone PLUM Cell 2 and Cell 3 prove figures are means of five fixed-config runs (ranges reported); the credential-relation prove figures are single runs (Section 7).

Conditional security. The security guarantees are conditional: the constraint-level soundness of the Griffin AIR (Assumption 6.1), the existence of a PLUM signature-transcript simulator (Assumption 6.2), the cross-table lookup-argument binding (Assumption 6.6), PLUM signature key-privacy for issuance anonymity (Assumption 6.3), and the quantum-random-oracle lift (Assumption 6.4) are carried as the five essential open assumptions (the one further named premise of Section 6, the black-box-transfer of Assumption 6.5(iii), is inherited or standard), together with the multi-shard extractor conjecture (Conjecture 1) on which the composite receipt’s knowledge soundness rests, and with the canonical prime instantiation a separate premise (Remark 3.5). Of these, Assumption 6.1 conditions only the positive results, the 14.24-minute feasibility figure and the security-preservation theorem; the negative findings, including the surviving obstruction (the second), the SHA-3 inversion, the precompile-free baseline’s non-completion within the time budget, and the absence of a wall-clock flexibility advantage, would survive or strengthen under an unsound AIR (the argument, that an unsound AIR only understates the Griffin-arm cost, is in Section 6).

Statement binding and wrap run-counts. The wrap figures are single runs; that is the limitation this paragraph carries, and the positive record it bounds is as follows. The SP1 prototype binds the receipt journal to the public statement for both credential relations, demonstrated end-to-end by two cross-issuer issuances and a $k = 2$ presentation, each accepting with its committed statement recovered (Section 7); a single statement-bound and zero-knowledge-wrapped run is measured for credential generation, a bound CreGen wrap completing and verifying in 4.18 h at 15.2 GiB peak, essentially the plain wrap’s cost, and the $k = 2$ showing counterpart likewise, a statement-bound wrap completing and verifying in 10.21 h at a 15.66 GiB peak (Section 7). The wrap’s 24 GB-sufficiency is established by completed end-to-end runs of the PLUM-verification, credential-generation, and both the $k = 1$ and $k = 2$ presentation wraps, each peaking near 15 GiB (Section 7).

9.4 Future Work

The next measurement. The most informative next measurement is a *same-scheme* per-proof comparison, running one scheme on both substrates at matched security: a 199-bit $\mathbb{F}_p$ Aurora harness to run PLUM statically (the current harness is 127-bit only), fixing the sign of the per-proof crossover that the decision rule of Section 8 currently leaves as a conjecture.

Security of the SHA-3 substitution. The SHA-3 arm of Section 7 is a diagnostic control, and its security is deliberately unexamined: PLUM’s analysis covers its own algebraic-hash instantiation, and a bit-oriented hash is ruled out by cost in PLUM’s native circuit setting, where hash evaluations already account for $\approx 91\%$ of the verification constraints (Section 4), before any security question arises. On the zkVM substrate that cost objection disappears, so the question becomes live. Settling it would require re-specifying the encoding of field elements into a bit-oriented sponge, the challenge-expansion sampling, and the Merkle commitment instantiation, and re-deriving the soundness bounds of the underlying IOP under those choices. If the substituted scheme proves secure, the inversion of

Section 7 becomes deployment-relevant rather than diagnostic, and the case for algebraic hashes on this substrate narrows further. We leave the analysis as future work.

The open assumptions. The open assumptions are the substantive cryptographic work that remains: Assumption 6.1, whose per-row layer carries a written proof (Appendix B) and rejection-direction fault-injection coverage for seven of the ten constraint families, by extending the suite through a full-prover harness to the lookup-borne families (round count, memory binding, cross-row threading) and by machine-checking the code-to-constraint transcription, for instance by the method of Arguzz [44]; Assumption 6.2 by constructing a PLUM signature-transcript simulator; and the post-quantum-anonymity goal by a post-quantum-sound zero-knowledge wrap (a STARK- or lattice-based recursive argument) in place of the pairing-based wrap the toolchain currently exposes.

A post-quantum-anonymity path. The post-quantum-anonymity obstruction of Section 6.3 is largely an engineering gap plus one open lemma. The classical zero-knowledge the produced wrap already supplies can be made post-quantum without a pairing-based wrap, by masking the prover’s own low-degree test rather than wrapping its output: for a FRI/STARK-style prover by masked FRI [47], and for the multilinear Basefold-style commitment of the SP1 prover by the VEIL compiler [51], both plausibly post-quantum and neither reintroducing a quantum-vulnerable assumption. VEIL exists as code (it ships in the SP1 fork, as yet unwired), but wiring it is not a flag-flip. Its one-mask-per-commitment discipline collides with the repeated openings of the multi-shard SP1 prover, and the honest-verifier zero-knowledge of the masked low-degree test at the deployed instantiation, a per-round leaf-masking statement for STIR/WHIR at $\eta = 4$ over the 199-bit field, is an open lemma we scope but do not prove here, the per-round-committed-oracle analog of the single-quotient statement of masked FRI [47]. What remains is therefore the reconciliation of VEIL’s opening discipline with multi-shard proving, and then measuring whether the resulting post-quantum-sound prover stays within the 24 GB budget the pairing-based wrap already meets. Closing the obstruction this way would deliver *computational* post-quantum anonymity in the quantum random-oracle model. It would not deliver *everlasting* anonymity: for the hash-committed showings, everlasting anonymity cannot be established in the standard model (Theorem 6.10), and reaching it would require a statistically-hiding trace commitment, a separate direction we leave open.

A precompile generator. A further direction concerns the precompile construction itself. The three chips of Section 5 instantiate one interface by hand, and the cost criterion decides, per primitive, which of them is worth building. The natural generalisation is a generator that emits a sound precompile directly from a primitive’s specification: given the S-box degree, state width, round structure, and field of a large-prime algebraic primitive, it would produce the AIR together with a machine-checkable soundness obligation from a single meta-theorem, rather than writing and auditing each chip separately. This would turn the interface of Section 5.5 from a contract instantiated three times into a reusable construction method, and it would let the criterion drive an automated build-or-skip decision in the manner of the autoprecompile line [45], but over native large-field arithmetic rather than merged emulation. The soundness meta-theorem is the hard part, and we leave it as future work.

9.5 Closing

Post-quantum anonymous credential systems require both cryptographic soundness and operational evolvability, and this thesis measured the price of pursuing both on the same substrate. Its contribution is to map that frontier precisely. The precompile makes PLUM signature verification at the 80-bit level finite to prove on a personal computer. The deployment-lifecycle separation between the static and zkVM regimes is real, though it surfaces as a re-audit footprint and stays out of the wall-clock numbers. The dual obstruction to deployable post-quantum anonymity reduces to its single remaining obstruction, the second: the anonymity-providing wrap completes within the memory budget, but it is pairing-based and therefore classical rather than post-quantum, and no amount of additional memory, favourable timing, or resolution of Assumption 6.1 makes it post-quantum. That obstruction, named precisely and with its cost accounted for, is the main result this thesis leaves for the design of the next such system.

Acknowledgement

References

- [1] BBC News, “India students facing Canada removal over fake documents get reprieve,” 2023, accessed 2026-06-13. [Online]. Available: <https://www.bbc.com/news/world-us-canada-65899113>
- [2] Nixon Peabody LLP, “Reported LinkedIn data breach: 700 million users’ data exposed,” 2021, accessed 2026-06-13. [Online]. Available: <https://www.nixonpeabody.com/insights/articles/2021/06/30/reported-linkedin-data-breach-700-million-users-data-exposed>
- [3] National Institute of Standards and Technology, “Transition to post-quantum cryptography standards,” U.S. Department of Commerce, NIST Internal Report (Initial Public Draft) 8547 ipd, Nov. 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ir/2024/NIST.IR.8547.ipd.pdf>
- [4] H. Adkins, and S. Schmieg, “Quantum frontiers may be closer than they appear,” Google (The Keyword) security blog, 25 March 2026, 2026, sets 2029 as Google’s target for completing post-quantum migration, one year ahead of the NIST IR 8547 deprecation date. Accessed 2026-07-15. [Online]. Available: <https://blog.google/innovation-and-ai/technology/safety-security/cryptography-migration-timeline/>
- [5] Z. Z. Li, X. Zhang, H. Cui, J. Zhao, and X. Chen, “Bdec: Enhancing learning credibility via post-quantum digital credentials,” in *Provable and Practical Security: 18th International Conference, ProvSec 2024, Gold Coast, QLD, Australia, September 25–27, 2024, Proceedings, Part II*. Berlin, Heidelberg: Springer-Verlag, 2025, pp. 45–64. [Online]. Available: https://doi.org/10.1007/978-981-96-0957-4_3
- [6] X. Zhang, R. Steinfeld, M. F. Esgin, J. K. Liu, D. Liu, and S. Ruj, “Loquat: A SNARK-friendly post-quantum signature based on the legendre PRF with applications in ring and aggregate signatures,” Cryptology ePrint Archive, Paper 2024/868, 2024. [Online]. Available: <https://eprint.iacr.org/2024/868>
- [7] X. Zhang, Q. Fu, R. Steinfeld, J. K. Liu, T. H. Yuen, and M. H. Au, “Plum: SNARK-friendly post-quantum signature based on power residue PRFs,” in *Provable and Practical Security: 19th International Conference, ProvSec 2025, Yokohama, Japan, October 10–12, 2025, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2025, pp. 111–129. [Online]. Available: https://doi.org/10.1007/978-981-95-2961-2_6
- [8] Succinct Labs, “SP1 zkVM documentation,” <https://docs.succinct.xyz/docs/sp1/introduction>, 2024, accessed 2026-05-29.
- [9] L. Grassi, Y. Hao, C. Rechberger, M. Schofnegger, R. Walch, and Q. Wang, “Horst meets fluid-SPN: Griffin for zero-knowledge applications,” in *Advances in Cryptology – CRYPTO 2023, Part III*, ser. Lecture Notes in Computer Science, vol. 14083. Springer, 2023, pp. 573–606, full version: Cryptology ePrint Archive, Paper 2022/403, <https://eprint.iacr.org/2022/403>.
- [10] C. Jeudy, A. Roux-Langlois, and O. Sanders, “Lattice signature with efficient protocols, application to anonymous credentials,” in *Advances in Cryptology – CRYPTO 2023*, ser. Lecture Notes in Computer Science, vol. 14082. Springer, 2023, pp. 351–383, cryptology ePrint Archive, Paper 2022/509. [Online]. Available: <https://eprint.iacr.org/2022/509>

-
- [11] G.-V. Policharla, B. Westerbaan, A. Faz-Hernández, and C. A. Wood, “Post-quantum privacy pass via post-quantum anonymous credentials,” *Cryptology ePrint Archive*, Paper 2023/414, 2023, presented at Real World Crypto 2023 (non-archival). [Online]. Available: <https://eprint.iacr.org/2023/414>
- [12] T. Feneuil, and M. Rivain, “CAPSS: A framework for SNARK-friendly post-quantum signatures,” *Cryptology ePrint Archive*, Paper 2025/061, 2025. [Online]. Available: <https://eprint.iacr.org/2025/061>
- [13] D. Chaum, “Security without identification: Transaction systems to make big brother obsolete,” *Communications of the ACM*, vol. 28, no. 10, pp. 1030–1044, 1985. [Online]. Available: <https://doi.org/10.1145/4372.4373>
- [14] J. Camenisch, and A. Lysyanskaya, “Signature schemes and anonymous credentials from bilinear maps,” in *Advances in Cryptology – CRYPTO 2004*, ser. Lecture Notes in Computer Science, vol. 3152. Springer, 2004, pp. 56–72.
- [15] D. Boneh, X. Boyen, and H. Shacham, “Short group signatures,” in *Advances in Cryptology – CRYPTO 2004*, ser. Lecture Notes in Computer Science, vol. 3152. Springer, 2004, pp. 41–55, *cryptology ePrint Archive*, Paper 2004/174. [Online]. Available: <https://eprint.iacr.org/2004/174>
- [16] D. Pointcheval, and O. Sanders, “Short randomizable signatures,” in *Topics in Cryptology – CT-RSA 2016*, ser. Lecture Notes in Computer Science, vol. 9610. Springer, 2016, pp. 111–126, *cryptology ePrint Archive*, Paper 2015/525. [Online]. Available: <https://eprint.iacr.org/2015/525>
- [17] A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, “Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers,” in *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2019. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/coconut-threshold-issuance-selective-disclosure-credentials-with-applications-to-distributed-ledgers/>
- [18] World Wide Web Consortium, “Verifiable credentials data model v2.0,” W3C, W3C Recommendation, May 2025, w3C Recommendation, 15 May 2025. [Online]. Available: <https://www.w3.org/TR/vc-data-model-2.0/>
- [19] M. Rosenberg, J. White, C. Garman, and I. Miers, “zk-creds: Flexible anonymous credentials from zkSNARKs and existing identity infrastructure,” in *IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2023, iACR ePrint 2022/878.
- [20] J. Bootle, V. Lyubashevsky, N. K. Nguyen, and A. Sorniotti, “A framework for practical anonymous credentials from lattices,” in *Advances in Cryptology – CRYPTO 2023*, ser. LNCS, vol. 14082. Springer, 2023, pp. 384–417, iACR ePrint 2023/560.
- [21] S. Argo, T. Güneysu, C. Jeudy, G. Land, A. Roux-Langlois, and O. Sanders, “Practical post-quantum signatures for privacy,” in *ACM SIGSAC Conference on Computer and Communications Security (CCS ’24)*. ACM, 2024, pp. 1523–1537, full version: IACR ePrint 2024/131.

-
- [22] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” U.S. Department of Commerce, Federal Information Processing Standards Publication (FIPS) 204, 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.204.pdf>
- [23] —, “Stateless Hash-Based Digital Signature Standard,” U.S. Department of Commerce, Federal Information Processing Standards Publication (FIPS) 205, 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.205.pdf>
- [24] —, “FIPS 206: FN-DSA, a FALCON-based digital signature standard (forthcoming),” U.S. Department of Commerce, NIST, Federal Information Processing Standards Publication (in preparation), 2025, forthcoming. As of the writing, no public draft had been released: the draft was submitted for internal/Department of Commerce clearance on 28 August 2025 and an Initial Public Draft (IPD) had not yet been published. Status presented at the Sixth NIST PQC Standardization Conference, 25 September 2025. [Online]. Available: <https://csrc.nist.gov/pubs/fips/206>
- [25] M. Chase, D. Derler, S. Goldfeder, C. Orlandi, S. Ramacher, C. Rechberger, D. Slamanig, and G. Zaverucha, “Post-quantum zero-knowledge and signatures from symmetric-key primitives,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1825–1842, cryptology ePrint Archive, Report 2017/279. [Online]. Available: <https://eprint.iacr.org/2017/279>
- [26] C. Baum, C. D. de Saint Guilhem, D. Kales, E. Orsini, P. Scholl, and G. Zaverucha, “Banquet: Short and fast signatures from aes,” in *Public-Key Cryptography – PKC 2021*, ser. Lecture Notes in Computer Science, vol. 12710. Springer, 2021, pp. 266–297, cryptology ePrint Archive, Report 2021/068. [Online]. Available: <https://eprint.iacr.org/2021/068>
- [27] C. D. de Saint Guilhem, E. Orsini, and T. Tanguy, “Limbo: Efficient zero-knowledge mpcith-based arguments,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2021, pp. 3022–3036. [Online]. Available: <https://doi.org/10.1145/3460120.3484595>
- [28] J. Katz, V. Kolesnikov, and X. Wang, “Improved non-interactive zero knowledge with applications to post-quantum signatures,” in *ACM CCS 2018*, 2018, ePrint 2018/475.
- [29] W. Beullens, and C. D. de Saint Guilhem, “LegRoast: Efficient post-quantum signatures from the Legendre PRF,” in *Post-Quantum Cryptography (PQCrypto 2020)*, ser. LNCS, vol. 12100. Springer, 2020, pp. 130–150, ePrint 2020/128.
- [30] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev, “STIR: Reed–solomon proximity testing with fewer queries,” Cryptology ePrint Archive, Paper 2024/390, 2024. [Online]. Available: <https://eprint.iacr.org/2024/390>
- [31] X. Zhang, Z. Li, R. Steinfeld, R. K. Zhao, J. K. Liu, and T. H. Yuen, “Pegasus and PegaRing: Efficient (ring) signatures from sigma-protocols for power residue PRFs with (Q)ROM security,” Cryptology ePrint Archive, Paper 2025/1841, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1841>

-
- [32] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward, “Aurora: Transparent succinct arguments for R1CS,” Cryptology ePrint Archive, Paper 2018/828, 2018. [Online]. Available: <https://eprint.iacr.org/2018/828>
 - [33] A. Chiesa, D. Ojha, and N. Spooner, “Fractal: Post-quantum and transparent recursive proofs from holography,” Cryptology ePrint Archive, Paper 2019/1076, 2019. [Online]. Available: <https://eprint.iacr.org/2019/1076>
 - [34] A. Chiesa, Y. Hu, M. Maller, P. Mishra, P. Vesely, and N. P. Ward, “Marlin: Preprocessing zkSNARKs with universal and updatable SRS,” in *Advances in Cryptology – EUROCRYPT 2020*, ser. Lecture Notes in Computer Science, vol. 12105. Springer, 2020, pp. 738–768, cryptology ePrint Archive, Paper 2019/1047. [Online]. Available: <https://eprint.iacr.org/2019/1047>
 - [35] StarkWare, “S2morrow: STARK-based signature aggregation for Falcon and SPHINCS+,” Non-peer-reviewed engineering artifact, <https://github.com/starkware-bitcoin/s2morrow>, 2025, falcon-512 and SPHINCS+ verified in the Cairo zkVM.
 - [36] A. Saygan, M. S. Gündoğan, A. Arslan, and M. E. Gönen, “HAPPIER: Hash-based, aggregatable, practical post-quantum signatures implemented efficiently with Risc0,” in *Lightweight Cryptography for Security and Privacy (LightSec 2025)*, ser. LNCS, vol. 16216. Springer, 2026, pp. 3–22.
 - [37] Kota, “Building a zero-knowledge verifier for Dilithium signatures: NTT gadget implementation in SP1 zkVM,” Non-peer-reviewed engineering write-up, Medium, 2025-12-08, <https://medium.com/@phillyj1026/building-a-zero-knowledge-verifier-for-dilithium-signatures-ntt-gadget-implementation-in-sp1-6c50ab26283> 2025, mL-DSA verification in SP1 via a guest-code NTT gadget; proofs via the SP1 prover network.
 - [38] J. Bruestle, P. Gafni, and the RISC Zero Team, “RISC Zero zkVM: Scalable, Transparent Arguments of RISC-V Integrity,” RISC Zero, Draft, Aug. 2023, draft, August 11. [Online]. Available: <https://dev.risczero.com/proof-system-in-detail.pdf>
 - [39] A. Arun, S. Setty, and J. Thaler, “Jolt: SNARKs for virtual machines via lookups,” Cryptology ePrint Archive, Paper 2023/1217, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1217>
 - [40] OpenVM Contributors, “OpenVM: A performant and modular zkVM framework built for customization,” <https://github.com/openvm-org/openvm>, 2025, whitepaper and open-source framework; accessed 2026-07.
 - [41] N. Kaskov, M. Komarov, and M. Aksenov, “zkLLVM Circuit Compiler: A compiler from high-level programming languages into provable-computations protocol input representations,” =nil; Foundation, Technical Report, Apr. 2024, accessed April 6, 2024. [Online]. Available: https://cms.nil.foundation/uploads/zkllvm_a27f4af786.pdf
 - [42] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina, “Circom: A circuit description language for building zero-knowledge applications,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4733–4751, 2023.

-
- [43] Aztec Labs, “Noir: A domain-specific language for SNARK proving systems,” <https://noir-lang.org>, 2024, accessed 2026-07-13.
- [44] C. Hochrainer, V. Wüstholtz, and M. Christakis, “Arguzz: Testing zkVMs for soundness and completeness bugs,” arXiv preprint arXiv:2509.10819; to appear, USENIX Security 2026, 2025. [Online]. Available: <https://arxiv.org/abs/2509.10819>
- [45] powdr labs, “Automatic circuit acceleration of guest programs: Autoprecompiles,” powdr labs technical report (blog), 2025, <https://www.powdr.org/blog/auto-acc-circuits>; see also *powdr-OpenVM: end-to-end autoprecompiles* (2025) and *Accelerating Ethereum with Autoprecompiles* (2025).
- [46] I. Zhang, K. Kulkarni, T. Li, D. Wong, T. Kim, J. Guibas, U. Roy, B. Pellegrino, and R. Zarick, “vApps: Verifiable applications at internet scale,” arXiv preprint arXiv:2504.14809, 2025, <https://arxiv.org/abs/2504.14809>.
- [47] U. Haböck, and A. Kindi, “A note on adding zero-knowledge to STARKs,” Cryptology ePrint Archive, Paper 2024/1037, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1037>
- [48] RISC Zero, “RISC Zero zkVM: Notes on zero-knowledge,” Security advisory GHSA-5xgj-pmjj-gw49, 2024. [Online]. Available: <https://github.com/risc0/risc0/security/advisories/GHSA-5xgj-pmjj-gw49>
- [49] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Advances in Cryptology – EUROCRYPT 2016*, ser. Lecture Notes in Computer Science, vol. 9666. Springer, 2016, pp. 305–326, cryptology ePrint Archive, Paper 2016/260. [Online]. Available: <https://eprint.iacr.org/2016/260>
- [50] Succinct Labs, “Proof types — SP1 documentation,” SP1 Documentation, 2025, accessed 2026-06-01. [Online]. Available: <https://docs.succinct.xyz/docs/sp1/generating-proofs/proof-types>
- [51] R. Dalal, T. Hemo, E. Rabinovich, and R. Rothblum, “VEIL: Lightweight zero-knowledge for hash-based multilinear proof systems,” Cryptology ePrint Archive, Paper 2026/683, 2026. [Online]. Available: <https://eprint.iacr.org/2026/683>
- [52] T. Moran, and M. Naor, “Receipt-free universally-verifiable voting with everlasting privacy,” in *Advances in Cryptology – CRYPTO 2006*, ser. Lecture Notes in Computer Science, vol. 4117. Springer, 2006, pp. 373–392.
- [53] J. Müller-Quade, and D. Unruh, “Long-term security and universal composability,” *Journal of Cryptology*, vol. 23, no. 4, pp. 594–671, 2010, cryptology ePrint Archive, Paper 2006/422; preliminary version in TCC 2007.
- [54] R. Chairattana-Apirom, N. Döttling, A. Lysyanskaya, and S. Tessaro, “Everlasting anonymous rate-limited tokens,” Cryptology ePrint Archive, Paper 2025/1030 (ASIACRYPT 2025), 2025. [Online]. Available: <https://eprint.iacr.org/2025/1030>

-
- [55] D. Ott, C. Peikert *et al.*, “Identifying research challenges in post quantum cryptography migration and cryptographic agility,” arXiv:1909.07353 (Computing Community Consortium workshop report), 2019. [Online]. Available: <https://arxiv.org/abs/1909.07353>
- [56] Y. Yang, Y. Cheng, H. Tang, G. Yang, B. Zhang, and K. Ren, “SoK: Understanding zkVM: From research to practice,” Cryptology ePrint Archive, Paper 2026/525; to appear, ACM AsiaCCS 2026, 2026. [Online]. Available: <https://eprint.iacr.org/2026/525>
- [57] E. Ben-Sasson, A. Chiesa, and N. Spooner, “Interactive oracle proofs,” in *Theory of Cryptography – TCC 2016-B, Part II*, ser. Lecture Notes in Computer Science, vol. 9986. Springer, 2016, pp. 31–60, cryptology ePrint Archive, Paper 2016/116. [Online]. Available: <https://eprint.iacr.org/2016/116>
- [58] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Fast Reed–Solomon interactive oracle proofs of proximity,” in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, 2018.
- [59] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev, “WHIR: Reed–Solomon proximity testing with super-fast verification,” Cryptology ePrint Archive, Paper 2024/1586, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1586>
- [60] T. Dokchitser, and A. Bulkin, “Zero knowledge virtual machine step by step,” Cryptology ePrint Archive, Paper 2023/1032, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1032>
- [61] R. Lavin, X. Liu, H. Mohanty, L. Norman, G. Zaarour, and B. Krishnamachari, “A survey on the applications of zero-knowledge proofs,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.00243>
- [62] U. Haböck, D. Levit, and S. Papini, “Circle STARKs,” Cryptology ePrint Archive, Paper 2024/278, 2024. [Online]. Available: <https://eprint.iacr.org/2024/278>
- [63] Polygon Zero Team, “Plonky3: A toolkit for implementing polynomial IOPs,” <https://github.com/Plonky3/Plonky3>, 2024, accessed 2026-07-12.
- [64] L. Grassi, D. Khovratovich, and M. Schofnegger, “Poseidon2: A faster version of the poseidon hash function,” in *Progress in Cryptology – AFRICACRYPT 2023*, ser. Lecture Notes in Computer Science, vol. 14064. Springer Nature Switzerland, 2023, pp. 177–203, cryptology ePrint Archive, Paper 2023/323. [Online]. Available: <https://eprint.iacr.org/2023/323>
- [65] D. Khovratovich, “Key recovery attacks on the Legendre PRFs within the birthday bound,” Cryptology ePrint Archive, Paper 2019/862, 2019, <https://eprint.iacr.org/2019/862>.
- [66] A. May, and F. Zveydinger, “Legendre PRF (multiple) key attacks and the power of preprocessing,” in *2022 IEEE 35th Computer Security Foundations Symposium (CSF)*. IEEE, 2022, pp. 428–438, ePrint 2021/645.
- [67] S. Pailoor *et al.*, “Automated detection of under-constrained circuits in zero-knowledge proofs,” in *Proc. ACM Program. Lang. (PLDI 2023)*, 2023, doi 10.1145/3591282; tool QED2; full author list to be filled from the PLDI 2023 record.

-
- [68] M. Bellare, A. Boldyreva, A. Desai, and D. Pointcheval, “Key-privacy in public-key encryption,” in *Advances in Cryptology – ASIACRYPT 2001*, ser. LNCS, vol. 2248, 2001, pp. 566–582.
- [69] J. Bobolz, F. Eidens, S. Krenn, S. Ramacher, and K. Samelin, “Issuer-hiding attribute-based credentials,” in *Cryptology and Network Security (CANS 2021)*, ser. LNCS, 2021, pp. 158–178, ePrint 2022/213.
- [70] J. Don, S. Fehr, C. Majenz, and C. Schaffner, “Security of the Fiat-Shamir transformation in the quantum random-oracle model,” Cryptology ePrint Archive, Paper 2019/190, 2019. [Online]. Available: <https://eprint.iacr.org/2019/190>
- [71] A. B. Grilo, K. Hövelmanns, A. Hülsing, and C. Majenz, “Tight adaptive reprogramming in the QROM,” in *Advances in Cryptology – ASIACRYPT 2021*, 2021, ePrint 2020/1361.
- [72] G. Bertoni, J. Daemen, M. Peeters, and G. V. Assche, “On the indifferentiability of the sponge construction,” in *Advances in Cryptology – EUROCRYPT 2008*, ser. LNCS, vol. 4965, 2008, pp. 181–197.
- [73] W. van Dam, S. Hallgren, and L. Ip, “Quantum algorithms for some hidden shift problems,” *SIAM Journal on Computing*, vol. 36, no. 3, pp. 763–778, 2006, preliminary version in Proc. SODA 2003, pp. 489–498.
- [74] P. Frixons, and A. Schrottenloher, “Quantum security of the Legendre PRF,” *Transactions on Mathematical Cryptology*, vol. 1, no. 2, pp. 52–69, 2022, ePrint 2021/149 (2021); journal volume published 2022.
- [75] U. Haböck, “Multivariate lookups based on logarithmic derivatives,” Cryptology ePrint Archive, Paper 2022/1530, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1530>
- [76] W. Beullens, T. Beyne, A. Udovenko, and G. Vitto, “Cryptanalysis of the Legendre PRF and generalizations,” *IACR Transactions on Symmetric Cryptology*, vol. 2020, no. 1, pp. 313–330, 2020, ePrint 2019/1357.
- [77] A. Fiat, and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Advances in Cryptology – CRYPTO ’86*, ser. Lecture Notes in Computer Science, vol. 263. Springer, 1987, pp. 186–194.
- [78] S. Papini, and U. Haböck, “Improving logarithmic derivative lookups using GKR,” Cryptology ePrint Archive, Paper 2023/1284, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1284>
- [79] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty, and S. Saraf, “Proximity gaps for reed–solomon codes,” Cryptology ePrint Archive, Paper 2020/654, 2020. [Online]. Available: <https://eprint.iacr.org/2020/654>
- [80] U. Haböck, “Basefold in the list decoding regime,” Cryptology ePrint Archive, Paper 2024/1571, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1571>

-
- [81] B. E. Diamond, and A. Gruen, “Proximity gaps in interleaved codes,” Cryptology ePrint Archive, Paper 2024/1351, 2024, also in Communications in Cryptology, Vol. 1, No. 4. [Online]. Available: <https://eprint.iacr.org/2024/1351>
 - [82] H. Zeilberger, B. Chen, and B. Fisch, “BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes,” Cryptology ePrint Archive, Paper 2023/1705 (CRYPTO 2024), 2023. [Online]. Available: <https://eprint.iacr.org/2023/1705>
 - [83] E. Ben-Sasson, “ethSTARK documentation,” Cryptology ePrint Archive, Paper 2021/582, 2021. [Online]. Available: <https://eprint.iacr.org/2021/582>
 - [84] J. Avigad, L. Goldberg, D. Levit, Y. Seginer, and A. Titelman, “A verified algebraic representation of Cairo program execution,” in *Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, 2022, <https://arxiv.org/abs/2109.14534>.
 - [85] A. Bariant, A. Boeuf, A. Lemoine, I. M. Ayala, M. Øygarden, L. Perrin, and H. Raddum, “The algebraic FreeLunch: Efficient Gröbner basis attacks against arithmetization-oriented primitives,” Cryptology ePrint Archive, Paper 2024/347 (CRYPTO 2024), 2024. [Online]. Available: <https://eprint.iacr.org/2024/347>
 - [86] A. Bariant, A. Boeuf, P. Briaud, M. Hostettler, M. Øygarden, and H. Raddum, “Improved resultant attack against arithmetization-oriented primitives,” Cryptology ePrint Archive, Paper 2025/259 (CRYPTO 2025), 2025. [Online]. Available: <https://eprint.iacr.org/2025/259>
 - [87] A. Bak, A. Bariant, A. Boeuf, P. Briaud, M. Øygarden, and A. Phanse, “The algebraic CheapLunch: Extending FreeLunch attacks on arithmetization-oriented primitives beyond CICO-1,” Cryptology ePrint Archive, Paper 2025/2040, 2025. [Online]. Available: <https://eprint.iacr.org/2025/2040>
 - [88] A. Bak, A. Bariant, M. Hostettler, and V. Neiger, “Resultants meet resultant: Improving CICO-1 and CICO-2 attacks on ZK-friendly permutations,” Cryptology ePrint Archive, Paper 2026/1281, 2026. [Online]. Available: <https://eprint.iacr.org/2026/1281>
 - [89] Succinct Labs, “Security model — SP1 documentation,” SP1 Documentation, 2025, accessed 2026-06-01. [Online]. Available: <https://docs.succinct.xyz/docs/sp1/security/security-model>
 - [90] A. Chiesa, P. Manohar, and N. Spooner, “Succinct arguments in the quantum random oracle model,” Cryptology ePrint Archive, Paper 2019/834 (TCC 2019), 2019. [Online]. Available: <https://eprint.iacr.org/2019/834>
 - [91] A. Chiesa, Z. Di, Z. Hu, and Y. Zheng, “How to prove post-quantum security for succinct non-interactive reductions,” Cryptology ePrint Archive, Paper 2025/2166 (EUROCRYPT 2026), 2026, cryptology ePrint Archive, Paper 2025/2166. [Online]. Available: <https://eprint.iacr.org/2025/2166>
 - [92] E. Crites, and A. Stewart, “On Reed–Solomon proximity gaps conjectures,” Cryptology ePrint Archive, Paper 2025/2046, 2025. [Online]. Available: <https://eprint.iacr.org/2025/2046>

-
- [93] R. Chai, and X. Fan, “FRI soundness above the Johnson bound via threshold halving,” Cryptology ePrint Archive, Paper 2026/858, 2026. [Online]. Available: <https://eprint.iacr.org/2026/858>
 - [94] R. Canetti, O. Goldreich, and S. Halevi, “The random oracle methodology, revisited,” Journal of the ACM 51(4):557–594 (Cryptology ePrint Archive, Paper 1998/011), 2004. [Online]. Available: <https://eprint.iacr.org/1998/011>
 - [95] I. Haitner, J. J. Hoch, O. Reingold, and G. Segev, “Finding collisions in interactive protocols: A tight lower bound on the round complexity of statistically-hiding commitments,” FOCS 2007 (ECCC TR07-038), 2007. [Online]. Available: <https://eccc.weizmann.ac.il/report/2007/038/>
 - [96] H. Wee, “One-way permutations, interactive hashing and statistically hiding commitments,” Theory of Cryptography (TCC) 2007, LNCS 4392, pp. 419–433, 2007. [Online]. Available: <https://www.iacr.org/archive/tcc2007/43920418/43920418.pdf>
 - [97] C. Baum, I. Damgård, V. Lyubashevsky, S. Oechsner, and C. Peikert, “More efficient commitments from structured lattice assumptions,” Security and Cryptography for Networks (SCN) 2018 (Cryptology ePrint Archive, Paper 2016/997), 2018. [Online]. Available: <https://eprint.iacr.org/2016/997>
 - [98] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge,” Cryptology ePrint Archive, Paper 2019/953, 2019. [Online]. Available: <https://eprint.iacr.org/2019/953>
 - [99] Kardas, Kiraz, Savonin, Wang, and Dziadziuk, “Performance improvements of ZK-prover for rWasm: A sound and efficient AIR for 32-bit division and remainder,” Cryptology ePrint Archive, Paper 2025/2220, 2025. [Online]. Available: <https://eprint.iacr.org/2025/2220>

A Formal Development of the Security Preservation

The body (Section 6) states the security-preservation theorem (Theorem 6.5) and sketches its proof. This appendix gives the formal development for audit: the building-block properties the reduction treats as black boxes, each carrying an error term; the relation-preservation lemma; the step-by-step reductions; a corollary for the case in which the open premises are closed; and a remark relating the result to BDEC. Throughout, $\mathcal{A}$ denotes a classical probabilistic polynomial-time adversary and all probabilities are in the random-oracle model. The advantages $\text{Adv}_{\text{zkVM-Cred}}^\bullet$ are those of Definition 6.1.

A.1 Building-block properties

Each property is stated as the existence of an algorithm together with an error term. The premises of Theorem 6.5 assert the relevant errors negligible; we mark which properties are assumed and which carry an open premise.

Definition A.1 (Knowledge soundness). The zkVM argument Π is *knowledge-sound with knowledge error* ϵ_{KS} if there is a probabilistic polynomial-time extractor $\mathcal{E}$ with oracle access to the prover such that, for every prover P^* that makes $\Pi.\text{Verify}(x, \cdot)$ accept with probability ε , the extractor $\mathcal{E}^{P^*}(x)$ outputs a witness w with $R(x; w) = 1$ with probability at least $\varepsilon - \epsilon_{\text{KS}}$. This is the standard knowledge-soundness-with-error formulation; ϵ_{KS} is the additive extraction-failure term that enters the unforgeability bound (5), and the concrete soundness error of the STARK and BCS arguments we rely on is treated in §B.5. (Assumed; classical random-oracle model.)

Definition A.2 (Arithmetisation soundness defect). Define $\epsilon_{\text{AIR}} = 0$ if no trace satisfying the constraint set $\mathcal{C}$ (Definition B.2) commits a transition different from the reference permutation π of Definition B.1, and $\epsilon_{\text{AIR}} = 1$ otherwise. The quantity is Boolean rather than statistical: whether a fixed committed trace satisfies $\mathcal{C}$ is deterministic, no argument-verifier randomness entering, since the soundness of the argument that the committed trace satisfies $\mathcal{C}$ at all is accounted separately in the knowledge error ϵ_{KS} (Definition A.1), of which ϵ_{lookup} is one contributor. Assumption 6.1 asserts $\epsilon_{\text{AIR}} = 0$, and “ ϵ_{AIR} negligible” elsewhere in this thesis reads as that Boolean statement; the term is nonetheless carried symbolically, with its union-bound multiplicity N_G , in Equation (5), so that a future quantitative per-permutation bound on wrong-witness acceptance could replace the Boolean without restructuring the assembled bound. The supporting argument is Appendix B.

Definition A.3 (Signature-transcript simulatability). PLUM *admits a signature-transcript simulator with error* ϵ_{sZK} if there is a probabilistic polynomial-time algorithm $\mathcal{S}_\sigma$ such that, for every message m and public key, the output of $\mathcal{S}_\sigma$ run without the secret key is ϵ_{sZK} -indistinguishable from a credential or pseudonym transcript produced with the secret key. Assumption 6.2 asserts ϵ_{sZK} negligible. This assumption is *open*: PLUM’s analysis establishes EUF-CMA security, which does not by itself furnish such a simulator.

Definition A.4 (Zero knowledge). The zkVM argument Π is *zero-knowledge with error* ϵ_{ZK} if there is a probabilistic polynomial-time simulator $\mathcal{S}_\Pi$ such that, for every (x, w) with $R(x; w) = 1$, the simulated proof $\mathcal{S}_\Pi(x)$ is ϵ_{ZK} -indistinguishable from an honest proof $\Pi.\text{Prove}(x, w)$. (Assumed for the analysis. In the measured deployment this premise is met only by the ZK-wrapped modes, the `groth16()` and `plonk()` proof modes, which are pairing-based and hence not post-quantum; the default `prove` path has no established ZK simulator. Deployment status: Section 6.3.)

Definition A.5 (Existential unforgeability). PLUM is *EUFCMA secure with error* ϵ_{EUF} if every probabilistic polynomial-time forger, given the public key and a signing oracle, produces a valid signature on a message it never queried with probability at most ϵ_{EUF} . (Established in PLUM’s analysis, [7, Theorem 1], in the classical random-oracle model; its carry-over to the precompile-equipped guest is the black-box-transfer hypothesis, Assumption 6.5(iii).)

A.2 Relation preservation

Proof of Lemma 6.4. We account for the operations a guest evaluation invokes. The credential-generation relation $R_{\text{cre}}^{\text{Sig}}$, two PLUM verifications under pk_U , decomposes into large-field multiplications, returned identically by the modular-multiplication precompile (Lemma 5.1), and Griffin permutations inside PLUM’s hashing and inside every Merkle-path and low-degree-test recomputation, returned identically by the Griffin precompile under the lookup binding (Assumption 6.6), together with native control flow, whose faithful execution follows from the *soundness* of the zkVM argument for the RISC-V execution relation (an accepting proof certifies a valid execution); the stronger *knowledge-soundness* (Definition A.1) that extracts a witness is invoked separately, by the unforgeability reduction. For $R_{\text{cre}}^{\text{Sig}}$ this enumeration is complete at the predicate level, with the statement-binding refinement now in place (Section 7) and conditional only on Assumption 6.1. The presentation relation $R_{\text{show}}^{\text{Sig}}$ adds only $k + 2$ further signature-verification predicates built from the same multiplications and Griffin permutations; its ledger-membership, non-revocation, and disclosure checks are performed by the relying party outside the arithmetised relation. *Composition, by induction over the guest execution.* Fix the input; the guest evaluation is then a finite, deterministic sequence of operations $o_1, \dots, o_N$, each an instance of one of three classes: a large-field modular multiplication (returned identically by the modular-multiplication precompile, Lemma 5.1), a Griffin permutation (returned identically by the Griffin precompile under the lookup binding, Assumption 6.6, modulo Assumption 6.1), or a native RISC-V instruction (whose faithful execution follows from the *soundness* of the zkVM argument for the RISC-V execution relation). Let σ_i denote the machine state after o_i under the precompile-equipped execution and $\hat{\sigma}_i$ the state after o_i under the reference field-emulated execution, and take as invariant $I(i) : \sigma_i = \hat{\sigma}_i$. *Base case.* The input is fixed, so $\sigma_0 = \hat{\sigma}_0$. Operation o_1 belongs to one of the three classes and, applied to the common input state, returns the same output; hence $\sigma_1 = \hat{\sigma}_1$, that is $I(1)$. *Inductive step.* Assume $I(i - 1)$, i.e. $\sigma_{i-1} = \hat{\sigma}_{i-1}$. The operation o_i is a deterministic function of the state before it, of one of the three classes; by the per-operation equality for that class its output and its state transition agree under the two executions, so $\sigma_i = \hat{\sigma}_i$, that is $I(i)$. The step invokes one invariant class only, state equality, with no appeal to any other argument. *Conclusion.* By induction $I(N)$ holds, so $\sigma_N = \hat{\sigma}_N$. The terminating accept-or-reject bit is a fixed projection of the final state and therefore agrees under the two executions, yielding equality of the accept-or-reject decision at the program level: the precompile-equipped guest accepts $(x; w)$ if and only if $R^{\text{Sig}}(x; w) = 1$, using that the reference field-emulated baseline $\hat{\sigma}$ decides R^{Sig} by construction, the guest being written to evaluate R^{Sig} , so equality of the two decisions is equality with R^{Sig} . This program-level equality lifts to credential-verifier acceptance only when the zkVM journal commits the public statement x ; the specification assumes this binding, and its implementation status for both credential relations is recorded in Section 7. The Griffin step’s constraint-level soundness rests on Appendix B, which is argued rather than machine-verified, so the equality holds modulo Assumption 6.1. $\square$

A.3 Proof of the preservation theorem

We re-derive BDEC’s unforgeability (Theorem 1) and unlinkability (Theorem 3) reductions and the showing-anonymity bound (6) [5] with PLUM as the signature scheme Sig and the zkVM as the zk-SNARK Π ; issuance anonymity (BDEC Theorem 2) is treated separately in Section 6 under Assumption 6.3. By Lemma 6.4 the two provers accept the same relation.

Proof of Theorem 6.5, unforgeability. We recast the direct reduction as a labelled sequence of games G_0, G_1, G_2 on a single probability space, with success event $S_i = [G_i \text{ outputs } 1]$ and target probability 0. The adversary $\mathcal{A}$ plays the zkVM credential unforgeability experiment of Section 6 with the issuance and showing oracles and the honestly issued log $\mathcal{L}$; it wins by outputting an accepting (x^*, π^*) whose statement lies outside $\mathcal{L}$ and passes the experiment’s verifier-side conjuncts (journal binding, authorised teaching authority, ledger anchoring). Each hop below is a transition based on a failure event: we exhibit the event F , check the Difference-Lemma event identity $S_i \wedge \neg F \iff S_{i+1} \wedge \neg F$ explicitly on the common probability space, and bound the gap by $\Pr[F]$ (Shoup’s Difference Lemma).

Game G_0 . The credential unforgeability experiment verbatim, so $\Pr[S_0] = \text{Adv}_{\text{zkVM-Cred}}^{\text{unf}}(\mathcal{A})$.

Hop $G_0 \rightarrow G_1$: a transition based on a failure event (F_1 , extractor failure). G_1 runs exactly as G_0 and, on the accepting (x^*, π^*) , additionally invokes the knowledge extractor $\mathcal{E}$ of Definition A.1 to obtain a witness w^* for the credential relation the statement names, writing R^{Sig} for either relation as in Lemma 6.4: $w^* = (\text{pk}_U^*, \text{psk}^*)$ for $R_{\text{cre}}^{\text{Sig}}$, and the $(k+2)$ -conjunct witness $(\text{pk}_U^*, (\text{psk}^{(j)*})_j, \text{psk}_{U,V}^*, c_{U,V}^*)$ for $R_{\text{show}}^{\text{Sig}}$, the showing case running identically below with its $k+2$ conjuncts. Extraction is for the zkVM relation R_{vm} ; that the decoded witness satisfies R^{Sig} is the relation-preservation bridge (Lemma 6.4), whose hypotheses the theorem carries. G_1 sets $\text{bad}_1 \leftarrow \text{true}$ and aborts, outputting 0, if $\mathcal{E}$ returns no w^* with $R^{\text{Sig}}(x^*; w^*) = 1$. Let $F_1 = [\text{bad}_1 \text{ set}]$; F_1 is defined purely as extractor failure and fuses no other concern. On $\neg F_1$ no line of G_1 past the extraction differs from G_0 , so $S_0 \wedge \neg F_1 \iff S_1 \wedge \neg F_1$ and the Difference Lemma gives $|\Pr[S_0] - \Pr[S_1]| \leq \Pr[F_1]$. By Definition A.1, on an accepting instance the extractor fails with probability at most the knowledge error, so $\Pr[F_1] \leq \epsilon_{\text{KS}}$.

Hop $G_1 \rightarrow G_2$: a transition based on a failure event (F_2 , AIR mismatch across the verification’s Griffin permutations). A PLUM verification commits N_G Griffin transitions, one per permutation it invokes ($N_G \approx 1052$ at the measured parameters, Section 6); the accepting trace underlying π^* carries all of them. Define, purely over those committed transitions and independently of whether extraction succeeded, $F_2 = [\text{some one of the } N_G \text{ committed Griffin transitions differs from the reference permutation } \pi \text{ of Definition B.1}]$. Because F_2 is a property of the trace alone, evaluated on the post-extraction state, the hop needs one argument class only, arithmetisation soundness. G_2 is G_1 with $\text{bad}_2 \leftarrow \text{true}$ and abort whenever F_2 occurs, so $S_1 \wedge \neg F_2 \iff S_2 \wedge \neg F_2$ and the Difference Lemma gives $|\Pr[S_1] - \Pr[S_2]| \leq \Pr[F_2]$. By Definition A.2 a committed wrong Griffin transition exists only if the Boolean defect $\epsilon_{\text{AIR}} = 1$, so under the open Assumption 6.1 ($\epsilon_{\text{AIR}} = 0$) we have $\Pr[F_2] = 0$; the symbolic union bound over the N_G committed transitions, $\Pr[F_2] \leq N_G \epsilon_{\text{AIR}}$, holds in either case and is the form carried into the assembled bound. On $\neg F_2$ every committed transition equals π , so by relation preservation (Lemma 6.4) the extracted w^* is a genuine witness for $R^{\text{Sig}}(x^*; \cdot)$.

Game G_2 : embedding into PLUM EUF-CMA under a guessed honest key. In G_2 every surviving run carries an extracted witness w^* with $R^{\text{Sig}}(x^*; w^*) = 1$ on a fresh statement $(x^* \notin \mathcal{L})$ in the abbreviation of Section 6). The long-term key pk_U^* is prover-chosen (a witness component, Section 3), so we cannot assert $\text{pk}_U^* = \text{pk}$; the forger $\mathcal{B}$ instead guesses the honest key the forgery implicates. $\mathcal{B}$

receives $(\text{pp}_{\text{Sig}}, \text{pk})$ and a signing oracle $\mathcal{O}_{\text{Sign}}$ (recording queried messages in Q) from its EUF-CMA challenger (Definition A.5). At setup $\mathcal{B}$ samples i^* uniformly from the N honestly generated signing keys in the registry (the total honest signing keys, that is the honest users together with the authorised teaching authorities, Section 6) and embeds its challenge key there, $\text{pk}_{i^*} \leftarrow \text{pk}$, the honest-key embedding of BDEC’s unforgeability reduction, cf. the unforgeability experiment of Section 6 and [5, Appendix A.1]; it generates the other $N - 1$ keys itself, and runs $\mathcal{A}$:

- *Issuance queries.* For a query implicating key i^* , $\mathcal{B}$ obtains the required PLUM signatures (the pseudonym secrets psk and teaching-authority credentials, structurally PLUM signatures under pk_{i^*} , Section 3) by calling $\mathcal{O}_{\text{Sign}}$ on the corresponding messages; for every other key it signs with the keys it holds. It records each issued tuple in $\mathcal{L}$.
- *Showing queries.* $\mathcal{B}$ answers with the honest prover on the witnesses it holds; for showings of user i^* it obtains the showing-time signatures (the verifier-facing pseudonym binding and the shown credential) from $\mathcal{O}_{\text{Sign}}$ as well. Every message signed under pk , in answering issuance and showing queries for i^* , thus passes through $\mathcal{O}_{\text{Sign}}$ and is recorded by the EUF-CMA challenger in Q .

Since i^* is uniform and hidden from $\mathcal{A}$, $\mathcal{A}$ ’s view is distributed as in G_2 . On $\mathcal{A}$ ’s accepting (x^*, π^*) outside $\mathcal{L}$, $\mathcal{B}$ runs $\mathcal{E}$ to recover w^* ; on the surviving $(\neg F_1 \wedge \neg F_2)$ path the extracted signature σ^* (the conjunct selected by the freshness rule below: a pseudonym binding, the credential $c_{U,TA}$, or the shown credential $c_{U,V}$) is a valid PLUM signature under the honest user key pk_U^* that the forgery implicates, every conjunct of both credential relations verifying under the user’s key (Section 3), on the message m^* that x^* binds. Let E^* be the event that this implicated key equals the guessed embedding $\text{pk}_{i^*} = \text{pk}$. On E^* , $\mathcal{B}$ outputs (m^*, σ^*) . The experiment’s anchoring conjunct (Section 6) is what ties every surviving run to the registry: an anchored pseudonym appears in an accepted ledger record, created by $\mathcal{O}_{\text{Issue}}$ for a registered honest user under an authorised teaching authority, and BDEC’s key-binding argument [5, Appendix A.1], which ties the anchored pseudonym to the registered key its ledger record binds, excludes the residual case of an extracted witness verifying under a non-registered key; we inherit that step unchanged under Assumption 6.5(iii) and do not re-derive it here. The experiment’s freshness clause is statement-freshness $x^* \notin \mathcal{L}$; message-unqueriedness $m^* \notin Q$ is derived below inside the reduction, not assumed by the experiment.

Freshness $m^ \notin Q$, by conjunct selection.* $\mathcal{B}$ answers every signing need implicating $\text{pk}_{i^*} = \text{pk}$ through its oracle $\mathcal{O}_{\text{Sign}}$, so the messages honestly signed under pk are exactly the recorded set Q , which $\mathcal{B}$ holds. Suppose every conjunct message of x^* under the implicated key lay in Q . The recorded honest signatures on those messages would then assemble into a witness under the registered honest key pk satisfying $R^{\text{Sig}}(x^*; \cdot)$, built entirely from honestly produced material, so x^* would fail the experiment’s freshness clause (Section 6) and the run would not have survived. At least one conjunct message therefore lies outside Q ; $\mathcal{B}$ selects that conjunct and outputs its (m^*, σ^*) . The selection is well defined at the message level because $\text{Encodes}_{\text{Sig}}$ is deterministic and injective (Section 3), distinct protocol values encoding to distinct messages. This is the same freshness step as BDEC’s own EUF-CMA reduction ([5, Appendix A.1]) in its generic setting, and the black-box precompile substitution (Assumption 6.5(iii)) changes none of the reduction’s inputs, leaving the signature scheme, its EUF-CMA experiment, and the bookkeeping untouched. With $\text{Sig.Verify}(\text{pp}_{\text{Sig}}, \text{pk}, m^*, \sigma^*) = 1$ on E^* and $m^* \notin Q$, the pair (m^*, σ^*) makes $\text{Exp}_{\text{Sig}, \mathcal{B}}^{\text{euf-cma}}$ output 1, a forgery against the unmodified scheme.

Resources and advantage. $\mathcal{B}$ runs $\mathcal{A}$ once, answers each of its q issuance and showing queries with one $\mathcal{O}_{\text{Sign}}$ call and one prover invocation, and runs $\mathcal{E}$ once, so $t_{\mathcal{B}} = t_{\mathcal{A}} + t_{\mathcal{E}} + q \cdot t_{\text{prove}} + O(q)$ and $\mathcal{B}$ makes at most q signing queries. $\mathcal{B}$ wins whenever a run survives and its guess is correct. Every surviving run implicates a registered honest key, by the anchoring conjunct together with the inherited key-binding step above, and i^* is uniform over the N registered honest keys and hidden from $\mathcal{A}$'s view, so $\Pr[E^* \mid \text{surviving run}] \geq 1/N$ and $\text{Adv}_{\text{Sig}}^{\text{euf-cma}}(\mathcal{B}) \geq \frac{1}{N} \Pr[S_2]$, i.e. $\Pr[S_2] \leq N \epsilon_{\text{EUF}}$ by Definition A.5.

Summing the chain,

$$\text{Adv}_{\text{zkVM-Cred}}^{\text{unf}}(\mathcal{A}) = \Pr[S_0] \leq \Pr[S_2] + \Pr[F_1] + \Pr[F_2] \leq N \epsilon_{\text{EUF}} + \epsilon_{\text{KS}} + N_{\text{G}} \epsilon_{\text{AIR}},$$

which is (5). Each leaf is pointed at its source: ϵ_{KS} at Definition A.1, ϵ_{AIR} at the open Assumption 6.1, and ϵ_{EUF} at PLUM's EUF-CMA analysis (Definition A.5); the multiplicities N (the honest-key guess) and N_{G} (Griffin permutations per verification) are the assembly's polynomial factors, not bounds on any leaf. This appendix bounds none of the leaves, only their assembled sum. $\square$

Proof of Theorem 6.5, anonymity. We proceed through a labelled sequence $G_0 \rightarrow G_1 \rightarrow G_2$ on a single probability space over the challenge showing of cred_b , with success event $S_i = [b' = b]$ in G_i and target probability $\frac{1}{2}$; the anonymity advantage is $|\Pr[S_0] - \frac{1}{2}|$. The first hop is itself a *hybrid* over the $k + 2$ PLUM signature transcripts the challenge showing carries under pk_U .

Game G_0 . The anonymity experiment verbatim.

Hop $G_0 \rightarrow G_1$: a hybrid of $k + 2$ indistinguishability sub-hops (gap $\leq (k + 2) \epsilon_{\text{sZK}}$). The challenge showing evaluates $R_{\text{show}}^{\text{Sig}}$, whose witness carries $k + 2$ PLUM signature transcripts under pk_U (Section 3): the k teaching-authority pseudonym signatures $\text{psk}_{U,TA}^{(1)}, \dots, \text{psk}_{U,TA}^{(k)}$, the verifier-facing pseudonym signature $\text{psk}_{U,V}$, and the shown credential $c_{U,V}$. Enumerate them $\tau_1, \dots, \tau_{k+2}$. Define hybrids $G_0 = H_0, H_1, \dots, H_{k+2} = G_1$, where in H_i the transcripts $\tau_1, \dots, \tau_i$ are produced by the signature-transcript simulator $\mathcal{S}_{\sigma}$ (Definition A.3) without sk_U , and $\tau_{i+1}, \dots, \tau_{k+2}$ are still produced with sk_U ; the two adjacent hybrids differ only in slot i . For each $i \in \{1, \dots, k + 2\}$ we exhibit an interpolating distinguisher $\mathcal{D}_{\sigma}^{(i)}$: it receives a single transcript τ drawn from one of

- $\text{Real}_{\sigma}^{(i)}$: τ_i produced with sk_U on its message and public key;
- $\text{Sim}_{\sigma}^{(i)}$: the output of $\mathcal{S}_{\sigma}$ on the same message and public key, run without sk_U ;

produces slots $1, \dots, i - 1$ by $\mathcal{S}_{\sigma}$ and slots $i + 1, \dots, k + 2$ with sk_U , places τ in slot i , assembles the resulting witness, runs the common remainder of H_{i-1}/H_i (in which the proof is still produced honestly from the witness), obtains $\mathcal{A}$'s guess b' , and outputs $[b' = b]$. Then $\Pr[\mathcal{D}_{\sigma}^{(i)}(\text{Real}_{\sigma}^{(i)}) = 1] = \Pr[H_{i-1} \text{ succeeds}]$ and $\Pr[\mathcal{D}_{\sigma}^{(i)}(\text{Sim}_{\sigma}^{(i)}) = 1] = \Pr[H_i \text{ succeeds}]$, so each sub-hop gap is $\text{Adv}(\mathcal{D}_{\sigma}^{(i)}) \leq \epsilon_{\text{sZK}}$ by Definition A.3. Telescoping over the $k + 2$ sub-hops,

$$|\Pr[S_0] - \Pr[S_1]| \leq \sum_{i=1}^{k+2} |\Pr[H_{i-1} \text{ succeeds}] - \Pr[H_i \text{ succeeds}]| \leq (k + 2) \epsilon_{\text{sZK}}.$$

This is the load-bearing hop that produces the challenge showing's transcripts without the challenge user's sk_U , matching BDEC's anonymity simulator; its gap is governed by the open Assumption 6.2, scaled by the $k + 2$ transcripts.

Hop $G_1 \rightarrow G_2$: a single transition based on indistinguishability ($\text{gap} \leq \epsilon_{\text{ZK}}$). G_2 produces the one proof with the zkVM simulator $\mathcal{S}_\Pi$ (Definition A.4) rather than from the witness; the two games differ only in the proof. The interpolating distinguisher $\mathcal{D}_\Pi$ receives a proof string ϖ drawn from one of

- Real_Π : the honest proof $\Pi.\text{Prove}(\text{crs}, x_{\text{show}}, w)$ on the $\mathcal{S}_\sigma$ -simulated statement of G_1 ;
- Sim_Π : the simulated proof $\mathcal{S}_\Pi(x_{\text{show}})$;

hands (stmt, ϖ) to $\mathcal{A}$ in the common remainder of G_1/G_2 , and outputs $[b' = b]$. Then $|\Pr[S_1] - \Pr[S_2]| = \text{Adv}(\mathcal{D}_\Pi) \leq \epsilon_{\text{ZK}}$ by Definition A.4. There is one proof, so this hop enters once, with no factor of $k + 2$.

Game G_2 : *information-theoretic equalisation* ($\Pr[S_2] = \frac{1}{2}$). We show, self-contained inside G_2 , that the challenge statement $x_{\text{show}} = (A^\downarrow, (\text{ppk}_{U,TA}^{(j)})_{j=1}^k, \text{ppk}_{U,V})$ is distributed identically under $b = 0$ and $b = 1$. The disclosed attribute set $A^\downarrow$ is equalised across the two credentials by the well-formedness condition $\text{Leak}(x_0) = \text{Leak}(x_1)$. The verifier-facing pseudonym $\text{ppk}_{U,V}$ is freshly sampled at showing time, by the same pseudonym generation for either credential and independently of which is shown, so its distribution is the same regardless of b . The teaching-authority pseudonyms $(\text{ppk}_{U,TA}^{(j)})_j$ are issuance-time, credential-bound values that the well-formedness condition equalises across the two challenge credentials, since Leak includes the multiset $\{\text{ppk}_{U,TA}^{(j)}\}$ and the statement carries it in the holder's canonical record order, so multiset equality equalises the ordered component too (Section 6). The secret keys, the long-term key pk_U , and the $k + 2$ signature transcripts (the k teaching-authority pseudonym signatures, the verifier-facing pseudonym signature, and the shown credential $c_{U,V}$) are witness-only: each transcript was replaced by $\mathcal{S}_\sigma$ across the $G_0 \rightarrow G_1$ hybrid, and the whole witness was removed from the proof by $\mathcal{S}_\Pi$ in G_2 . Every b -dependent component being equalised, freshly sampled, or simulated, the challenge is independent of b , whence $\mathcal{A}$'s view is independent of b and $\Pr[S_2] = \Pr[b' = b] = \frac{1}{2}$.

Chaining,

$$\text{Adv}_{\text{zkVM-Cred}}^{\text{anon}}(\mathcal{A}) = \left| \Pr[S_0] - \frac{1}{2} \right| \leq |\Pr[S_0] - \Pr[S_1]| + |\Pr[S_1] - \Pr[S_2]| + \left| \Pr[S_2] - \frac{1}{2} \right| \leq (k+2) \epsilon_{\text{sZK}} + \epsilon_{\text{ZK}},$$

which is (6). Both segments are indistinguishability transitions and the terminal is exact; no failure-event hop appears, so ϵ_{KS} does not enter (no witness is extracted) and ϵ_{AIR} does not enter (the challenge showing is honestly generated). $\square$

Proof of Theorem 6.5, unlinkability. BDEC's unlinkability experiment ([5, §3.3], backing its Theorem 3) issues a *single* challenge credential $c^{(b)}$ under one of two challenger-internal pseudonyms $\text{ppk}^{(b)}$; the paper's challenge forwards only the raw credential, while its Theorem 3 proof (Appendix A.3) simulates the full showing transcript, and we adopt that transcript object as the challenge view: the challenger forwards the showing receipt produced from $c^{(b)}$, with the verifier-facing pseudonym sampled freshly as in the anonymity step, a strictly larger adversary view than the paper's game; the distinguisher $\mathcal{A}$ guesses b with advantage $|\Pr[b' = b] - \frac{1}{2}|$. We mirror the anonymity sequence on the one challenge transcript, the challenge bit now selecting the pseudonym key rather than the credential; success event $S_i = [b' = b]$, target probability $\frac{1}{2}$.

Game G_0 . The unlinkability experiment verbatim.

Hop $G_0 \rightarrow G_1$: a transition based on indistinguishability ($\text{gap} \leq \epsilon_{\text{sZK}}$). G_1 produces the single challenge pseudonym-binding transcript with $\mathcal{S}_\sigma$ in place of the secret key. The interpolating distinguisher $\mathcal{D}_\sigma$ receives one transcript τ from either Real_σ (the binding signature $\text{psk}^{(b)}$ on $\text{Encode}_{\text{Sig}}(\text{ppk}^{(b)})$ produced with sk_U) or Sim_σ ($\mathcal{S}_\sigma$'s output on the same message and public key, without sk_U), substitutes it for the challenge transcript, runs the common remainder of G_0/G_1 , and outputs $[b' = b]$. Applied *once* to the single challenge transcript, this gives $|\Pr[S_0] - \Pr[S_1]| \leq \epsilon_{\text{sZK}}$ by Definition A.3 (the open Assumption 6.2); there is one transcript, so no hybrid and no factor of $k + 2$.

Hop $G_1 \rightarrow G_2$: a transition based on indistinguishability ($\text{gap} \leq \epsilon_{\text{ZK}}$). G_2 produces the proof with $\mathcal{S}_\Pi$ in place of the witness. The interpolating distinguisher $\mathcal{D}_\Pi$ receives ϖ from either $\text{Real}_\Pi = \Pi.\text{Prove}(\text{crs}, x_{\text{show}}, w)$ or $\text{Sim}_\Pi = \mathcal{S}_\Pi(x_{\text{show}})$, hands (stmt, ϖ) to $\mathcal{A}$ in the common remainder, and outputs $[b' = b]$, giving $|\Pr[S_1] - \Pr[S_2]| \leq \epsilon_{\text{ZK}}$ by Definition A.4. Applied once to the single transcript, the two simulators each enter once: no factor of two.

Game G_2 : information-theoretic equalisation ($\Pr[S_2] = \frac{1}{2}$). Simulation alone does not close this game, because the challenge *statement* itself carries $\text{ppk}^{(b)}$, and Definition A.3's simulator runs on the same message and public key, removing secret-key dependence but not message dependence. The terminal is instead distributional. The receipt's statement $x_{\text{show}} = (A^\downarrow, \text{ppk}^{(b)}, \text{ppk}_{U,V})$ is identically distributed under $b = 0$ and $b = 1$: $A^\downarrow$ is common by construction, $\text{ppk}_{U,V}$ is freshly sampled independently of b , and $\text{ppk}^{(b)}$ is one of two identically and independently sampled pseudonyms, neither of which $\mathcal{A}$ has seen outside the challenge receipt (both are internal to the challenger, per the experiment of Section 6), so its marginal distribution is the same for either b . Beyond the statement, the challenge transcript is $\mathcal{S}_\sigma$'s output on the message $\text{Encode}_{\text{Sig}}(\text{ppk}^{(b)})$, a function of a statement component alone, and the proof is $\mathcal{S}_\Pi(x_{\text{show}})$; conditioned on the statement, neither carries further b -dependence. Every component of $\mathcal{A}$'s view being identically distributed, freshly sampled, or a function of the statement, the view is independent of b and $\Pr[S_2] = \Pr[b' = b] = \frac{1}{2}$. Had the challenger instead handed $\mathcal{A}$ the pseudonyms $\text{ppk}^0, \text{ppk}^1$, the statement would reveal b outright and the game would be trivially winnable; the withheld-pseudonym formulation is load-bearing.

Chaining the two hops,

$$\text{Adv}_{\text{zkVM-Cred}}^{\text{unlink}}(\mathcal{A}) = \left| \Pr[S_0] - \frac{1}{2} \right| \leq \epsilon_{\text{sZK}} + \epsilon_{\text{ZK}},$$

the single-transcript specialisation ($k + 2 \rightarrow 1$) of (6): the one challenge transcript incurs each simulation once, with no factor of $k + 2$ and no factor of two, and $\epsilon_{\text{KS}}, \epsilon_{\text{AIR}}$ do not enter. One difference of presentation from BDEC's A.3: there $c^{(b)}$ is a public-statement component, so its $\mathcal{S}_\sigma$ hop is forced, whereas in this thesis's relation $R_{\text{show}}^{\text{Sig}}$ the credential $c_{U,V}$ is a witness component (Section 3), so the $\mathcal{S}_\Pi$ hop already hides it and $\mathcal{S}_\sigma$ is needed only to produce $c^{(b)}$ without sk_U inside the reduction. Either way ϵ_{sZK} enters once. $\square$

Corollary A.6 (Preservation under closed assumptions). *Suppose Assumptions 6.1 and 6.2 hold, so that ϵ_{AIR} and ϵ_{sZK} are negligible; the cross-table lookup argument binds the precompile tables to the trace (Assumption 6.6) and PLUM's EUF-CMA reduction transfers black-box under the precompile substitution (Assumption 6.5(iii)); the zkVM is knowledge-sound and zero-knowledge with negligible $\epsilon_{\text{KS}}, \epsilon_{\text{ZK}}$; PLUM's signatures are key-private, so that issuance anonymity transfers (Assumption 6.3); and PLUM is EUF-CMA secure. Then the zkVM credential scheme is unforgeable, anonymous, and unlinkable. Against quantum adversaries the same conclusion holds once the quantum-random-oracle*

lifts of PLUM’s EUF-CMA reduction and of the nested Fiat–Shamir compilation are established, and provided the power-residue PRF is Q1 key-recovery-hard (Assumption 6.5(ii)) and Griffin admits its (quantum) random-oracle instantiation at the deployed parameters (Assumption 6.4).

Proof. Each bound of Theorem 6.5 is a sum of the named error terms with polynomial multiplicities $(N, N_G, k + 2)$, all polynomial in λ , so under the stated hypotheses every term is negligible; hence each advantage is negligible. The quantum clause adds the two open lifts, together with Assumptions 6.5(ii) and 6.4, as further hypotheses. $\square$

Remark A.7 (Relation to BDEC). Theorem 6.5 re-derives BDEC’s security for the concrete pair (PLUM, zkVM); the map to BDEC’s three theorems, proved for a generic (signature, zkSNARK) pair, is not one-to-one (Section 6): it develops unforgeability (Theorem 1), unlinkability (Theorem 3), and the showing-anonymity bound, while BDEC’s Theorem 2 (issuance anonymity) is re-instantiated in Section 6 under Assumption 6.3. The contribution is not a new reduction but the explicit accounting of the substrate-specific errors ϵ_{AIR} (the precompile’s arithmetisation soundness) and $\epsilon_{\text{KS}}, \epsilon_{\text{ZK}}$ (the zkVM’s argument), together with the identification of ϵ_{sZK} as the open Assumption 6.2 that BDEC’s generic anonymity argument silently requires of the signature. We further note that the relation $R_{\text{show}}^{\text{Sig}}$ places the credential $c_{U,V}$ in the witness rather than, as in BDEC’s literal layout, the public statement; the result is therefore a re-derivation of BDEC’s security for this concrete pair, not a verbatim re-instantiation of its three theorems.

B Evidence for Assumption 6.1: Specification-Level Soundness of the Griffin- $\mathbb{F}_p^{192}$ AIR

This appendix develops the soundness argument for the Griffin- $\mathbb{F}_p^{192}$ permutation AIR of Section 5, on which the conditional precompile-soundness theorem (Theorem 5.2) and the relation-preservation lemma of Section 6 rest. We follow the standard practice for arithmetisation correctness. As in ethSTARK [83] (Definitions 3 and 4), the verified Cairo representation [84], and per-gadget AIR-soundness arguments such as [99], the formal object is a constraint-soundness statement, a universally quantified implication over satisfying traces, together with a separate completeness statement, while the knowledge-soundness of the surrounding argument is treated apart in Section B.5. The malicious prover is captured by the quantifier “for every trace that satisfies the constraints”, so the statement uses no challenger, oracle, or advantage function. The one component supported empirically rather than proved, that the implemented chip realises the constraint set analysed here, appears as the explicit premise of the soundness theorem and is discussed in Section B.6. The binding of the chip’s range and multiset conditions is the cross-table lookup argument (Assumption 6.6), named as a hypothesis wherever a family rests on it.

B.1 Setup

Algorithm 2 Reference Griffin- π permutation over $\mathbb{F}_p^4$ at the deployed parameters

Require: state $s = (s_0, s_1, s_2, s_3) \in \mathbb{F}_p^4$; round constants $\{c^{(r)}\}_{r=1}^{14}$; MDS matrix M_4 ; quadratic coefficients (α, β) scaled per lane by Griffin’s index law

Ensure: $\pi(s) \in \mathbb{F}_p^4$

- 1: **for** $r = 1$ **to** 14 **do**
 - 2: $u_0 \leftarrow s_0^{1/3}$ $\triangleright$ inverse S-box; AIR witnesses the root u_0 and constrains $u_0^3 = s_0$, valid since $\gcd(3, p-1) = 1$
 - 3: $u_1 \leftarrow s_1^3$ $\triangleright$ forward S-box, degree 3
 - 4: $(u_2, u_3) \leftarrow Q_{\alpha, \beta}(u_0, u_1, s_2, s_3)$ $\triangleright$ quadratic layer; L_3 built from round-input lane s_2 per Griffin’s $i \geq 3$ index law (§3.2.2)
 - 5: $s \leftarrow M_4 \cdot (u_0, u_1, u_2, u_3)^\top + c^{(r)}$ $\triangleright$ linear layer, branch number 5, then round constants
 - 6: **end for**
 - 7: $\hookleftarrow s$, each lane reduced to canonical $[0, p)$ $\triangleright$ output canonicity is a per-lane AIR range check (§5.3)
-

Definition B.1 (Reference permutation). Let $\pi : \mathbb{F}_p^4 \rightarrow \mathbb{F}_p^4$ denote one application of the Griffin permutation at PLUM’s parameters, a degree-3 S-box over a width-4 state with 14 rounds and the SHAKE256-derived round constants and base quadratic coefficients (α, β) , scaled per lane by Griffin’s index law, fixed in Section 3, as computed by the reference implementation. The reference permutation is the published Griffin- π of [9] at these parameters: its nonlinear layer forms the lane-3 combination L_3 from the round-input lane x_2 (Griffin’s $i \geq 3$ index law), and its linear layer is the standard width-4 MDS matrix M_4 (branch number 5), both fixed in Section 3. The lemmas below establish chip-reference equality; since the reference is the published design, this is equality to the specification.

Definition B.2 (Algebraic satisfiability). The Griffin AIR is a finite constraint set $\mathcal{C}$ over a trace w comprising 14 per-round rows and one controller row. $\mathcal{C}$ comprises (i) the constraint polynomials Q_i , each with an enforcement domain H_i ; (ii) the range conditions on limb columns (operand, result, and carry limbs in $[0, 2^8)$, offset witness limbs in $[0, 2^{16})$); and (iii) the multiset-balance conditions of the lookup interactions (the byte-table sends, the round-to-round threading, and the memory binding). In the chip, (ii) and (iii) are realised by the byte-table and cross-table lookup arguments, whose soundness is Assumption 6.6; a prover for whom the lookup argument fails to bind can violate them while leaving every Q_i vanishing. A trace w satisfies $\mathcal{C}$ on input $s \in \mathbb{F}_p^4$ with claimed output $s' \in \mathbb{F}_p^4$ if its first-row input lanes decode to s , its last-row output lanes decode to s' , and all of (i)–(iii) hold under w .

B.2 Soundness

Theorem B.3 (Specification-level AIR soundness). *Suppose the implemented chip, comprising the per-round chip together with its sibling controller chip, enforces the constraint set $\mathcal{C}$ analysed below, and suppose the cross-table lookup argument binds (Assumption 6.6). Then for every input $s \in \mathbb{F}_p^4$ and every trace w satisfying $\mathcal{C}$ (Definition B.2) with claimed output s' , we have $s' = \pi(s)$. Equivalently, $\mathcal{C}$ admits no satisfying trace whose committed output differs from the reference permutation.*

Proof. By Lemmas B.4, B.5, and B.6. Lemma B.4 bridges the limb identities the chip actually constrains over its small base field to congruences modulo p ; Lemma B.5 composes those congruences, for each row, into the reference round on the row’s input lanes; and Lemma B.6 forces the 14 rows to compose as 14 sequential rounds on a single state, bound on the controller row to the syscall’s memory input and output. Composing the per-round equalities along that schedule yields $s' = \pi(s)$. The per-row layer (Lemmas B.4 and B.5) is unconditional given the range predicate, which the byte-table lookups discharge under Assumption 6.6; the cross-row layer (Lemma B.6) is lookup-borne (Assumption 6.6) in its scheduling, threading, and memory-binding steps (round count, memory binding, row scheduling), while output canonicity is per-row algebra and parameter immutability holds by preprocessing. The status of the chip–constraint equality (Assumption 6.1) is discussed in Section B.6. $\square$

B.3 Per-family lemmas

The ten constraint families group into the round function (the round-function families) and the schedule, binding, and canonicity apparatus (the schedule-and-binding families). For the test-coverage census these ten split as seven per-row families, reachable by `debug_constraints`, and three lookup-borne families (round count, memory binding, and the cross-row threading of row scheduling, whose per-row selectors are already among the seven); parameter immutability is preprocessing-enforced, with no prover column, and is counted in neither. Row scheduling thus spans both groups while parameter immutability sits in neither, so the two offset and the distinct total is ten. The round-function families admit a proof at the row level, given below. It is useful to read it adversarially: the prover chooses every cell of a row, all 3,819 columns (Section 7), subject only to the constraint polynomials and the range checks, and the lemmas state that no choice decodes to an input–output pair outside the graph of the reference round. The proof requires no computational assumption beyond the lookup binding (Assumption 6.6) that discharges the range predicate. Of the assumption-bearing steps in the chip’s soundness, the round-to-round threading and the memory binding are collected in Lemma B.6; the byte-table range lookups enter as the range-predicate hypothesis of Lemmas B.4 and B.5; all are carried by Assumption 6.6.

The constraints are not enforced over $\mathbb{F}_p$ directly: the chip constrains 32-limb byte representations over the prover’s base field, and the bridge from limb identities over the small field to congruences modulo the 199-bit p is itself a proof obligation. We discharge it first. Throughout, q denotes the prover’s base-field modulus (KoalaBear, $q = 2^{31} - 2^{24} + 1 = 2,130,706,433$, in the deployed SP1 fork), $\text{int}(\cdot)$ decodes a limb vector as an integer in base 2^8 , and the *range predicate* of a field-operation cell asserts that its operand, result, and carry limbs lie in $[0, 2^8)$ and its offset witness limbs in $[0, 2^{16})$. In the chip these bounds are enforced by byte-table lookups, so the range predicate is exactly the part of a cell’s soundness carried by Assumption 6.6. The limb gadget below is internal to the Griffin chip; it is distinct from the standalone Fp192 modular-multiplication precompile (Lemma 5.1), whose soundness rests on the host modular-multiplication table’s upstream product-and-range guarantee rather than on the byte-limb identity analysed here.

Lemma B.4 (Limb-gadget soundness). *Consider one field-operation cell for $\circ \in \{+, \times\}$ with operand limb vectors a, b , result limbs c , carry limbs h , and witness limbs w stored with offset 2^{14} , all satisfying the range predicate. If the cell’s vanishing identity*

$$p_{\text{op}}(x) - p_c(x) - p_h(x) p_{\bar{p}}(x) = (p_w(x) - 2^{14} \mathbb{K}(x)) (x - 2^8)$$

holds coefficient-wise over $\mathbb{F}_q$, where $p_{\text{op}} = p_a p_b$ for $\times$ and $p_a + p_b$ for $+$, $p_{\bar{p}}$ is the limb polynomial of the Griffin modulus p , and $\mathbb{K}$ is the all-ones polynomial, then

$$\text{int}(a) \circ \text{int}(b) = \text{int}(c) + \text{int}(h) \cdot p \quad \text{over } \mathbb{Z}, \quad \text{hence} \quad \text{int}(c) \equiv \text{int}(a) \circ \text{int}(b) \pmod{p}.$$

The same conclusion holds for the multi-operand linear cells used by the linear layer, whose p_{op} is a fixed integer combination of the four operand polynomials with row-sum at most 16 (the standard M_4 rows).

Proof. Under the range predicate, each coefficient of the difference of the two sides is an integer of absolute value at most

$$\underbrace{32 \cdot 255^2}_{p_a p_b} + \underbrace{255}_{p_c} + \underbrace{32 \cdot 255^2}_{p_h p_{\bar{p}}} + \underbrace{(2^{16} - 2^{14})(2^8 + 1)}_{\text{offset witness, two shifts}} = 16,793,919 < q,$$

where 32 bounds the overlap of two 32-limb polynomials; the additive and linear-combination cases are smaller. An integer that vanishes in $\mathbb{F}_q$ and has absolute value below q is zero, so the identity holds in $\mathbb{Z}[x]$. Evaluating at $x = 2^8$ annihilates the right-hand side and evaluates each limb polynomial to its integer, giving $\text{int}(a) \circ \text{int}(b) - \text{int}(c) - \text{int}(h)p = 0$. $\square$

The lemma concludes a congruence, not canonicity: $\text{int}(c)$ is bounded by 2^{256} through its byte limbs but not by p , and the chip indeed admits non-canonical representatives in intermediate cells. Canonicity is restored where output canonicity and the hardening checks on the nonlinear-output cells impose it, which the next lemma makes explicit.

Lemma B.5 (Per-row round soundness: the round-function and canonicity families). *Fix a real row with round index $r \in \{0, \dots, 13\}$, input-lane limbs $s = (s_0, s_1, s_2, s_3)$, and claimed output-lane limbs $s' = (s'_0, \dots, s'_3)$. Suppose the row satisfies, over $\mathbb{F}_q$, the per-row constraints of the round-function and canonicity families and the per-row row-scheduling selectors, and that every cell satisfies the range predicate. Then each $\text{int}(s'_\ell)$ is the canonical representative of lane ℓ of $\rho_r(s \bmod p)$, where ρ_r is the reference Griffin round with constants $\text{RC}[4r], \dots, \text{RC}[4r+3]$ (zero for $r = 13$), and the $\mathbb{F}_p$ -classes of all intermediate result cells are determined by s .*

Proof. Selectors. The per-row row-scheduling constraints make each is_round_r boolean with $\sum_r \text{is_round}_r = 1$ and $\sum_r r \cdot \text{is_round}_r = \text{round_idx}$ on a real row, so exactly one selector is set, at position r , and the round-constant mux evaluates to the byte-bounded limbs of $\text{RC}[4r+\ell]$.

Forward S-box (lane 1). Lemma B.4 applied to the squaring cell and then the cube cell gives $\text{int}(\text{cube}) \equiv s_1^3 \pmod{p}$; the bitwise binding of the lane-1 nonlinear output to cube transfers this congruence to the lane.

Inverse S-box (lane 0). The chip witnesses the nonlinear output s_0^{nl} and checks the backward cube: two applications of Lemma B.4 give $\text{int}(\text{cube}_0) \equiv (s_0^{\text{nl}})^3 \pmod{p}$, and the bitwise binding $\text{cube}_0 = s_0$ yields $(s_0^{\text{nl}})^3 \equiv s_0 \pmod{p}$. Since $\gcd(3, p-1) = 1$ (verified for the deployed modulus in §5.3), cubing is a bijection on $\mathbb{F}_p$, so the class of s_0^{nl} is the unique cube root $s_0^{1/3}$, the reference inverse S-box.

Quadratic layer (lanes 2, 3). The composite relation $s_i^{\text{nl}} = s_i(L_i^2 + \alpha_{i-2}L_i + \beta_{i-2})$ is enforced as a chain of binary cells (sums building L_i , its square, the α -multiple, the shifted sum, and the final product). Each link satisfies the range predicate, since every cell's result limbs are byte-checked, so Lemma B.4 applies link by link and the congruences compose to $s_i^{\text{nl}} \equiv s_i(L_i^2 + \alpha_{i-2}L_i + \beta_{i-2})$

(mod p) with $L_2 = s_0^{\text{nl}} + s_1^{\text{nl}}$ and $L_3 = 2s_0^{\text{nl}} + s_1^{\text{nl}} + s_2$, whose third summand is the round-*input* lane s_2 (Griffin’s $i \geq 3$ index law), not the updated lane, and with the index law $\alpha_0 = \alpha$, $\beta_0 = \beta$, $\alpha_1 = 2\alpha$, $\beta_1 = 4\beta$ baked into the constant operands; the base pair (α, β) and the 52 round constants are read from SHAKE256 seeded by the domain string `PlumGriffin( $p, 4, 2, 128$ )` and enter the constraints as preprocessing-bound constants (parameter immutability), not prover columns.

Linear layer. The four linear cells, by the multi-operand case of Lemma B.4, give $\text{mds}_j \equiv \langle (M_4)_j, s^{\text{nl}} \rangle \pmod{p}$, where $(M_4)_j$ is row j of the standard Griffin width-4 MDS matrix M_4 with rows $(5, 7, 1, 3)$, $(4, 6, 1, 1)$, $(1, 3, 5, 7)$, $(1, 1, 4, 6)$ (branch number 5), as fixed in `build_matrix`.

Round constants. The four addition cells give $\text{int}(s'_\ell) \equiv \text{mds}_\ell + \text{RC}[4r+\ell] \pmod{p}$, the mux having been fixed by the selector step; the preprocessing table holds 56 entries of which the final round’s 4 are zero, so the uniform constraint covers all 14 rounds and degenerates to the bare linear layer on the last.

Canonicity. The bitwise less-than cells enforce $\text{int}(s'_\ell) < p$, so each output lane is the canonical representative of its class.

Composing the five layers, the class of every intermediate result cell is determined by s (lane 0 by bijectivity of cubing, the remaining cells as images of deterministic operations; the carry and witness columns vary with the representatives the prover chooses), and $\text{int}(s'_\ell)$ is the canonical representative of lane ℓ of $\rho_r(s \bmod p)$. $\square$

Lemma B.6 (Composition, binding, canonicity: the schedule-and-binding families). *Under the schedule-and-binding families and Assumption 6.6, every trace satisfying $\mathcal{C}$ realises its 14 rows as the 14 sequential reference rounds on a single state bound to the syscall’s memory input and output, with canonical output lanes.*

Proof. Round count. The schedule is fixed to 14 rounds by the trace-generation row count and the controller’s lookup anchors at directions 0 and `NB_ROUNDS`, rather than by a dedicated algebraic constraint; under Assumption 6.6 a trace with a different round count produces an unbalanced lookup and has no satisfying assignment. *Memory binding.* The controller chip constrains the input and output state lanes to the values at the syscall’s memory addresses and clock, a memory-consistency lookup constraint resolved under Assumption 6.6, so a trace reading or writing a different address or clock produces an unbalanced lookup. *Output canonicity.* Each output lane s'_j carries the range constraint $0 \leq s'_j < p$ enforced by a byte-wise less-than comparison (`FieldLtCols`); this is the per-row step already proved within Lemma B.5 and is restated here only for the family inventory. *Row scheduling.* The real/padding and first/last-round selector flags enforce one ordered run with cross-row state threading $\text{state_out}_r = \text{state_in}_{r+1}$, realised by the per-row lookup interactions (a `RECEIVE` at direction r and a `SEND` at direction $r+1$) under Assumption 6.6; a ghost or mis-ordered row creates an unbalanced lookup. *Parameter immutability.* The modulus, round count, linear-layer entries, round constants, and quadratic coefficients are preprocessing-bound rather than prover-supplied, so a per-row choice of any of them has no satisfying assignment.

By row scheduling, and given Assumption 6.6, the per-round equalities of Lemma B.5 chain into 14 sequential rounds on one state; with the round count fixed by the lookup schedule (round count, Assumption 6.6) and with output canonicity and parameter immutability, this is the reference permutation, and with memory binding it is bound to the syscall’s memory input and output. $\square$

B.4 Completeness

Lemma B.7 (Completeness). *For every input $s \in \mathbb{F}_p^4$, the honest Griffin execution on s yields a trace satisfying $\mathcal{C}$ with output $\pi(s)$. This direction is empirically supported by the smoke-prove and fault-injection positive controls, not proved by construction.*

Operational discharge. Following the practice of [84], where soundness is the load-bearing direction and completeness is discharged operationally, we support the claim by the tests that evaluate the constraints on an honest trace: the single-permutation smoke proof that completes execute, prove, and verify (Section 5), and the fault-injection suite’s positive control, in which the uncorrupted satisfying trace yields zero failing rows under `debug_constraints` (Section B.6). The 256-vector randomised suite and the hand-picked and zero/high-entropy vectors pin the trace generator’s underlying executor against the reference computation and do not themselves evaluate the constraints. $\square$

B.5 The argument layer

Theorem B.3 concerns the *arithmetisation*. The surrounding claim, that an accepting receipt implies a satisfying trace and that a witness is extractable, is the *knowledge-soundness* of the zkVM’s STARK, a separate object we do not re-establish. We take it in the IOP and extractor sense of ethSTARK [83] (Definition 4): for every prover producing an accepting proof with non-negligible probability there exists an extractor outputting a satisfying trace, up to a negligible knowledge error. This is the knowledge-soundness hypothesis of the preservation theorem of Section 6, itself obtained through the Fiat–Shamir and BCS compilation of an interactive oracle proof. The arithmetisation soundness of this appendix and the argument knowledge-soundness are independent objects. End-to-end soundness needs both, and we keep them separate.

B.6 The open step

The compositional argument of Theorem B.3 is unconditional given Lemmas B.4, B.5, and B.6, the lookup binding (Assumption 6.6), and the premise that the implemented chip enforces at least the analysed constraint set $\mathcal{C}$ (the chip carries further hardening checks beyond $\mathcal{C}$, which only shrink the satisfying set). That premise, the fidelity of the code-to-constraint transcription, is the step we do not machine-verify. The evidence for it is the cross-codebase agreement of the chip’s native compute with the reference permutation on 256 random vectors, with per-vector false-pass at most $1/p \approx 2^{-199}$ and aggregate at most 2^{-191} across the suite, together with the hand-picked and zero/high-entropy vectors. This per-vector bound assumes a wrong executor produces outputs uniform over $\mathbb{F}_p$ and independent of the reference; a deterministic structural bug agreeing on every sampled input would not be caught, so 2^{-191} bounds only random disagreement on the sampled set and is *not* a bound on the open constraint-level premise. The suite pins the *executor* against the reference, but the *constraint system* checked against that executor is what the vector suite does not reach. A separate fault-injection suite reaches the constraint system in the *rejection* direction for seven of the ten families (the per-row families): a single-cell corruption of a satisfying trace is rejected by the per-row constraints in each case, including the standard- M_4 linear-layer lanes 1 and 3 (rows (4, 6, 1, 1) and (1, 1, 4, 6)) and the lane-2-to-lane-3 dependency that enforces L_3 ’s round-input wiring. This evidence is itself bounded, and we state the bounds. It is evaluated through `debug_constraints`, which checks the per-row

algebraic constraints but not the cross-table lookups, so the three lookup-borne families (round count, memory binding, and the cross-row threading of row scheduling), whose binding is Assumption 6.6, lie outside its reach; a separate lookup-balance harness (`debug_interactions`) reaches them by computing the net multiplicity of the Griffin cross-table lookup, a nonzero value being exactly what the logarithmic-derivative argument the verifier checks rejects: a short round chain (round count) and a broken cross-row state thread (row scheduling) are rejected, and memory binding is rejected for the syscall-level state handoff, its global-memory-consistency half still needing the full machine. Parameter immutability is enforced by construction, with no prover-chosen column to reject. The suite thus shows that specific local deviations are caught, not that no wrong witness satisfies $\mathcal{C}$, which remains the open premise. Closing it fully requires deriving the AIR witness directly from the reference code and checking, ideally by machine, that $\mathcal{C}$ admits only that witness. We therefore report Theorem B.3 as *proved* at the specification level for the per-row layer (Lemmas B.4 and B.5, conditional on the range predicate that the byte-table lookups discharge under Assumption 6.6), argued for the lookup-borne layer (Lemma B.6) and, for that layer, fault-tested in the rejection direction (round count and cross-row threading; memory binding for the syscall handoff), and supported empirically at the implementation level; what Assumption 6.1 still carries is the code-to-constraint transcription fidelity together with the lookup-borne families (Assumption 6.6), for both this theorem and the conditional precompile-soundness theorem Theorem 5.2.